

TW128: A Tree-Structured Duplex Authenticated-Encryption Mode

Coda Hale

Hale Institute for Bicycle Appreciation

Abstract. We present TW128, a nonce-based authenticated-encryption mode with associated data built as a two-level tree of keyed duplexes over the KECCAK- p [1600, 12] permutation. A serial root duplex binds the key, nonce, and associated data, encrypts the first message chunk, and aggregates a tag from every remaining chunk into a single authentication tag; the remaining chunks are handled by independent leaf duplexes that share no state once the key and nonce are fixed and can therefore be evaluated in parallel across SIMD lanes.

We analyze TW128 in the public-permutation model, proving each bound first against an intermediate random oracle and then transferring it to the real permutation by the flat-sponge lift. We obtain concrete bounds for all-in-one authenticated encryption, multi-user indistinguishability, and s -way committing security in the strongest full-input sense (CMTDs-4), obtained directly rather than through a generic transform. At the implemented parameters all three notions meet a 128-bit security target. Vectorized `amd64` and `arm64` implementations realize the tree’s leaf parallelism, reaching multi-gigabyte-per-second throughput within a small factor of hardware-accelerated AES-128-GCM.

Keywords: authenticated encryption · committing security · sponge constructions · duplex constructions · Keccak

1 Introduction

Authenticated encryption with associated data is the workhorse of applied cryptography, and the sponge and duplex constructions over a single public permutation offer an attractive route to it. The duplex of [BDPVA11] turns one permutation—here KECCAK- p [1600, 12], the round-reduced permutation underlying the [Nat15] standard—into a single-pass authenticated cipher with a small, constant memory footprint and no separate block cipher or universal-hash key schedule. Its security rests on the indistinguishability of the sponge from a random oracle [BDPVA08], which reduces analysis of the full construction to a random-oracle argument plus one capacity-birthday loss.

This compactness comes at a structural cost. A duplex is inherently *sequential*: each call absorbs its input into a state produced by the previous call, so a single duplex cannot exploit the wide SIMD and multicore parallelism of modern processors, on which competing designs built from AES reach many bytes per cycle. A second, orthogonal gap is *commitment*. Widely deployed AEADs such as AES-GCM and ChaCha20-Poly1305 are not key-committing: an adversary can craft a single ciphertext that decrypts to valid plaintexts under two or more keys [DGRW18, ADG⁺22]. Far from a curiosity, this enables concrete abuses—the “invisible salamander” that defeats Facebook’s attachment-franking scheme [DGRW18], ciphertexts that open to different well-formed files under different keys [ADG⁺22], and partitioning-oracle attacks that recover passwords from systems built on non-committing AEAD [LGR21]. The committing-AE line that addresses these threats was initiated by [GLR17] and developed into formal committing notions with efficient

generic transforms by [BH22]. A mode that targets those notions natively, rather than through an add-on transform, avoids the transform’s overhead and binds commitment to the same permutation it already uses for confidentiality and integrity.

TW128 addresses both gaps with a two-level tree of keyed duplexes. A serial *root* duplex, initialized with the key K and nonce N , absorbs the associated data, encrypts the first message chunk, and absorbs a tag from every other chunk before emitting the final authentication tag. The remaining chunks are processed by independent *leaf* duplexes, each keyed by (K, N, j) and operating on disjoint state and disjoint input. Because the leaves share nothing once (K, N) is fixed, they can be evaluated in parallel across SIMD lanes, recovering the throughput a flat duplex forgoes; and because the root binds the associated data, the first chunk, and every leaf tag into the single output tag, that tag is a sponge image of the entire (K, N, A, M) input. The structure ensures only that distinct (K, N, A) triples reach distinct root transcripts; the committing security itself rests on the indistinguishability of the sponge, which makes those transcripts’ tags collision-resistant (Section 8). Every duplex still operates on a bounded 1600-bit state, so the memory footprint remains constant in the message length.

We analyze TW128 in the public-permutation model and obtain concrete bounds for three notions: all-in-one authenticated encryption in the sense of [RS06], its multi-user indistinguishability lift in the sense of [BT16], and s -way committing security in the strongest full-input sense (CMTDs-4) of [BH22]. Each bound is proved first against an intermediate random oracle and then transferred to the real permutation by the flat-sponge lift of [BDPVA08]. At the implemented parameters ($k = c = \tau_{\text{root}} = \tau_{\text{leaf}} = 256$), all three notions meet a 128-bit security target for adversaries running up to $N_{\text{tot}} \leq 2^{63}$ KECCAK- p calls—on the order of 2^{71} bytes of processed data, beyond any realistic deployment lifetime. Optimized `amd64` and `arm64` implementations turn the tree’s leaf parallelism into measured throughput within a small factor of hardware-accelerated AES-128-GCM (Section 9).

1.1 Our Contributions

- **The TW128 mode.** We specify TW128, a nonce-based AEAD with associated data built as a two-level tree of strict [BDPVA11] duplexes over KECCAK- p [1600, 12] (Section 3): a serial root duplex that binds (K, N, A) , the first chunk, and the leaf tags into a single authentication tag, over a set of parallel, independently keyed leaf duplexes. All phases are domain-separated within one permutation, and the state is constant-size regardless of message length.
- **All-in-one AE security.** We prove a concrete bound on the all-in-one AE advantage [RS06] in the public-permutation model (Theorem 1), obtained by composing random-oracle privacy and INT-CTXT bounds through the generic-composition argument of [BN00] and lifting the result to the real permutation.
- **Multi-user security.** We give a concrete multi-user indistinguishability bound [BT16] for D users (Theorem 2), obtained by a key-collision-stopped hybrid over the users that reuses the single-user privacy and INT-CTXT envelopes.
- **Committing security.** We prove that TW128 achieves s -way CMTDs-4 committing security—commitment to the full (K, N, A, M) input, for any number $s \geq 2$ of claimed openings—with a concrete root-collision bound (Theorem 3), and we derive the remaining [BH22] committing notions (CMT and CMTD for $\ell \in \{1, 3, 4\}$) as direct bounds (Section C). Committing security is native to the mode, with no generic transform.
- **Proof methodology.** We carry out the analysis in a dedicated TW128 random-oracle model and transfer every bound to the public-permutation setting through

the [BDPVA08] flat-sponge lift (Section B). A derived-adversary argument shows that each direct primitive query induces at most one random-oracle query, so the random-oracle envelopes are instantiated at the direct-query count N_{prim} rather than the total work N_{tot} , separating the single capacity-birthday loss $\text{Lift}_c(N_{\text{tot}})$ from the scheme-specific terms.

- **Optimized implementations.** We describe and evaluate vectorized amd64 (AVX-512 and AVX2) and arm64 (NEON with the SHA-3 extension) implementations whose batched leaf cores realize the tree’s parallelism in practice, reaching multi-gigabyte-per-second throughput on contemporary hardware (Section 9).

1.2 Related Work

Sponge and duplex AEADs. Permutation-based authenticated encryption built on the sponge and duplex constructions of [BDPVA08, BDPVA11] is well established. Keyak [BDP⁺15] is a CAESAR candidate built on the duplex over round-reduced KECCAK- p , whose Motorist mode already admits parallel duplex instances. Ascon [DEMS21], the lightweight winner of the CAESAR competition and the basis of the NIST lightweight-cryptography standard, and Xoodyak [DHP⁺20], a finalist in that process whose Cyclist mode is a deliberately serial simplification of Motorist, are duplex AEADs over the Ascon and Xoodoo permutations. TW128 shares this single-permutation, constant-state duplex foundation but differs in structure: rather than one duplex session, it is a two-level tree whose leaves are *independently keyed* duplexes under domain-separated chunk identifiers. Like Keyak’s Motorist mode, the tree admits data-parallel evaluation across SIMD lanes; the designs differ in where the degree of parallelism lives. Keyak fixes it as a mode parameter—each named instance sets the number Π of parallel duplex instances across which a message is spread, so Π is part of the wire format and must be agreed by the encryptor and decryptor in advance. In TW128 the leaf decomposition is fixed by the mode, but the number of leaves evaluated simultaneously is not part of the transcript: an implementation chooses it at run time to match the host’s vector width, and the ciphertext and tag are identical however it is chosen, so a wide-SIMD encryptor and a scalar decryptor interoperate.

Tree hashing. The two-level shape is inspired by tree hashing over the same permutation, in particular KangarooTwelve [BDP⁺16], which splits its input into fixed-size chunks hashed independently under KECCAK- p [1600, 12] and absorbs their chaining values into a root node. TW128 reuses the round count and the final-node/leaf split of that design and adapts it to authenticated encryption: each leaf chunk is encrypted and tagged independently, and the leaf tags are absorbed by the root in place of chaining values. It also inherits the short-message behavior of KangarooTwelve’s *kangaroo hopping*, which merges the first chunk with the hashing of the remaining chunks’ chaining values so that an input fitting in a single chunk incurs no tree overhead: in TW128 a single-chunk message engages no leaf duplexes and no aggregation phase, the root processing it directly and emitting the tag as a plain keyed duplex would. Short messages therefore pay none of the tree’s cost and retain the latency profile of an unstructured duplex (Section 9.4); the leaf layer, and its parallelism, engage only once a message spans more than one chunk.

Committing AEAD. The study of committing authenticated encryption was initiated by [GLR17], who introduced compactly committing AEAD for message franking, and by [DGRW18], who built it from a dedicated primitive (encryptment); [BH22] subsequently organized the notions into the CMT-1/3/4 and CMTD hierarchy and gave efficient generic transforms (UtC, RtC, HtE) that add commitment to a base scheme. A complementary line shows that widely used and dedicated AEADs fail these notions: the invisible-salamander

[DGRW18] and partitioning-oracle [LGR21] attacks, multi-format ciphertexts [ADG⁺22], the context-discovery and commitment attacks that break CCM, EAX, SIV, GCM, and OCB3 [MLGR23], and an $O(1)$ key-committing attack against all variants of AEGIS [WP14] that refutes the prior belief that AEGIS was committing [IR23]. TW128 instead targets the strongest full-input notion, CMTDs-4, directly: its committing security follows from the indistinguishability of the sponge, rather than from a generic transform applied to a non-committing base scheme.

1.3 Paper Organization

Section 2 recalls the notation, sponge and duplex constructions over KECCAK- p , the authenticated-encryption and committing-security [BH22] notions used throughout, and the multi-user indistinguishability notion of [BT16]. Section 3 specifies the TW128 construction and its parameters. Section 4 fixes the public-permutation games, the intermediate random-oracle model, and the resource accounting that all later bounds reference. Section 5 states the headline AE, mu-ind, and CMTDs-4 theorems and their concrete TW128-parameter forms; Sections 6 and 7 prove the random-oracle privacy and INT-CTXT bounds that feed those theorems, and Section 8 proves the random-oracle CMTDs-3 bound. Section 9 reports the performance of the reference and optimized implementations. Section 10 compares TW128 to prior sponge AEADs and committing constructions and discusses limitations. Section 11 concludes. Section A collects the structural decoding and accounting lemmas that form the proof core; Section B carries out the [BDPVA08] flat-sponge lift that converts each random-oracle bound to its public-permutation form; Section C gives the remaining [BH22] committing-security bounds; and Section D lists TW128 test vectors.

2 Preliminaries

2.1 Notation

Strings. Write $\{0, 1\}$ for the binary alphabet. For an integer $n \geq 0$, $\{0, 1\}^n$ denotes the set of n -bit strings, $\{0, 1\}^* = \bigcup_{n \geq 0} \{0, 1\}^n$ the set of all finite-length bit strings, and ε the empty string. For $x \in \{0, 1\}^*$, $|x|$ is the bit length and $x \parallel y$ is the concatenation; the bitwise XOR of two equal-length strings $x, y \in \{0, 1\}^n$ is $x \oplus y$. A *byte string* is an element of $(\{0, 1\}^8)^*$; its byte length is denoted $\|x\| = |x|/8$. For an integer $0 \leq n < 2^{64}$, $\langle n \rangle_8 \in \{0, 1\}^{64}$ is its eight-byte little-endian encoding. For a finite or infinite bit string x and $0 \leq \tau \leq |x|$ when x is finite, $[x]_\tau \in \{0, 1\}^\tau$ denotes the leading τ bits of x .

Sampling and probability. We write $x \leftarrow \$ S$ for sampling x uniformly at random from a finite set S , with implicit coins independent across samples. For an event E in a probability experiment, $\Pr[E]$ denotes its probability.

Sets of maps. For a set S , $\text{Perm}(S)$ denotes the set of permutations on S ; for a positive integer b , $\text{Perm}(\{0, 1\}^b)$ is the set of b -bit permutations.

Adversaries and games. An adversary $\mathcal{A}$ is a (possibly randomized) algorithm. A *game* G is an experiment that samples its private randomness, runs $\mathcal{A}$ with access to a specified set of oracles $O_1, \dots, O_k$, and returns a Boolean outcome. We write $\mathcal{A}^{O_1, \dots, O_k}$ for the adversary's interface and $\Pr[G^{\mathcal{A}} \Rightarrow 1]$ for the probability that G returns 1.

Advantages. For an indistinguishability-style experiment with two games G_0, G_1 , the distinguishing advantage of $\mathcal{A}$ against a scheme Π in security notion X is the absolute

182 difference

$$183 \quad \text{Adv}_{\Pi}^X(\mathcal{A}) = |\Pr[G_0^{\mathcal{A}} \Rightarrow 1] - \Pr[G_1^{\mathcal{A}} \Rightarrow 1]|.$$

184 For a one-shot or win-event experiment without a hidden challenge bit, $\text{Adv}_{\Pi}^X(\mathcal{A}) =$
 185 $\Pr[G^{\mathcal{A}} \text{ wins}]$, where “ $G^{\mathcal{A}}$ wins” denotes the event that the game returns 1. A bit-guessing
 186 experiment with a hidden challenge bit instead uses the rescaled form $|2\Pr[G^{\mathcal{A}} \text{ wins}] - 1|$,
 187 as in the multi-user notion of Section 2.4.

188 **Concrete security.** All bounds in this paper are concrete: each advantage is an explicit
 189 function of the resource caps introduced in Section 4.3. We make no use of asymptotic or
 190 negligibility conventions.

191 **Reference summary.** Table 1 summarizes the resource caps and loss terms used through-
 192 out. The caps are defined formally in Section 4.3, the loss terms in Section 5.2.

Table 1: Resource caps and loss terms.

Symbol	Meaning	Defined
N	Construction-interface cost (KECCAK- p calls)	§ 4.3
N_{prim}	Direct primitive-interface queries	§ 4.3
N_{tot}	$N + N_{\text{prim}}$ (total query-sequence cost)	§ 4.3
q_v, Q_v	Valid verification queries / cap (INT-CTXT)	§ 4.3
$q_{\text{RO}}, Q_{\text{RO}}$	Direct random-oracle queries / cap (RO-model)	§ 4.3
$n_{\text{leaf}}, N_{\text{leaf}}$	Distinct construction-generated final leaf transcripts / cap	§ 4.3
$\text{Lift}_c(N_{\text{tot}})$	$N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$, [BDPVA08] flat-sponge loss	§ 5.2
$\text{KeyGuess}_k(Q)$	$Q/2^k$, ideal-system key guess	§ 5.2
$\text{RootColl}_{\tau}(Q, s)$	$\binom{Q+s}{s}/2^{\tau(s-1)}$, s -way root-collision	§ 5.2

193 2.2 Sponges and Duplexes over Keccak- p

194 **The Keccak- p permutation.** The KECCAK- p family of permutations, introduced as part
 195 of the Keccak design and standardized within FIPS 202 [Nat15], is parameterized by a
 196 state width b and a round count n_r ; we write $\text{KECCAK} - p[b, n_r] \in \text{Perm}(\{0, 1\}^b)$ for the
 197 corresponding permutation. TW128 uses $\text{KECCAK} - p[1600, 12]$, the same primitive as
 198 KangarooTwelve [BDP⁺16], fixed throughout. In the public-permutation security model of
 199 Section 4.1 and the flat-sponge lift of Section B, this permutation is modeled as a uniformly
 200 random element of $\text{Perm}(\{0, 1\}^{1600})$.

201 **The sponge construction.** Following [BDPVA08], fix a permutation $\pi \in \text{Perm}(\{0, 1\}^b)$,
 202 a *sponge-compliant* padding rule pad , and a rate $1 \leq r < b$ with capacity $c = b - r$. A
 203 padding rule appends $\text{pad}[r](|M|)$, a bit string determined by the input length and the
 204 rate, to an input M , extending it to a sequence of r -bit blocks; it is sponge-compliant
 205 [BDPVA11, Definition 1] if $M \parallel \text{pad}[r](|M|)$ is never empty and, for all $M \neq M'$ and all
 206 $n \geq 0$, $M \parallel \text{pad}[r](|M|) \neq M' \parallel \text{pad}[r](|M'|) \parallel 0^{nr}$. The sponge function $\text{Sponge}[\pi, \text{pad}, r] :$
 207 $\{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$ maps an input M and an output length ℓ to an ℓ -bit string by
 208 absorbing $M \parallel \text{pad}[r](|M|)$ into the all-zero b -bit state in r -bit blocks (each separated by an
 209 application of π) and squeezing ℓ bits in r -bit blocks from the resulting state. [BDPVA08,
 210 Theorem 2] shows that $\text{Sponge}[\pi, \text{pad}10^*, r]$, where the simple rule $\text{pad}10^*$ appends a single
 211 1 bit followed by the minimum number of 0 bits reaching a multiple of r , is indistinguishable
 212 from a random oracle when π is uniformly random. We bound its distinguishing advantage
 213 by $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$, derived in Section 5.2 from the exact differentiating
 214 probabilities of [BDPVA08, Lemmas 4 and 5].

The duplex construction. The *duplex* construction of [BDPVA11] extends the sponge to a stateful interface. A duplex object $\text{Duplex}[\pi, \text{pad}, r]$ is initialized to the all-zero state. Each call $\text{duplexing}(\sigma, \ell)$, with $\ell \leq r$, absorbs an input string σ of at most $\rho_{\max}(\text{pad}, r)$ bits via π and returns the first ℓ bits of the resulting state. Here $\rho_{\max}(\text{pad}, r) = \min\{x : x + |\text{pad}[r](x)| > r\} - 1$ is the largest input length for which the padded input fits in a single r -bit block; for the pad10^*1 rule, $\rho_{\max}(\text{pad10}^*1, r) = r - 2$ [BDPVA11]. [BDPVA11, Lemma 3] establishes the duplex-to-sponge equality: for any call sequence $((\sigma_0, \ell_0), \dots, (\sigma_i, \ell_i))$, the i -th output equals

$$\text{Sponge}[\pi, \text{pad}, r](\sigma_0 \parallel \text{pad}_0 \parallel \dots \parallel \sigma_i, \ell_i),$$

where $\text{pad}_j = \text{pad}[r](|\sigma_j|)$ is the padding the duplex appends in its j -th call; the flattened input carries these interior paddings explicitly, and the sponge appends only the final pad_i . The flattening map is injective by [BDPVA11, Lemma 4]: the duplex-call sequence is recoverable from the flattened sponge input.

TW128 instantiation. TW128 fixes the padding rule pad10^*1 and the rate $r = 1344$, so the capacity is $c = 256$ bits. The largest input length per duplex call is therefore $\rho_{\max}(\text{pad10}^*1, 1344) = 1342$ bits.

Bridge to the H -input domain. The indistinguishability theorem of [BDPVA08] is stated for the simpler pad10^* padding over its unpadded H -input domain, while TW128 uses pad10^*1 internally. [BDPVA11, Theorem 3] supplies an injective preprocessing bridge $I[r, r_{\max}]$ between the two domains. For TW128, $r = r_{\max} = 1344$, so the bridge reduces to appending $1 \parallel 0^q$ with q determined by the input length modulo $r_{\max}$; on the bridge image, q is recovered as the trailing-zero count. Section 4.2 instantiates this bridge on typed TW128 duplex transcript prefixes as the map $\text{flat}(\cdot)$ used by the random-oracle proofs of Sections C and 6 to 8.

2.3 Authenticated Encryption with Associated Data

Syntax. A *nonce-based authenticated-encryption scheme with associated data* (AEAD scheme) is a pair $\text{SE} = (\text{Enc}, \text{Dec})$ of deterministic algorithms with associated key space $\mathcal{K} \subseteq (\{0, 1\}^8)^*$, nonce space $\mathcal{N} \subseteq (\{0, 1\}^8)^*$, associated-data space $\mathcal{A} \subseteq (\{0, 1\}^8)^*$, message space $\mathcal{M} \subseteq (\{0, 1\}^8)^*$, and ciphertext space $\mathcal{C} \subseteq (\{0, 1\}^8)^*$. The encryption algorithm $\text{Enc} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M} \rightarrow \mathcal{C}$ produces a ciphertext $C = \text{Enc}_K(N, A, M)$. The decryption algorithm $\text{Dec} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{C} \rightarrow \mathcal{M} \cup \{\perp\}$ returns either a plaintext or the rejection symbol $\perp$. The *tag length in bytes* is the constant $\|\text{Enc}_K(N, A, M)\| - \|M\|$, fixed by the scheme. In this generic syntax, C denotes the full AEAD ciphertext, including any tag material. The TW128 algorithm section (Section 3) writes this same value as $C \parallel T$, with C the ciphertext body and T the root tag.

Correctness. For every $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$, $\text{Dec}_K(N, A, \text{Enc}_K(N, A, M)) = M$.

Privacy (IND-\$\text{CPA}\$). The privacy game samples $K \leftarrow \mathcal{K}$ and gives the adversary $\mathcal{A}$ either the real encryption oracle $\text{Enc}_K(\cdot, \cdot, \cdot)$ or the random-output oracle Rand that, on every query (N, A, M) , returns a fresh uniformly random byte string of length $\|\text{Enc}_K(N, A, M)\| = \|M\| + \tau/8$ where τ is the scheme's tag length in bits. The adversary is *nonce-respecting*: distinct encryption queries must use distinct nonces. Define

$$\text{Adv}_{\text{SE}}^{\text{priv}}(\mathcal{A}) = |\Pr[\mathcal{A}^{\text{Enc}_K} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Rand}} \Rightarrow 1]|.$$

Ciphertext integrity (INT-CTXT). The INT-CTXT game samples $K \leftarrow \$ \mathcal{K}$ and gives $\mathcal{A}$ access to Enc_K and a verification oracle Ver_K that returns 1 if $\text{Dec}_K(N, A, C) \neq \perp$ and 0 otherwise. Each encryption query records the resulting tuple (N, A, C) . The adversary wins if some verification query returns 1 on a tuple (N, A, C) not previously recorded by the encryption oracle. No nonce uniqueness is imposed on either oracle. Define $\text{Adv}_{\text{SE}}^{\text{int-ctxt}}(\mathcal{A})$ as the probability that $\mathcal{A}$ wins.

All-in-one AE. The all-in-one authenticated-encryption notion of [RS06], instantiated for nonce-based AE following [BT16], combines privacy and integrity into a single experiment. The challenger samples $K \leftarrow \$ \mathcal{K}$ and grants $\mathcal{A}$ either the pair $(\text{Enc}_K, \text{Ver}_K)$ or the ideal pair $(\text{Rand}, \text{Replay})$. In the ideal pair, Rand returns a fresh uniformly random byte string of length $\|\text{Enc}_K(N, A, M)\|$ on each accepted query (N, A, M) and records the resulting tuple (N, A, C) in a set W ; $\text{Replay}_W(N, A, C)$ returns 1 if $(N, A, C) \in W$ and 0 otherwise. The adversary is nonce-respecting on encryption queries. Define

$$\text{Adv}_{\text{SE}}^{\text{ae}}(\mathcal{A}) = |\Pr[\mathcal{A}^{\text{Enc}_K, \text{Ver}_K} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Rand}, \text{Replay}} \Rightarrow 1]|.$$

Lemma 1 (Bellare–Namprempre Composition). *For every adversary $\mathcal{A}$ against the all-in-one AE security of SE, there exist adversaries $\mathcal{A}_{\text{priv}}$ and $\mathcal{A}_{\text{int-ctxt}}$ with the same resources as $\mathcal{A}$ such that*

$$\text{Adv}_{\text{SE}}^{\text{ae}}(\mathcal{A}) \leq \text{Adv}_{\text{SE}}^{\text{priv}}(\mathcal{A}_{\text{priv}}) + \text{Adv}_{\text{SE}}^{\text{int-ctxt}}(\mathcal{A}_{\text{int-ctxt}}).$$

Lemma 1 is the all-in-one form of the privacy-plus-integrity composition theorem of [BN00]. The standard proof inserts the hybrid $(\text{Enc}_K, \text{Replay})$, where Replay accepts exactly the tuples recorded by the real encryption oracle. The hop from $(\text{Enc}_K, \text{Ver}_K)$ to $(\text{Enc}_K, \text{Replay})$ is bounded by INT-CTXT: the two interfaces differ only if verification accepts a tuple not previously recorded by encryption. The hop from $(\text{Enc}_K, \text{Replay})$ to $(\text{Rand}, \text{Replay})$ is bounded by privacy: $\mathcal{A}_{\text{priv}}$ runs $\mathcal{A}$, forwards encryption queries to its own oracle, records the returned tuples, and answers verification queries by replay-table lookup. The lemma is applied in the random-oracle model in the proof of Theorem 1.

2.4 Multi-User Indistinguishability

The multi-user indistinguishability notion of [BT16, Fig. 3] is the multi-user lift of the single-user all-in-one notion of Section 2.3. The number of users u is adversary-determined: it is the number of **New** queries the adversary makes, i.e., the final value of the user counter v in the game below. A challenge bit $b \leftarrow \$ \{0, 1\}$ selects between the real construction interface (real ciphertexts, real verification) and an ideal interface (uniform strings of matching length, verification that accepts only previously-issued encryption tuples). The adversary also accesses an ideal primitive interface, shared between the two worlds. The adversary outputs a guess b' and wins if $b' = b$.

Game. The game $\text{G}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A})$ of [BT16, Fig. 3] samples $b \leftarrow \$ \{0, 1\}$ and maintains a user counter v , a set V of (d, N) pairs already used by encryption, and a set W of recorded encryption tuples (d, N, C, A) . We write the user index as d and the associated-data input as A , where [BT16] writes i and H respectively, to match the notation of Section 4.4; unlike [BT16], whose advantage is the signed quantity $2 \Pr[\text{G}] - 1$, we take the absolute value so that the advantage equals the two-game distinguishing form below. The adversary has access to

- **New**(): increment v , sample $K_v \leftarrow \$ \mathcal{K}$.

- 301 • $\text{Enc}(d, N, M, A)$: reject inactive users and repeated user-nonce pairs. Otherwise
 302 compute $C_1 = \text{Enc}_{K_d}(N, A, M)$, sample $C_0 \leftarrow \$ \{0, 1\}^{8\|C_1\|}$, record (d, N) in V and
 303 (d, N, C_b, A) in W , and return C_b .
- 304 • $\text{Vf}(d, N, C, A)$: reject if $d \notin \{1, \dots, v\}$; if $(d, N, C, A) \in W$ return **true**; if $b = 0$ return
 305 **false**; else return $[\text{Dec}_{K_d}(N, A, C) \neq \perp]$.
- 306 • The shared ideal primitive interface.

307 The adversary outputs $b' \in \{0, 1\}$ and the game returns 1 if $b' = b$ and 0 otherwise. The
 308 advantage is

$$309 \quad \mathbf{Adv}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A}) = |2 \Pr[\text{G}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A}) \text{ returns } 1] - 1|.$$

310 **Equivalence with the two-game form.** Since b is uniform, a standard calculation gives

$$311 \quad \mathbf{Adv}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A}) = |\Pr[\mathcal{A}^{\text{Enc}, \text{Vf}, b=1} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Enc}, \text{Vf}, b=0} \Rightarrow 1]|,$$

312 so $\mathbf{Adv}_{\text{SE}}^{\text{mu-ind}}$ coincides with the two-game distinguishing advantage between the real
 313 (Enc, Ver) pair and the ideal $(\text{Rand}, \text{Replay})$ pair, instantiated per user. The single-user
 314 case $u = 1$ recovers the all-in-one AE notion used in Section 2.3, augmented with the
 315 ideal primitive interface present in both worlds; in particular, $\mathbf{Adv}_{\text{SE}}^{\text{mu-ind}}$ generalizes $\mathbf{Adv}^{\text{ae}}$,
 316 since an all-in-one adversary embeds into the $u = 1$ game with identical advantage by
 317 making no primitive queries.

318 **Per-user nonce uniqueness.** Nonce uniqueness is enforced per user via the set V : the
 319 same nonce may appear under distinct users but not twice under the same user. This is
 320 strictly weaker than the global nonce-uniqueness requirement that would force every (d, N)
 321 pair across all users to be unique.

322 The multi-key committing-security story does not need a separate multi-user notion:
 323 the s -way CMTDs- ℓ games of Section 2.5 already grant the adversary full control of all s
 324 keys, so they are themselves the multi-key committing statement.

325 2.5 Committing Security

326 [BH22] formalize a family of committing-security notions for nonce-based AEAD schemes,
 327 parameterized by how much of the encryption input is committed and by whether commit-
 328 ment is established through decryption (the D-notions) or through encryption agreement
 329 (the E-notions). We recall their definitions and the s -way generalization used throughout
 330 this paper.

331 **What-is-committed functions.** For $\ell \in \{1, 3, 4\}$, the *what-is-committed* function WiC_ℓ
 332 projects an encryption input $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$ to the portion it should
 333 commit:

$$\begin{aligned} 334 \quad & \text{WiC}_1(K, N, A, M) = K, \\ & \text{WiC}_3(K, N, A, M) = (K, N, A), \\ & \text{WiC}_4(K, N, A, M) = (K, N, A, M). \end{aligned}$$

335 **CMTD- ℓ and CMT- ℓ (two-key form).** Following [BH22, Fig. 4], the two-key CMTD- ℓ
 336 game samples no key. The adversary outputs a ciphertext C together with two tuples
 337 $(K_1, N_1, A_1, M_1), (K_2, N_2, A_2, M_2)$ with $\text{WiC}_\ell(K_1, N_1, A_1, M_1) \neq \text{WiC}_\ell(K_2, N_2, A_2, M_2)$,
 338 and wins if $\text{Dec}_{K_i}(N_i, A_i, C) = M_i$ for both $i \in \{1, 2\}$. The CMT- ℓ game has the
 339 adversary output the two tuples (without C); the adversary wins if $\text{Enc}_{K_1}(N_1, A_1, M_1) =$
 340 $\text{Enc}_{K_2}(N_2, A_2, M_2)$. No tuple entry is allowed to be $\perp$ and all entries must lie in the
 341 scheme's syntactic domain. Define $\mathbf{Adv}_{\text{SE}}^{\text{CMTD-}\ell}(\mathcal{A})$ and $\mathbf{Adv}_{\text{SE}}^{\text{CMT-}\ell}(\mathcal{A})$ as the probabilities
 342 that the adversary wins the respective game.

s -way generalization. For $s \geq 2$, [BH22, Fig. 5] extends the games to tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$ with pairwise distinct WiC_ℓ -images. The s -way CMTDs- ℓ game has the adversary output a ciphertext C alongside the tuples and returns 1 if $\text{Dec}_{K_i}(N_i, A_i, C) = M_i$ for every i . The s -way CMTs- ℓ game returns 1 if all encryptions $\text{Enc}_{K_i}(N_i, A_i, M_i)$ are equal. The two-tuple case $s = 2$ recovers CMTD- ℓ and CMT- ℓ . Advantages $\text{Adv}_{\text{SE}}^{\text{CMTDs-}\ell}(\mathcal{A})$ and $\text{Adv}_{\text{SE}}^{\text{CMTs-}\ell}(\mathcal{A})$ are the probabilities that the adversary wins the respective s -way game; the parameter s is suppressed from the notation when clear from context. The CMTDs- ℓ and CMTs- ℓ definitions are instantiated for TW128 in Section 4.1, with the public-permutation lift deferred to that section.

Relations. [BH22, Figs. 4 and 5] record the implications among the notions. For the D-notions we use:

$$\text{CMTDs-3} \Rightarrow \text{CMTDs-}\ell \text{ for } \ell \in \{1, 4\}$$

(the $\ell = 1$ case is immediate since pairwise distinct keys imply pairwise distinct (K, N, A) triples; the $\ell = 4$ case uses deterministic decryption, as recalled in Section C). The E-notions are bounded directly in Section C by the same final-root candidate-collision argument, rather than through an encryption reduction.

3 The TW128 Construction

3.1 Design Overview

TW128 is a nonce-based authenticated-encryption scheme with associated data, built as a two-level tree of duplex objects over the KECCAK- $p[1600, 12]$ permutation. A single *root* duplex, initialized with the key K and nonce N , absorbs the associated data A when it is nonempty, emits keystream for the first plaintext chunk M_0 of at most β bytes, and absorbs the resulting ciphertext chunk C_0 ; when A is empty the associated-data phase is elided and the initialization call itself emits the initial M_0 keystream; the remaining plaintext is partitioned into *leaf* chunks $M_1, \dots, M_n$, each handled by an independent leaf duplex initialized with (K, N, j) . The leaf duplex emits the keystream for M_j , absorbs the resulting ciphertext chunk C_j , and produces a τ_{leaf} -bit leaf tag T_j . When $n \geq 1$ the leaf tags, together with the eight-byte little-endian length $\langle n \rangle_8$, are absorbed into the root duplex, which emits a single τ_{root} -bit root tag T ; when $n = 0$ the aggregation phase is elided and the closing message-phase call of the root chunk emits T directly, mirroring the way a leaf duplex emits its leaf tag. The construction is depicted in Figure 1.

Three design properties motivate the tree shape. First, the leaf duplexes operate on disjoint state and consume disjoint inputs once (K, N) is fixed, so leaves $1, \dots, n$ can be evaluated in parallel across SIMD lanes; Section 9 quantifies the resulting throughput on AMD64 and ARM64 hardware. Second, every duplex operates on a bounded 1600-bit state regardless of message length, so the memory footprint is constant in $\|M\|$. Third, the explicit aggregation of leaf tags through a single deterministic root transcript gives the construction its committing-security guarantee: distinct (K, N, A) triples force distinct final root transcripts before aggregation values matter (Corollary 1 and § 8), which is the structural fact behind the CMTDs-4 headline of Section 5.1.

3.2 Parameters and Domain

Table 2 lists the user-visible and derived scheme parameters. The implementation uses the same byte length for the root tag and each leaf tag, but the proofs of Sections 7 and 8 keep τ_{root} and τ_{leaf} as separate symbols.

The bound $M_{\text{max}} = 8183 \cdot 2^{64}$ bytes is the largest plaintext whose leaf index fits in $\langle \cdot \rangle_8$: writing $\text{leaves}(M) = \max(0, \lceil \|M\|/\beta \rceil - 1)$, every supported plaintext satisfies

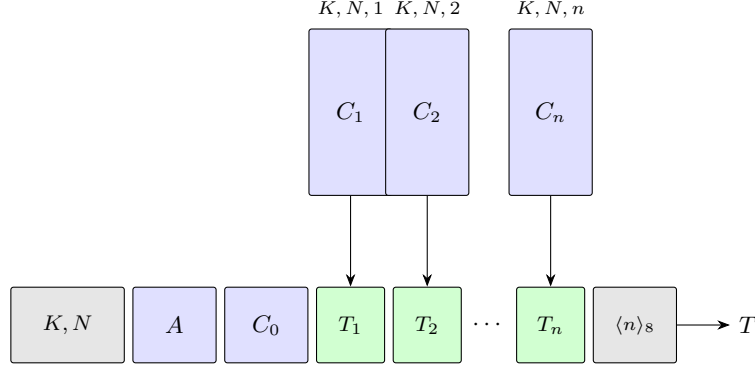


Figure 1: Schematic of TW128 for $n \geq 1$ leaves. The root duplex (bottom row) is initialized with (K, N) and absorbs in turn the associated data A , the root ciphertext chunk C_0 , each leaf tag $T_1, \dots, T_n$, and the eight-byte little-endian length $\langle n \rangle_8$; it emits the root tag T . Each leaf duplex (top row) is initialized with (K, N, j) , emits keystream for plaintext chunk M_j , absorbs the resulting ciphertext chunk C_j , and produces T_j . Leaves can be evaluated in parallel. The ciphertext chunks $C_0, C_1, \dots, C_n$ are obtained by XORing the keystream emitted by the corresponding duplex with the plaintext chunks $M_0, M_1, \dots, M_n$. When $n = 0$ the diagram reduces to the bottom row without any T_j boxes or aggregation block: the aggregation phase is elided and the closing message-phase call on C_0 emits the root tag T directly. When the associated data is empty the A block is absent: the associated-data phase is elided and the initialization call emits the initial M_0 keystream directly.

389 $0 \leq \text{leaves}(M) \leq 2^{64} - 1$, so leaf indices lie in $\{1, \dots, 2^{64} - 1\}$. The associated-data bound
 390 $A_{\max}$ is a fixed scheme parameter taken equal to $M_{\max}$ unless stated otherwise. Inputs
 391 exceeding these bounds are outside the scheme domain; every game in Section 4 rejects
 392 domain-violating queries.

393 The derived per-call absorption bound ρ follows the [BDPVA11] analysis: with $r = 1344$
 394 and pad10^*1 padding, $\rho_{\max}(\text{pad10}^*1, r) = r - 2 = 1342$ bits. The byte-aligned data field
 395 shares this allotment with the four-bit phase suffix of Section 3.4, so ρ is the largest integer
 396 satisfying $8\rho + 4 \leq 1342$, giving $\rho = 167$ data bytes (and $8\rho + 4 = 1340 \leq 1342$).

397 3.3 Parameter Choice Rationale

398 The four free parameters c , k , $\tau_{\text{root}} = \tau_{\text{leaf}}$, and β (the rate $r = b - c$ is then determined
 399 by the capacity) are chosen to give 128-bit single-key security against every term of the
 400 concrete bounds derived in Section 5.5 while keeping per-byte cost low. Specifically:

- 401 • $c = 256$. The dominant single-key term is the [BDPVA08] flat-sponge loss

$$402 \text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1},$$

403 which reaches the 2^{-128} threshold at $N_{\text{tot}} \approx 2^{(c-127)/2} \approx 2^{64.5}$ KECCAK- p calls; the
 404 headline cap of $N_{\text{tot}} \leq 2^{63}$ used in Section 5.6 keeps this term at most $2^{-131} + 2^{-194}$,
 405 and hence below 2^{-130} . Setting $c = 256$ thereby delivers 128-bit single-key security
 406 on this term. Under the mu-ind bound of Section 5.1 the per-user security margin
 407 degrades by $\log D$ bits in D users; the concrete D -indexed envelope appears in
 408 Section 5.6.

- 409 • $r = 1344$. With state width $b = 1600$ fixed by the KECCAK- $p[1600, 12]$ choice,
 410 $r = b - c = 1344$ is the largest rate compatible with the chosen capacity. The 12-
 411 round KECCAK- $p[1600, 12]$ permutation matches KangarooTwelve’s choice [BDP⁺16],

Table 2: TW128 parameters.

Parameter	Value	Description
k	256	Key length (bits)
ν	256	Nonce length (bits)
τ_{root}	256	Root tag length (bits)
τ_{leaf}	256	Leaf tag length (bits)
β	8183	Plaintext chunk size (bytes)
M_{max}	$8183 \cdot 2^{64}$ bytes	Maximum plaintext length
A_{max}	$8183 \cdot 2^{64}$ bytes	Maximum associated-data length
b	1600	Permutation state width (bits)
n_r	12	Permutation rounds
r	1344	Sponge rate (bits)
c	256	Sponge capacity (bits)
ρ	167	Maximum byte-aligned absorption per duplex call (bytes)

with a substantial cryptanalytic safety margin relative to the 24-round version used in SHA-3 [Nat15].

- $\tau_{\text{root}} = \tau_{\text{leaf}} = 256$. Matching the capacity ensures that the per-tag collision terms in the authenticity and committing bounds (Section 5.5) are dominated by the capacity birthday rather than the tag length.
- $k = 256$. Matching the capacity ensures that the per-user key guess

$$\text{KeyGuess}_k(N_{\text{prim}}) = N_{\text{prim}}/2^k$$

remains below the capacity birthday at the target D of Section 5.6.

- $\beta = 8183$. The chunk size is set to $49\rho = 49 \cdot 167$ bytes, the largest multiple of the per-call absorption bound ρ not exceeding 2^{13} bytes. Taking β to be an exact multiple of ρ makes every full leaf split into exactly 49 fully utilized message duplex calls with no short final block, so a full leaf costs one initialization duplex call plus 49 message duplex calls. This amortizes the fixed per-leaf initialization cost over a large chunk while keeping the leaf state working set ($8183 < 2^{13}$ bytes) small enough for L1 cache; Section 9.3 reports the resulting cycles-per-byte figures.

3.4 Domain Separation

Distinct duplex instances and distinct phases within a duplex are kept separate at the transcript level by two mechanisms. First, the initialization strings carry a 6-byte ASCII prefix and, for leaves, an eight-byte little-endian chunk index:

$$\begin{aligned} p_{\text{root}} &= \text{"TW128R"} \in \{0, 1\}^{48}, \\ p_{\text{leaf}} &= \text{"TW128L"} \in \{0, 1\}^{48}. \end{aligned}$$

The root duplex's first absorbed string is $p_{\text{root}} \parallel K \parallel N$, and the j -th leaf duplex's first absorbed string is $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$. Together with the fixed k -bit and ν -bit widths of K and N , these prefixes separate root from leaf transcripts and distinguish leaves with different chunk indices.

Second, every byte-aligned absorption carries a four-bit phase suffix. We identify each suffix with the four-bit string given by its Table 3 value written least-significant-bit first, so that a duplex call absorbing a byte-aligned block σ under a phase suffix σ_{ph} feeds the duplex interface of Section 2.2 the input $\sigma \parallel \sigma_{\text{ph}}$ before pad_{10^*1} padding. The seven

441 phase suffixes are summarized in Table 3; they correspond to initialization, non-final and
 442 final associated-data blocks, non-final and final message blocks, and non-final and final
 443 aggregation blocks. The seven values are pairwise distinct, so any two transcripts that
 444 differ in the phase of any block also differ in the flattened sponge input.

Table 3: Phase suffixes (4 bits each, appended LSB-first).

Name	Hex	Use
σ_{init}	0xC	End of initialization (the sole init call)
σ_{ad}^-	0x9	Non-final associated-data block
σ_{ad}^+	0xD	Final associated-data block
σ_{msg}^-	0xA	Non-final message block
σ_{msg}^+	0xE	Final message block
σ_{agg}^-	0xB	Non-final aggregation block
σ_{agg}^+	0xF	Final aggregation block (emits root tag)

445 The combined effect of the initialization prefixes and the phase suffixes is that the
 446 flattened sponge input of any TW128 duplex call uniquely determines the duplex instance,
 447 the leaf index when applicable, the phase, and the duplex-call position within that phase.
 448 Definition 1 (Well-formed TW128 transcript prefixes), Lemma 4 (Transcript Injectivity),
 449 and its structural corollaries formalize this parsing fact for the proofs of Sections C and 6
 450 to 8.

451 3.5 Encryption

452 Algorithm 2 specifies $\text{Enc}_K(N, A, M)$ in terms of two helper procedures, ABSORB and
 453 STREAMENC, defined in Algorithm 1. ABSORB drives a duplex through a sequence of
 454 blocks of a byte string with non-final σ^- and final σ^+ suffix tags, returning the output
 455 of the final call; the empty-input case yields a single empty block carrying the σ^+ tag.
 456 The streaming helpers use the same empty-input convention, so a zero-length chunk still
 457 performs one final message-phase duplex call and returns a zero-length body. Algorithm 2
 458 invokes ABSORB for the associated-data phase only when $A \neq \varepsilon$; when $A = \varepsilon$ the phase
 459 is elided and the root σ_{initcall} emits the initial M_0 keystream directly. STREAMENC
 460 encrypts a plaintext byte string block by block, XORing each plaintext block with the
 461 current keystream, absorbing the resulting ciphertext block back into the duplex with the
 462 appropriate phase tag, and using the duplex’s output as the next keystream. This is the
 463 duplex-level presentation of the overwrite mechanism of [BDPVA11, §6.2 and Algorithm 5]:
 464 when the current rate prefix emitted as keystream is Z , XOR-absorbing $m \oplus Z$ has the
 465 same update on the message bytes as overwriting that prefix with m , with the phase
 466 tag still carried as part of the duplex input. In both helpers, each duplex call absorbs
 467 a data block followed by its phase suffix— σ^- for a non-final block and σ^+ for the final
 468 block—and requests the output bits the caller consumes next: ℓ on the final call, and the
 469 next block’s keystream length (or zero) on non-final calls. Each call is the duplex interface
 470 of Section 2.2 applied to the suffixed input of Section 3.4.

471 The encryption procedure (Algorithm 2) instantiates the root duplex and obtains the
 472 initial keystream for the root chunk M_0 : when $A \neq \varepsilon$ it absorbs the associated data A ,
 473 and when $A = \varepsilon$ it elides the associated-data phase and takes the keystream from the
 474 σ_{initcall} itself. It then encrypts M_0 while absorbing the resulting ciphertext chunk C_0 .
 475 When $n = 0$ the closing σ_{msg}^+ call of this root chunk requests τ_{root} output bits, which are the
 476 root tag T , and the aggregation phase is elided. When $n \geq 1$ it independently encrypts
 477 each leaf chunk M_j while absorbing C_j , producing the leaf tag T_j ; the leaf tags together
 478 with $\langle n \rangle_8$ are then absorbed into the root duplex, and the final σ_{agg}^+ call emits the root tag
 479 T . The ciphertext is the concatenation $C_0 \parallel C_1 \parallel \dots \parallel C_n$. Thus this section writes C for

Algorithm 1 TW128 helper procedures.

```

1: procedure ABSORB( $D, s, \sigma^-, \sigma^+, \ell$ )
2:   Partition  $s$  into byte blocks  $s_1, \dots, s_t$  of exactly  $\rho$  bytes each, except that the last
   block may be shorter, with  $t \geq 1$  (the empty case yields the single empty block  $s_1 = \varepsilon$ ).
3:   for  $i \leftarrow 1$  to  $t - 1$  do
4:      $D.\text{duplexing}(s_i \parallel \sigma^-, 0)$ 
5:   end for
6:   return  $D.\text{duplexing}(s_t \parallel \sigma^+, \ell)$ 
7: end procedure

8: procedure STREAMENC( $D, m, Z, \sigma^-, \sigma^+, \ell$ )
9:   Partition  $m$  into byte blocks  $m_1, \dots, m_t$  of exactly  $\rho$  bytes each, except that the
   last block may be shorter, with  $t \geq 1$  (if  $m = \varepsilon$ , set  $t = 1$  and  $m_1 = \varepsilon$ ).
10:  for  $i \leftarrow 1$  to  $t - 1$  do
11:     $c_i \leftarrow m_i \oplus Z$ 
12:     $Z \leftarrow D.\text{duplexing}(c_i \parallel \sigma^-, 8\|m_{i+1}\|)$ 
13:  end for
14:   $c_t \leftarrow m_t \oplus Z$ 
15:   $Z' \leftarrow D.\text{duplexing}(c_t \parallel \sigma^+, \ell)$ 
16:  return  $(c_1 \parallel \dots \parallel c_t, Z')$ 
17: end procedure

18: procedure STREAMDEC( $D, c, Z, \sigma^-, \sigma^+, \ell$ )
19:  Partition  $c$  into byte blocks  $c_1, \dots, c_t$  of exactly  $\rho$  bytes each, except that the last
   block may be shorter, with  $t \geq 1$  (if  $c = \varepsilon$ , set  $t = 1$  and  $c_1 = \varepsilon$ ).
20:  for  $i \leftarrow 1$  to  $t - 1$  do
21:     $m_i \leftarrow c_i \oplus Z$ 
22:     $Z \leftarrow D.\text{duplexing}(c_i \parallel \sigma^-, 8\|c_{i+1}\|)$ 
23:  end for
24:   $m_t \leftarrow c_t \oplus Z$ 
25:   $Z' \leftarrow D.\text{duplexing}(c_t \parallel \sigma^+, \ell)$ 
26:  return  $(m_1 \parallel \dots \parallel m_t, Z')$ 
27: end procedure

```

480 the ciphertext body and $C \parallel T$ for the full AEAD ciphertext returned by $\text{Enc}_K(N, A, M)$.

481 The keystream bit length requested for the first message block of M_0 is $8 \min(\|M_0\|, \rho)$,
 482 eight times the size of the next message block. It is emitted by the closing σ_{ad}^+ call when
 483 $A \neq \varepsilon$, and by the root σ_{init} call itself when $A = \varepsilon$ and the associated-data phase is
 484 elided. Each leaf's σ_{init} call likewise emits $8 \min(\|M_j\|, \rho)$ keystream bits for M_j , so the
 485 empty-associated-data root σ_{init} call plays exactly the role a leaf σ_{init} call already plays.
 486 When the corresponding chunk is empty, the requested keystream length is zero and the
 487 next message-phase duplex call absorbs the empty ciphertext block carrying σ_{msg}^+ . The
 488 final σ_{msg}^+ call in the root chunk requests τ_{root} output bits when $n = 0$, namely the root tag
 489 T emitted by the elided-aggregation path, and zero output bits when $n \geq 1$ because the
 490 next duplex operation on the root is then the first aggregation block; the corresponding
 491 call on a leaf requests τ_{leaf} bits, namely the leaf tag. When $n \geq 1$ the final σ_{agg}^+ call requests
 492 τ_{root} bits, which is the root tag T .

493 The leaf loop is annotated **in parallel** to make explicit that the n leaf computations
 494 are mutually independent; Section 9.1 discusses the SIMD realizations.

Algorithm 2 $\text{Enc}_K(N, A, M)$.

Require: $K \in \{0, 1\}^k$, $N \in \{0, 1\}^\nu$, A with $\|A\| \leq A_{\max}$, M with $\|M\| \leq M_{\max}$.

Ensure: $C \parallel T$ with $\|C\| = \|M\|$ and $T \in \{0, 1\}^{\tau_{\text{root}}}$.

```

1:  $M_0 \leftarrow$  first  $\min(\|M\|, \beta)$  bytes of  $M$  (possibly  $\varepsilon$ ).
2: Partition the remaining suffix into chunks  $M_1, \dots, M_n$  of exactly  $\beta$  bytes each, except
   that the last chunk may be shorter but nonempty, so  $n = \text{leaves}(M)$ .
3:  $D_{\text{root}} \leftarrow$  new Duplex
4: if  $A = \varepsilon$  then
5:    $Z \leftarrow D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 8 \min(\|M_0\|, \rho))$ 
6: else
7:    $D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 0)$ 
8:    $Z \leftarrow \text{ABSORB}(D_{\text{root}}, A, \sigma_{\text{ad}}^-, \sigma_{\text{ad}}^+, 8 \min(\|M_0\|, \rho))$ 
9: end if
10: if  $n = 0$  then
11:    $(C_0, T) \leftarrow \text{STREAMENC}(D_{\text{root}}, M_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{root}})$ 
12:   return  $C_0 \parallel T$ 
13: end if
14:  $(C_0, \_) \leftarrow \text{STREAMENC}(D_{\text{root}}, M_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, 0)$ 
15: for  $j \leftarrow 1$  to  $n$  in parallel do
16:    $D_j \leftarrow$  new Duplex
17:    $Z_j \leftarrow D_j.\text{duplexing}(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8 \parallel \sigma_{\text{init}}, 8 \min(\|M_j\|, \rho))$ 
18:    $(C_j, T_j) \leftarrow \text{STREAMENC}(D_j, M_j, Z_j, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{leaf}})$ 
19: end for
20:  $G \leftarrow T_1 \parallel \dots \parallel T_n$ .
21:  $T \leftarrow \text{ABSORB}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma_{\text{agg}}^-, \sigma_{\text{agg}}^+, \tau_{\text{root}})$ 
22: return  $C_0 \parallel C_1 \parallel \dots \parallel C_n \parallel T$ 

```

3.6 Decryption

Algorithm 3 specifies $\text{Dec}_K(N, A, C \parallel T)$. The procedure mirrors Enc block for block: each ciphertext block c_i is absorbed into the appropriate duplex (root or leaf) under the corresponding σ_{msg}^- or σ_{msg}^+ suffix, and the plaintext block is recovered as $m_i = c_i \oplus Z^{(i)}$, writing $Z^{(i)}$ for the value the running keystream variable Z of STREAMDEC holds when block i is processed ($i < t$ inside the loop, $i = t$ after it), the same value the encryption procedure would have produced. The procedure absorbs ciphertext bytes before the final tag comparison, so the verification transcript is determined by (K, N, A, C) for every in-domain ciphertext, accepting or not. This property is used by the authenticity proof of Section 7 to argue that encryption-first and verification-first transcript classes are well defined regardless of the tag-comparison outcome.

For root-only inputs ($n = 0$), the leaf loop performs no iterations and the aggregation phase is elided: the closing σ_{msg}^+ call of the root chunk emits the candidate tag T' directly, and no $\langle 0 \rangle_8$ block is absorbed. For an empty body, $M_0 = C_0 = \varepsilon$; the streaming helper still makes the single empty σ_{msg}^+ call specified in Algorithm 1, now requesting τ_{root} output bits.

Correctness. For every $K \in \{0, 1\}^k$, $N \in \{0, 1\}^\nu$, A with $\|A\| \leq A_{\max}$, and M with $\|M\| \leq M_{\max}$, $\text{Dec}_K(N, A, \text{Enc}_K(N, A, M)) = M$. This follows by inspection: every duplex call in Dec_K receives the same input as the corresponding call in Enc_K (each ciphertext block is absorbed under the same phase tag in both directions), so the duplex states and the recomputed root tag T' exactly match. The constant-time comparison $T' = T$ succeeds, and the recovered $m_i = c_i \oplus Z^{(i)} = (m_i \oplus Z^{(i)}) \oplus Z^{(i)} = m_i$ at every position.

Algorithm 3 $\text{Dec}_K(N, A, C \parallel T)$.

Require: K, N, A as in Algorithm 2, ciphertext C with $\|C\| \leq M_{\max}$, candidate tag $T \in \{0, 1\}^{\tau_{\text{root}}}$.

Ensure: M with $\|M\| = \|C\|$, or $\perp$.

```

1:  $C_0 \leftarrow$  first  $\min(\|C\|, \beta)$  bytes of  $C$  (possibly  $\varepsilon$ ).
2: Partition the remaining suffix into chunks  $C_1, \dots, C_n$  of exactly  $\beta$  bytes each, except
   that the last chunk may be shorter but nonempty, so  $n = \text{leaves}(C)$ .
3:  $D_{\text{root}} \leftarrow$  new Duplex
4: if  $A = \varepsilon$  then
5:    $Z \leftarrow D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 8 \min(\|C_0\|, \rho))$ 
6: else
7:    $D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 0)$ 
8:    $Z \leftarrow \text{ABSORB}(D_{\text{root}}, A, \sigma_{\text{ad}}^-, \sigma_{\text{ad}}^+, 8 \min(\|C_0\|, \rho))$ 
9: end if
10: if  $n = 0$  then
11:    $(M_0, T') \leftarrow \text{STREAMDEC}(D_{\text{root}}, C_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{root}})$ 
12:   return  $M_0$  if  $T' = T$  (compared in constant time), else  $\perp$ .
13: end if
14:  $(M_0, \_) \leftarrow \text{STREAMDEC}(D_{\text{root}}, C_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, 0)$ 
15: for  $j \leftarrow 1$  to  $n$  in parallel do
16:    $D_j \leftarrow$  new Duplex
17:    $Z_j \leftarrow D_j.\text{duplexing}(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8 \parallel \sigma_{\text{init}}, 8 \min(\|C_j\|, \rho))$ 
18:    $(M_j, T_j) \leftarrow \text{STREAMDEC}(D_j, C_j, Z_j, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{leaf}})$ 
19: end for
20:  $G \leftarrow T_1 \parallel \dots \parallel T_n$ .
21:  $T' \leftarrow \text{ABSORB}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma_{\text{agg}}^-, \sigma_{\text{agg}}^+, \tau_{\text{root}})$ 
22: if  $T' = T$  (compared in constant time) then
23:   return  $M_0 \parallel M_1 \parallel \dots \parallel M_n$ 
24: else
25:   return  $\perp$ 
26: end if

```

4 Security Model

This section fixes the security model used throughout the paper. The scheme syntax and the authenticated-encryption and committing-security definitions of Section 2 are instantiated here for TW128 in the lifted public-permutation model (Section 4.1), and we define the intermediate random-oracle world through which every bound is first proved (Section 4.2). Section 4.3 fixes the resource accounting that all later bounds reference, and Section 4.4 instantiates the multi-user indistinguishability game for TW128.

4.1 Public-Permutation Games

The lifted AE-style public-permutation advantages below compare a real execution, with sampled $\pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600})$ and direct primitive interface (π, π^{-1}) , to a simulator-ideal execution. In the simulator-ideal execution, the construction interface is the ideal interface specified by the game and the public primitive interface is $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$. The simulator $\mathcal{S}^{\text{RO}_{\text{flat}}}$ together with its inverse $\mathcal{S}^{-1, \text{RO}_{\text{flat}}}$ is the public-primitive simulator fixed in Section B; both directions share an internal lazy table for the random oracle RO_{flat} of Section 4.2. INT-CTXT and the committing-security games remain one-world real public-permutation success probabilities; the simulator appears for them only inside the reductions of Section B.

534 In the real authenticated-encryption games (privacy, INT-CTXT, all-in-one AE), the
 535 challenger additionally samples a key $K \leftarrow \$\{0, 1\}^k$ and exposes the construction oracles
 536 $\text{Enc}_K[\pi]$ (the TW128 encryption algorithm of Algorithm 2 with the sampled π) and $\text{Ver}_K[\pi]$,
 537 defined by $\text{Ver}_K[\pi](N, A, C, T) = 1$ if $\text{Dec}_K[\pi](N, A, C \parallel T) \neq \perp$ and 0 otherwise. The
 538 committing-security games have no challenger-sampled key.

539 **Validity convention.** All construction-interface inputs are checked against TW128's public
 540 syntax and domain before any construction computation or construction-internal primitive
 541 lookup is performed. An encryption input is valid when $N \in \{0, 1\}^\nu$, A is a byte string
 542 with $\|A\| \leq A_{\max}$, and M is a byte string with $\|M\| \leq M_{\max}$. A verification or decryption
 543 input is valid when $N \in \{0, 1\}^\nu$, A is a byte string with $\|A\| \leq A_{\max}$, and the full
 544 ciphertext parses as $C \parallel T$ with $\|C\| \leq M_{\max}$ and $T \in \{0, 1\}^{\tau_{\text{root}}}$. In the committing games,
 545 each submitted tuple must be a valid encryption input including $K \in \{0, 1\}^k$, and a
 546 submitted CMTDs ciphertext must be a valid verification/decryption ciphertext. Invalid
 547 encryption queries are rejected and not recorded, invalid verification queries return 0, and
 548 invalid committing-game outputs make the game return 0. Such rejected inputs contribute
 549 no construction cost and create no construction transcripts. Throughout, an *accepted*
 550 encryption query is one that passes this validity check and any game-specific nonce or user
 551 check.

552 **Privacy.** The privacy game compares the real encryption interface with the fresh output
 553 oracle Rand that, on every accepted query (N, A, M) , returns a uniformly random byte
 554 string of length $\|M\| + \tau_{\text{root}}/8$. The adversary is *nonce-respecting*: no two encryption
 555 queries may share a nonce, and a query that repeats an earlier nonce is rejected and not
 556 recorded.

$$557 \quad \text{Adv}_{\text{TW128}}^{\text{priv}}(\mathcal{A}) = |\Pr[\mathcal{A}^{\text{Enc}_K[\pi], \pi, \pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Rand}, S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}}} \Rightarrow 1]|.$$

558 **Ciphertext integrity (INT-CTXT).** The INT-CTXT game exposes encryption, verifica-
 559 tion, and the public permutation interfaces π, π^{-1} . Each encryption query records the
 560 resulting tuple (N, A, C, T) . The adversary wins if some verification query returns 1 on a
 561 tuple (N, A, C, T) that was not previously recorded by the encryption oracle.

$$562 \quad \text{Adv}_{\text{TW128}}^{\text{int-ctxt}}(\mathcal{A}) = \Pr[\mathcal{A}^{\text{Enc}_K[\pi], \text{Ver}_K[\pi], \pi, \pi^{-1}} \text{ wins}].$$

563 No nonce uniqueness is imposed.

564 **All-in-one AE.** The all-in-one authenticated-encryption game [RS06] replaces the con-
 565 struction interface by the ideal pair $(\text{Rand}, \text{Replay})$. On each accepted encryption query
 566 (N, A, M) , Rand samples a uniformly random byte string of length $\|M\| + \tau_{\text{root}}/8$, parses
 567 it as $C \parallel T$ with $T \in \{0, 1\}^{\tau_{\text{root}}}$, records (N, A, C, T) , and returns $C \parallel T$. The oracle
 568 Replay returns 1 exactly on recorded tuples (N, A, C, T) and 0 otherwise. The adversary is
 569 nonce-respecting on encryption queries.

$$570 \quad \text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) = |\Pr[\mathcal{A}^{\text{Enc}_K[\pi], \text{Ver}_K[\pi], \pi, \pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Rand}, \text{Replay}, S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}}} \Rightarrow 1]|.$$

571 This lifted all-in-one AE advantage is the headline target of Section 5.1: a Bellare–
 572 Namprempre composition (Lemma 1) applied in the random-oracle model to derived
 573 privacy and INT-CTXT adversaries, combined with the flat-sponge lift, bounds it via the
 574 privacy and INT-CTXT envelopes of Section 5.3; the precise bound is Theorem 1.

Committing security. The s -way committing-security game follows [BH22, Figs. 4 and 5], extended to $s \geq 2$ tuples. There is no challenger-sampled key. After the primitive sampling, the adversary outputs a ciphertext $C \parallel T$ together with s tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$. The challenger applies the following deterministic checks, with the eager length rejection justified by Algorithm 3: $\text{Dec}_K(N, A, C \parallel T)$ returns either $\perp$ or a message of length $\|C\|$.

```

580    $\pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600}),$ 
581    $(C \parallel T, (K_i, N_i, A_i, M_i)_{i=1}^s) \leftarrow \$ \mathcal{A}^{\pi, \pi^{-1}},$ 
582   return 0 if  $C \parallel T$  is not a valid TW128 ciphertext,
583   return 0 if any  $(K_i, N_i, A_i, M_i)$  is not in the TW128 syntactic domain,
584   return 0 if any  $\|M_i\| \neq \|C\|,$ 
585   return 0 if  $(K_i, N_i, A_i)_{i=1}^s$  are not pairwise distinct,
586   return  $\bigwedge_{i=1}^s [\text{Dec}_{K_i}[\pi](N_i, A_i, C \parallel T) = M_i].$ 

```

The advantage is

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}) = \Pr[\text{the game returns 1}],$$

with the probability taken over the uniform choice of π and $\mathcal{A}$'s coins. No nonce uniqueness is imposed. For $\ell \in \{1, 4\}$, the CMTDs- ℓ game is identical except that the pairwise-distinctness check is applied to the WiC_ℓ -images—the keys K_i for $\ell = 1$ and the full tuples (K_i, N_i, A_i, M_i) for $\ell = 4$ —with the same check order and the same eager length rejection; $\text{Adv}_{\text{TW128}}^{\text{CMTDs-}\ell}(\mathcal{A})$ is defined analogously. The CMTs- ℓ source games are instantiated analogously from Section 2.5: after the primitive sampling, the adversary outputs s tuples, the challenger checks syntax and pairwise WiC_ℓ -distinctness, and then returns 1 iff the s deterministic encryptions $\text{Enc}_{K_i}[\pi](N_i, A_i, M_i)$ are all equal. Section C promotes the CMTDs-3 bound proved in Section 8 (Theorem 6) to the headline CMTDs-4 statement of Section 5.1 and proves matching direct bounds for the CMTs- ℓ source games.

4.2 The TW128 Random-Oracle Model

The proofs of Sections 6 to 8 and Section C, together with the headline reductions of Section 5.1, are carried out through an intermediate random-oracle model whose construction interfaces use the [BDPVA11] bridge of Section 2.2 to address a single random oracle. The flat-sponge lift of Section B then converts the resulting bounds to the public-permutation form.

The model has a single random oracle $\text{RO}_{\text{flat}} : \{0, 1\}^* \rightarrow \{0, 1\}^\infty$ over the unpadded H -input domain of the [BDPVA08] simple-padded sponge interface, with lazy sampling: each distinct input receives an independent infinite bit string, and repeated queries return the same string. A direct adversary query is finite-output: on input (Y, ℓ) with finite ℓ , the oracle returns the first ℓ bits of the lazy-sampled value for Y .

For a well-formed TW128 duplex transcript prefix X in the language of Definition 1, at one of the construction transcript lookup points formalized by Lemma 5 and Table 7, let $\text{flat}(X) = I[1344, 1344](\text{raw_flat}(X))$ be the bridged image of the native flattened input under the [BDPVA11, Theorem 3] preprocessing map of Section 2.2, and define

$$\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X)).$$

The construction interfaces Enc_K^{RO} , Dec_K^{RO} , and Ver_K^{RO} are the TW128 algorithms of Section 3 with every root or leaf duplex output replaced by the corresponding requested projection of $\text{RF}(X)$; the projection length is the duplexing call's output parameter ℓ , which may be zero. The bridge and the typed-restriction convention are formalized in Lemma 5; the direct-query key-hit predicate ignores the adversary's requested output length and is formalized as KeyedTWOut in Lemma 5.

The random-oracle-model games are the construction-interface variants of Sections 4.1 and 4.4: the primitive interface is replaced by direct adversary access to RO_{flat} , and each construction oracle uses the corresponding RO algorithm.

$$\text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B}) = |\Pr[\mathcal{B}^{\text{Enc}_{\text{K}}^{\text{RO}}, \text{RO}_{\text{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\text{Rand}, \text{RO}_{\text{flat}}} \Rightarrow 1]|,$$

$$\text{Adv}_{\text{RO}}^{\text{int-ctxt}}(\mathcal{B}) = \Pr[\mathcal{B}^{\text{Enc}_{\text{K}}^{\text{RO}}, \text{Ver}_{\text{K}}^{\text{RO}}, \text{RO}_{\text{flat}}} \text{ wins}],$$

$$\text{Adv}_{\text{RO}}^{\text{ae}}(\mathcal{B}) = |\Pr[\mathcal{B}^{\text{Enc}_{\text{K}}^{\text{RO}}, \text{Ver}_{\text{K}}^{\text{RO}}, \text{RO}_{\text{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\text{Rand}, \text{Replay}, \text{RO}_{\text{flat}}} \Rightarrow 1]|,$$

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-3}}(\mathcal{B}) = \Pr[\text{the random-oracle CMTDs-3 game returns 1}].$$

For $\ell \in \{1, 3, 4\}$, $\text{Adv}_{\text{RO}}^{\text{CMTs-}\ell}(\mathcal{B})$ denotes the corresponding random-oracle CMTs- ℓ game: the adversary has direct access to RO_{flat} , and the challenger computes its deterministic encryption checks with Enc^{RO} . For mu-ind, $\text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B})$ denotes the Section 4.4 game with the same New interface and per-user nonce discipline: when $b = 1$, user d uses $(\text{Enc}_{\text{K}_d}^{\text{RO}}, \text{Ver}_{\text{K}_d}^{\text{RO}})$, and when $b = 0$, user d uses the ideal pair $(\text{Rand}, \text{Replay})$. Equivalently, in the two-game notation of Section 2.4,

$$\text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}) = |\Pr[\mathcal{B}^{\text{Real}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\text{Ideal}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1]|,$$

where the superscripts name the entire multi-user construction interface in the corresponding world. Adversary queries to RO_{flat} are finite-output and counted in $q_{\text{RO}}(\mathcal{B})$ of Section 4.3. Construction-internal $\text{RF}(\cdot)$ lookups — those made by $\text{Enc}_{\text{K}}^{\text{RO}}, \text{Dec}_{\text{K}}^{\text{RO}}, \text{Ver}_{\text{K}}^{\text{RO}}$, or a committing challenger — are not.

4.3 Resources

Adversary resources are measured by fixed, execution-uniform caps on the number and size of oracle queries. An execution-uniform cap is one that holds on every execution path, including over adversary randomness, oracle randomness, and adaptive query choices. Proofs may also introduce local execution-uniform caps, such as q_e for the number of accepted encryption queries, solely to index a finite hybrid sequence. Such local caps are part of the adversary's fixed query bounds, but they are not listed as envelope parameters unless they affect the final bound.

N : the construction-interface cost, measured as the number of $\text{KECCAK} - p[1600, 12]$ calls that the adversary's construction-interface queries induce in the real TW128 computation after the validity convention above has been applied. In ideal games, the same accounting applies to the corresponding real TW128 computation on the valid query, even when an ideal oracle answers by table lookup or without running the construction. N counts at per-block granularity, summing each root initialization, associated-data block, root message block, leaf initialization, leaf message block, and aggregation block across the entire query sequence. Valid INT-CTXT verification computations are counted whether they accept or reject. In the committing games there is no adversary construction oracle: for CMTDs- ℓ , N counts only the deterministic decryption checks actually performed by the challenger on the common ciphertext, while for the CMTs- ℓ source games of Section C, N counts only the deterministic encryption checks actually used by the challenger to compare the s ciphertexts.

N_{prim} : the number of direct adversary queries to the primitive interface (the pair (π, π^{-1}) in public-permutation games; the simulator's RO_{flat} table is internal and is not exposed to the application adversary in those games).

667 $N_{\text{tot}} = N + N_{\text{prim}}$: the total query-sequence cost. The flat-sponge lift of Section B is
 668 applied at N_{tot} because the indistinguishability replacement sees both the construction-
 669 internal sponge calls counted by N and the adversary's direct primitive-interface
 670 queries counted by N_{prim} in one chronological public-primitive transcript.

671 $q_v(\mathcal{A})$, Q_v : the number of verification queries with valid inputs, and a cap on it. Used
 672 in INT-CTXT and absorbed into the AE composition of Section 5.1. Every valid
 673 verification query contributes at least one verification/decryption construction call
 674 in the N accounting, so $q_v(\mathcal{A}) \leq N$.

675 $q_{\text{RO}}(\mathcal{B})$, Q_{RO} : the number of direct finite-output queries to RO_{flat} by a random-oracle-
 676 model adversary $\mathcal{B}$, and a cap on it. Repeated direct queries on the same Y are
 677 counted again. Construction-internal $\text{RF}(\cdot)$ lookups are not counted. After the
 678 derived-adversary construction, q_{RO} counts only simulator-induced RO_{flat} calls from
 679 the adversary's direct primitive-interface queries, not the construction calls already
 680 counted in N for the flat-sponge lift.

681 $n_{\text{leaf}}(\mathcal{B})$, N_{leaf} : the number of distinct construction-generated final leaf transcripts in the
 682 execution of $\mathcal{B}$, and a cap on it. Distinct final leaf transcripts are generated by valid
 683 encryption computations, by valid INT-CTXT verification computations (whether
 684 they accept or reject), and by committing challenger construction checks that pass
 685 the early validity checks.

686 For a byte string X ,

$$687 \text{leaves}(X) = \max(0, \lceil \|X\|/\beta \rceil - 1)$$

688 counts the leaf chunks induced by a body byte string of length $\|X\|$. Thus n_{leaf} is bounded
 689 by the sum of $\text{leaves}(X)$ over the valid construction computations that create final leaf
 690 transcripts, where X is the plaintext body for encryption computations and the submitted
 691 ciphertext body for verification and committing decryption checks. Each distinct final leaf
 692 transcript contains a final leaf duplex call counted in N , so $n_{\text{leaf}}(\mathcal{B}) \leq N$ unconditionally.
 693 The ratio n_{leaf}/N is largest when most processed bytes lie in leaf chunks, where each full
 694 leaf contributes $1 + \beta/\rho = 50$ duplex calls to N per final leaf transcript.

695 All theorems are stated for adversaries whose execution-uniform caps hold at the stated
 696 bounds.

697 4.4 Multi-User Setting

698 This subsection instantiates the mu-ind game of [BT16] (Section 2.4) for TW128 in the lifted
 699 public-permutation model, using the same ideal-side simulator convention as the single-user
 700 AE-style games of Section 4.1. The challenger samples a challenge bit $b \leftarrow \$ \{0, 1\}$. If $b = 1$,
 701 it samples a permutation $\pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600})$ and exposes (π, π^{-1}) as the primitive
 702 interface; if $b = 0$, it exposes $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ as the primitive interface. The adversary
 703 $\mathcal{A}$ creates users on demand via **New**, and the game maintains an active-user counter v . We
 704 write D for an execution-uniform cap on the number of **New** calls. Equivalently, the game
 705 may sample independent keys $K_1, \dots, K_D$ at the start of the experiment and let the d -th
 706 **New** call activate K_d ; inactive indices are never valid users.

707 The mu-ind construction oracles inherit the validity convention of Section 4.1. The
 708 encryption oracle on input (d, N, M, A) rejects, without recording, if (N, A, M) is outside
 709 the TW128 encryption domain, if $d \notin \{1, \dots, v\}$, or if $(d, N) \in V$; otherwise it sets
 710 $C = \text{Enc}_{K_d}[\pi](N, A, M)$ when $b = 1$, and samples $C \leftarrow \$ \{0, 1\}^{8\|M\| + \tau_{\text{root}}}$ when $b = 0$.
 711 The game records (d, N) in V and (d, N, C, A) in W , and returns C . The verification
 712 oracle on input (d, N, C, A) rejects if $d \notin \{1, \dots, v\}$ or if (N, A, C) is not a valid TW128
 713 verification/decryption input; if $(d, N, C, A) \in W$ it returns **true**; if $b = 0$ it returns **false**;

otherwise it returns $[\text{Dec}_{K_d}[\pi](N, A, C) \neq \perp]$. Nonce uniqueness is enforced per user by the set V .

After querying these oracles, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. The mu-ind advantage of $\mathcal{A}$ against TW128 is

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) = |2 \Pr[b' = b] - 1|.$$

Resource caps split per potential user $d \leq D$: $N^{(d)}$, $q_v^{(d)}$, $n_{\text{leaf}}^{(d)}$ are the per-user counts under Section 4.3, with inactive users contributing zero. The global totals $N = \sum_{d=1}^D N^{(d)}$, $q_v = \sum_{d=1}^D q_v^{(d)}$, $n_{\text{leaf}} = \sum_{d=1}^D n_{\text{leaf}}^{(d)}$ appear in the final bound: N through the lift term $\text{Lift}_c(N_{\text{tot}})$, and n_{leaf} and q_v through the leaf-collision and verification-tag terms. N_{prim} remains a single shared count over the primitive interface.

The mu-ind treatment applies only to indistinguishability. The committing-security games already give the adversary control of all s keys, so the s -way CMTDs- ℓ statements of Section 4.1 are themselves the multi-key committing statement and need no separate mu-cmt notion.

5 Main Results

5.1 Headline Theorems

TW128 delivers three headline guarantees in the lifted public-permutation model of Section 4.1: all-in-one AE security [RS06], its multi-user indistinguishability lift [BT16], and s -way CMTDs-4 committing security [BH22]. For the AE-style notions the ideal side exposes the BDPV simulator; committing security remains a real public-permutation success probability. Theorems 1 to 3 state these bounds; the loss terms Lift_c , KeyGuess_k , and RootColl_τ are unpacked in Section 5.2, and the random-oracle envelopes $\varepsilon_{\text{RO}}^{\text{priv}}$, $\varepsilon_{\text{RO}}^{\text{int-ctxt}}$ in Section 5.3.

Theorem 1 (All-in-One AE Security). *For every nonce-respecting AE adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \varepsilon_{\text{RO}}^{\text{priv}}(N_{\text{prim}}) + \varepsilon_{\text{RO}}^{\text{int-ctxt}}(N_{\text{prim}}, N_{\text{leaf}}, Q_v).$$

Proof. Let

$$p_0 = \Pr[\mathcal{A}^{\text{Enc}_K[\pi], \text{Ver}_K[\pi], \pi, \pi^{-1}} \Rightarrow 1], \quad p_1 = \Pr[\mathcal{A}^{\text{Rand, Replay}, \mathcal{S}^{\text{ROflat}}, \mathcal{S}^{-1, \text{ROflat}}} \Rightarrow 1].$$

Then $\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) = |p_0 - p_1|$ by Section 4.1. By the flat-sponge lift bound (Lemma 18), replacing the real primitive interfaces by the simulator and replacing the construction by $(\text{Enc}_K^{\text{RO}}, \text{Ver}_K^{\text{RO}})$ costs at most $\text{Lift}_c(N_{\text{tot}})$:

$$|p_0 - p'_0| \leq \text{Lift}_c(N_{\text{tot}}), \quad p'_0 = \Pr[\mathcal{A}^{\text{Enc}_K^{\text{RO}}, \text{Ver}_K^{\text{RO}}, \mathcal{S}^{\text{ROflat}}, \mathcal{S}^{-1, \text{ROflat}}} \Rightarrow 1].$$

Let $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ be the adversary of Lemma 19. The derived-adversary construction gives

$$p'_0 = \Pr[\mathcal{B}^{\text{Enc}_K^{\text{RO}}, \text{Ver}_K^{\text{RO}}, \text{ROflat}} \Rightarrow 1], \quad p_1 = \Pr[\mathcal{B}^{\text{Rand, Replay, ROflat}} \Rightarrow 1],$$

with $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$, $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$, and $q_v(\mathcal{B}) \leq Q_v$. Therefore

$$\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\text{ae}}(\mathcal{B}).$$

The all-in-one composition lemma (Lemma 1), applied in the random-oracle model, gives

$$\text{Adv}_{\text{RO}}^{\text{ae}}(\mathcal{B}) \leq \text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B}_{\text{priv}}) + \text{Adv}_{\text{RO}}^{\text{int-ctxt}}(\mathcal{B}_{\text{int-ctxt}}),$$

for derived adversaries $\mathcal{B}_{\text{priv}}$ (which runs $\mathcal{B}$, records the outputs of its encryption-oracle queries, and answers each verification query by replay-table lookup) and $\mathcal{B}_{\text{int-ctxt}}$ (which runs $\mathcal{B}$, forwards encryption and verification queries to its own oracles, and halts on the first accepting non-replay verification response), both inheriting $\mathcal{B}$'s $(q_{\text{RO}}, n_{\text{leaf}}, q_v)$. Because $\mathcal{B}_{\text{priv}}$ forwards $\mathcal{B}$'s encryption queries unchanged and $\mathcal{B}$ inherits the nonce-respecting discipline of $\mathcal{A}$, the adversary $\mathcal{B}_{\text{priv}}$ is itself nonce-respecting, so the privacy envelope of Section 5.3 applies. Substituting the random-oracle envelopes of Section 5.3 gives the stated bound. $\square$

Lemma 2 (Mu-Ind Adjacent-Hybrid Bound). *Let $\mathcal{B}$ be a flat-RO mu-ind adversary in the pre-sampled-key formulation of Section 4.4, with keys $K_1, \dots, K_D$ sampled at the start of the experiment. Fix $d \in \{1, \dots, D\}$. Let H_{d-1}^* and H_d^* be the adjacent stopped hybrids that output a fixed bit if any two sampled keys collide, and otherwise run $\mathcal{B}$ with one shared lazy RO_{flat} table as follows: users $e < d$ are answered by $(\text{Rand}, \text{Replay})$, users $e > d$ are answered by $(\text{Enc}_{K_e}^{\text{RO}}, \text{Ver}_{K_e}^{\text{RO}})$, and user d is answered by $(\text{Enc}_{K_d}^{\text{RO}}, \text{Ver}_{K_d}^{\text{RO}})$ in H_{d-1}^* and by $(\text{Rand}, \text{Replay})$ in H_d^* . If $n_{\text{leaf}}^{(d)}$ and $q_v^{(d)}$ are the user- d resource caps, then*

$$|\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \leq 2\text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}}.$$

Proof. The fixed-output branch on a key collision is identical in H_{d-1}^* and H_d^* , so only the branch with pairwise distinct keys contributes. On that branch, the two hybrids differ only in user d : users $e < d$ are ideal in both hybrids, and users $e > d$ are real in both hybrids.

Let $\mathcal{E}_d$ denote the common surrounding environment. It runs $\mathcal{B}$, answers users $e < d$ by their $(\text{Rand}, \text{Replay})$ interfaces, answers users $e > d$ by $\text{Enc}_{K_e}^{\text{RO}}, \text{Ver}_{K_e}^{\text{RO}}$, forwards $\mathcal{B}$'s direct RO_{flat} queries unchanged, and shares the same lazy RO_{flat} table with user d . The Bellare–Namprempre insertion is pathwise in this environment. Insert the interface in which user d 's encryption oracle is still real but user d 's verification oracle is replaced by its replay table. The first hop is the user- d INT-CTXT event; the second hop is the user- d privacy replacement with the same environment and replay table. The replay table is determined by user d 's encryption answers and adds no direct RO_{flat} queries or construction-generated transcripts.

The privacy and INT-CTXT accounting used in these two hops is local to user d 's keyed transcript language. In the privacy hop, the one-query replacement of Lemma 11 only requires that no direct query has read a user- d counted transcript before $\text{bad}_{\text{key}}^{(d)}$, the bad_{key} event of Sections 6 and 7 instantiated at user d 's key K_d ; any extra lazy-table entries written by the real user- d encryption computation are unobservable before that abort. The accepted user- d encryption queries are nonce-respecting because the mu-ind interface enforces nonce uniqueness per user. In the INT-CTXT hop, Lemmas 13 to 16 refer only to user- d hidden-key transcript points, user- d final leaf transcripts, and user- d non-replay verification submissions.

It remains to justify that the same-RO environment does not enlarge the direct-query term. The real users $e > d$ may perform construction-internal RO_{flat} lookups while simulating $\text{Enc}_{K_e}^{\text{RO}}$ and $\text{Ver}_{K_e}^{\text{RO}}$. Whenever such a lookup lies in a counted keyed transcript class, Lemma 5 decodes its candidate key as K_e ; on the non-collision branch this is different from K_d . If the lookup is outside the counted keyed language, it is irrelevant to $\text{bad}_{\text{key}}^{(d)}$. Thus foreign-user construction lookups neither trigger user d 's key-test event nor pre-read a user- d hidden transcript point. They also do not create user- d leaf tag collisions or user- d verification attempts, which are counted by $n_{\text{leaf}}^{(d)}$ and $q_v^{(d)}$.

The key-test contribution is taken in the original pre-sampled-key experiment, not in a renormalized experiment conditioned on the non-collision branch. Fix the foreign keys and run the visible-prefix coupling used in the user- d privacy and INT-CTXT proofs, with foreign-key transcript addresses left unchanged. Before the first user- d key hit,

the visible prefix is invariant under renaming K_d among keys not already tested and not equal to any fixed foreign key. For each fresh candidate key tested by one of $\mathcal{B}$'s original direct RO_{flat} queries, the event contributing on the stopped branch is contained in $\{K_d \text{ equals that candidate}\}$; if the candidate is a foreign key, it contributes nothing on the non-collision branch. Hence each fresh original direct test costs at most 2^{-k} in the unconditioned key experiment, and at most $q_{\text{RO}}(\mathcal{B})$ such tests occur.

The proofs of Theorems 4 and 5 therefore apply to the two user- d hops with the same direct-query cap $q_{\text{RO}}(\mathcal{B})$ and the same user- d construction resources $n_{\text{leaf}}^{(d)}, q_v^{(d)}$. The surrounding environment affects oracle scheduling and foreign construction-internal table entries, but not the counted user- d events. The privacy hop contributes $\text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B}))$, and the INT-CTXT hop contributes

$$\text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}}.$$

Combining the two hops gives the stated bound. $\square$

Theorem 2 (Multi-User Indistinguishability). *For every mu-ind adversary $\mathcal{A}$ against TW128 with execution-uniform user cap D and the global resources of Section 4.4,*

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \frac{D(D-1)}{2^{k+1}} + 2D \cdot \text{KeyGuess}_k(N_{\text{prim}}) + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{Q_v}{2^{\tau_{\text{root}}}}.$$

Proof. Let $\text{Real}_{\text{TW128}}^{\text{mu}}[\pi]$ denote the $b = 1$ multi-user construction interface of Section 4.4, with user d using $(\text{Enc}_{K_d}[\pi], \text{Ver}_{K_d}[\pi])$, and let Ideal^{mu} denote the $b = 0$ interface with per-user (Rand, Replay). Use the pre-sampled-key formulation of Section 4.4: sample keys $K_1, \dots, K_D$ at the start of the experiment, and let the d -th New call activate K_d . Unused keys are ignored. Define

$$p_0 = \Pr[\mathcal{A}^{\text{Real}_{\text{TW128}}^{\text{mu}}[\pi], \pi, \pi^{-1}} \Rightarrow 1], \quad p_1 = \Pr[\mathcal{A}^{\text{Ideal}^{\text{mu}}, S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}}} \Rightarrow 1].$$

Then $\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) = |p_0 - p_1|$ by Sections 2.4 and 4.4. By the flat-sponge lift bound (Lemma 18), replacing (π, π^{-1}) on the real side by $(S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}})$ and each real user construction oracle by its random-oracle form costs at most $\text{Lift}_c(N_{\text{tot}})$:

$$|p_0 - p'_0| \leq \text{Lift}_c(N_{\text{tot}}), \quad p'_0 = \Pr[\mathcal{A}^{\text{Real}_{\text{RO}}^{\text{mu}}, S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}}} \Rightarrow 1],$$

where $\text{Real}_{\text{RO}}^{\text{mu}}$ uses $(\text{Enc}_{K_d}^{\text{RO}}, \text{Ver}_{K_d}^{\text{RO}})$ for every created user d . Let $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ be the adversary of Lemma 19. The derived-adversary construction gives

$$p'_0 = \Pr[\mathcal{B}^{\text{Real}_{\text{RO}}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1], \quad p_1 = \Pr[\mathcal{B}^{\text{Ideal}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1],$$

with $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ and the same per-user construction-side transcript caps. Therefore

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}).$$

Let coll denote the event that two of the pre-sampled keys $K_1, \dots, K_D$ collide; $\Pr[\text{coll}] \leq \binom{D}{2}/2^k = D(D-1)/2^{k+1}$ by union bound. In the random-oracle model, use a standard hybrid over the D users: H_d answers users $1, \dots, d$ with (Rand, Replay) and users $d+1, \dots, D$ with the real construction interface, so H_0 is the real mu-ind game and H_D is the ideal one. Let H_d^* be H_d with a fixed output bit on coll . Since coll has the same probability in all hybrids,

$$\text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}) \leq \Pr[\text{coll}] + |\Pr[\mathcal{B}^{H_0^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_D^*} \Rightarrow 1]|.$$

By the triangle inequality, it remains to bound each adjacent stopped hybrid. For each d , Lemma 2 gives the adjacent hybrid bound

$$|\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \leq 2\text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}}.$$

Using $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$, each hybrid step costs $2\text{KeyGuess}_k(N_{\text{prim}})$ plus the per-user leaf-collision and verification-tag terms. Summing across $d = 1, \dots, D$ gives $2D\text{KeyGuess}_k(N_{\text{prim}})$ for the key-guess contribution. The leaf-collision sum $\sum_d n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)$ is bounded by $N_{\text{leaf}}(N_{\text{leaf}} - 1)$ by writing, for $x_d = n_{\text{leaf}}^{(d)}$, $\sum_d (x_d^2 - x_d) \leq (\sum_d x_d)^2 - \sum_d x_d$, where $\sum_d x_d^2 \leq (\sum_d x_d)^2$ holds for nonnegative x_d by expanding $(\sum_d x_d)^2$. Then $\sum_d n_{\text{leaf}}^{(d)} = n_{\text{leaf}} \leq N_{\text{leaf}}$ and monotonicity of $y(y - 1)$ pass from $n_{\text{leaf}}(n_{\text{leaf}} - 1)$ to $N_{\text{leaf}}(N_{\text{leaf}} - 1)$. The verification-tag contribution satisfies

$$\sum_d q_v^{(d)} / 2^{\tau_{\text{root}}} \leq Q_v / 2^{\tau_{\text{root}}}.$$

Combining with the coll penalty $D(D - 1)/2^{k+1}$ and $\text{Lift}_c(N_{\text{tot}})$ yields the stated bound. $\square$

Theorem 3 (CMTDs-4 Committing Security). *For every $s \geq 2$ and every s -way CMTDs-4 adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s) = \text{Lift}_c(N_{\text{tot}}) + \frac{\binom{N_{\text{prim}} + s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

Proof. For deterministic Dec, every CMTDs-4 winner is also a CMTDs-3 winner, yielding

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A})$$

by Section C. The public-permutation CMTDs-3 bound from Corollary 3,

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s),$$

then gives the claim. $\square$

Remark 1 (Looseness of the Root-Collision Cap). The N_{prim} in Theorem 3’s root-collision term counts every primitive query, not only those whose induced simulator RO_{flat} call lands at the root tag output point ($\mathcal{R}_{\text{tag}}$ for multi-chunk ciphertexts, and $\mathcal{R}_{\text{msg}}^+$ for single-chunk ciphertexts whose aggregation phase is elided). By Corollary 2, calls landing at any other output-point class cannot contribute to the candidate sequence, and hence cannot contribute to the candidate-collision event of Theorem 6’s proof. A sharper candidate-sequence analysis could replace N_{prim} in the root-collision term by a class-filtered effective query count, no larger than N_{prim} and potentially smaller. The direct CMTs- ℓ bounds of Section C use the same accounting boundary: their challenger encryption checks are construction-internal work counted in N , so they do not enlarge the direct-query cap N_{prim} . The headline CMTDs-4 statement nevertheless keeps the N_{prim} form for simplicity and uniformity with Theorems 1 and 2; the birthday threshold at $\tau_{\text{root}} = 256$ is unaffected.

5.2 Loss Terms

Three closed-form expressions recur across the headline bounds of Section 5.1— Lift_c in all three, KeyGuess_k and RootColl_τ in some. We collect them here for reference.

Flat-sponge loss. The flat-sponge differentiating loss is

$$\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}.$$

[BDPVA08, Lemmas 4 and 5] give the differentiating probabilities in exact product form: the random-transformation probability is $f_T(N_{\text{tot}}) = 1 - \prod_{i=1}^{N_{\text{tot}}} (1 - i/2^c)$, and each factor of the random-permutation probability $f_P(N_{\text{tot}})$ is the corresponding f_T factor divided by a

880 quantity $1 - i/2^{r+c} \in (0, 1]$, hence at least as large, so $f_P(N_{\text{tot}}) \leq f_T(N_{\text{tot}})$. For $N_{\text{tot}} < 2^c$
 881 every factor lies in $[0, 1]$ and

$$882 \quad f_P(N_{\text{tot}}) \leq f_T(N_{\text{tot}}) = 1 - \prod_{i=1}^{N_{\text{tot}}} \left(1 - \frac{i}{2^c}\right) \leq \sum_{i=1}^{N_{\text{tot}}} \frac{i}{2^c} = \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{c+1}} = \text{Lift}_c(N_{\text{tot}}),$$

883 the second inequality by the Weierstrass product inequality $\prod_i (1 - x_i) \geq 1 - \sum_i x_i$ for
 884 $x_i \in [0, 1]$. [BDPVA08, Theorem 2] carries the side condition $N_{\text{tot}} < 2^c$; for $N_{\text{tot}} \geq 2^c$,
 885 $\text{Lift}_c(N_{\text{tot}}) \geq 1$ and the advantage bound is trivial without any appeal to indistinguishability,
 886 and already at $N_{\text{tot}} \geq 2^{c/2}$ the bound exceeds 1/2 and provides no meaningful security.

887 **Key-guessing term.** The ideal-system key-guessing term of [BDPVA11] is

$$888 \quad \text{KeyGuess}_k(Q) = Q/2^k.$$

889 It accounts for direct RO_{flat} queries whose inputs satisfy the decoder KeyedTWOOut of
 890 Table 7. Each such query is an adaptive test of the decoded candidate key against the
 891 hidden key K , independent of the query's requested output length. Lemma 7 bounds the
 892 probability that any of Q direct queries tests the right key by $Q/2^k$.

893 **Root-collision term.** The s -way root tag multi-collision term is

$$894 \quad \text{RootColl}_\tau(Q, s) = \binom{Q + s}{s} / 2^{\tau(s-1)},$$

895 with the abbreviation $\text{RootColl}_{\tau_{\text{root}}}(Q, s) = \text{RootColl}_\tau(Q, s)$ at $\tau = \tau_{\text{root}}$. The $+s$ is
 896 candidate-sequence slack for the s final root transcript inputs generated by the committing
 897 challenger after the adversary outputs its tuples; these lookups are construction-internal
 898 and are not added to $q_{\text{RO}}(\mathcal{B})$. In the public-permutation CMTDs-3 and direct CMTs- ℓ
 899 bounds, the committing proof builds a chronological candidate sequence and bounds only
 900 the event that some s distinct candidates have equal root tag projections. The flat-sponge
 901 lift of Section B gives $Q = N_{\text{prim}}$: only the adversary's N_{prim} direct primitive queries can
 902 induce simulator-side RO_{flat} queries that contribute to that sequence.

903 5.3 Supporting Bounds

904 The headline theorems of Section 5.1 feed on three random-oracle-model bounds—the
 905 privacy and INT-CTXT envelopes and the CMTDs-3 bound—whose proofs occupy Sec-
 906 tions 6 to 8; the remaining committing-notion bounds live in Section C. Resource-indexed
 907 suprema range over flat-RO adversaries (Section 4.2) with the displayed resources, and the
 908 flat-sponge lift of Section B instantiates each envelope at $q_{\text{RO}} = N_{\text{prim}}$ when transferring to
 909 the public-permutation setting.

910 **Privacy envelope.** Define

$$911 \quad \varepsilon_{\text{RO}}^{\text{priv}}(Q_{\text{RO}}) = \sup\{\text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B}) : q_{\text{RO}}(\mathcal{B}) \leq Q_{\text{RO}}\},$$

912 the supremum taken over nonce-respecting privacy adversaries against Enc_K^{RO} . The flat-RO
 913 privacy theorem of Section 6 gives

$$914 \quad \varepsilon_{\text{RO}}^{\text{priv}}(Q_{\text{RO}}) \leq \text{KeyGuess}_k(Q_{\text{RO}}).$$

INT-CTXT envelope. Define

$$\varepsilon_{\text{RO}}^{\text{int-ctxt}}(Q_{\text{RO}}, N_{\text{leaf}}, Q_v) = \sup\{\text{Adv}_{\text{RO}}^{\text{int-ctxt}}(\mathcal{B}) : q_{\text{RO}}(\mathcal{B}) \leq Q_{\text{RO}}, n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}, q_v(\mathcal{B}) \leq Q_v\}.$$

The flat-RO INT-CTXT theorem of Section 7 gives

$$\varepsilon_{\text{RO}}^{\text{int-ctxt}}(Q_{\text{RO}}, N_{\text{leaf}}, Q_v) \leq \text{KeyGuess}_k(Q_{\text{RO}}) + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{Q_v}{2^{\tau_{\text{root}}}}.$$

CMTDs-3 bound. The flat-RO CMTDs-3 theorem of Section 8 is the direct root-collision bound

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-3}}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s) = \binom{q_{\text{RO}}(\mathcal{B}) + s}{s} / 2^{\tau_{\text{root}}(s-1)},$$

with no key-guessing or leaf-collision term. The keys are adversary-supplied, so a direct query on an input accepted by the keyed transcript decoder **KeyedTWOut** is one of the adversary's visible RO_{flat} queries and is counted in $q_{\text{RO}}(\mathcal{B})$; leaf tag collisions cannot merge the final root transcripts whose tags are compared, since by Corollary 1 distinct (K, N, A) triples force distinct final root transcripts before aggregation values matter.

5.4 Lift Sketch

Section B carries out the flat-sponge lift in two hops. Hop 1 applies the indistinguishability theorem of [BDPVA08] with the bound of Lemma 18: the real permutation interface is replaced by the corresponding flat-RO TW128 construction interface and the [BDPVA08] public-primitive simulator at cost at most $\text{Lift}_c(N_{\text{tot}})$, where $N_{\text{tot}} = N + N_{\text{prim}}$ counts construction-internal sponge calls and direct primitive queries before the derived-adversary construction. Hop 2 defines a derived flat-RO adversary $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ whose primitive queries are answered by running the simulator with direct RO_{flat} queries; this hop is a lossless equality of joint distributions. One inspection of [BDPVA08, Algorithm 3] shows that the simulator issues an RO_{flat} call only on a forward primitive query whose input node is rooted and has no outgoing edge in the simulator's internal graph, whose constructed path is paddable, and only while the simulator is unsaturated (Lemma 17); inverse queries and forward queries violating any of those conditions return RO_{flat} -free outputs. Every primitive query therefore induces at most one RO_{flat} call, giving the key accounting

$$q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}},$$

so the random-oracle envelopes of Section 5.3 are instantiated at N_{prim} , not N_{tot} , in the headline theorems. The same inspection underlies the CMTDs-3 candidate-sequence length bound $m \leq N_{\text{prim}} + s$ of Section 8 and the direct CMTs- ℓ bounds of Section C; the CMTs challenger encryption checks are construction-internal and are counted in N , not in N_{prim} .

5.5 Concrete TW128 Bounds

The implemented parameters of Section 3.2 fix

$$k = c = \tau_{\text{root}} = \tau_{\text{leaf}} = 256.$$

Substituting into Theorems 1 to 3 and using the random-oracle envelopes of Section 5.3 gives the following closed-form bounds.

All-in-one AE.

$$\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{2N_{\text{prim}} + Q_v}{2^{256}} + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{257}}.$$

Multi-user indistinguishability.

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{2DN_{\text{prim}} + Q_v}{2^{256}} + \frac{D(D - 1) + N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{257}}.$$

s-way CMTDs-4 committing security. For every $s \geq 2$,

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{\binom{N_{\text{prim}} + s}{s}}{2^{256(s-1)}}.$$

Remark 2 (Two-Key CMTD-4 Specialization). Setting $s = 2$ recovers the ordinary two-key CMTD-4 statement of [BH22]:

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{\binom{N_{\text{prim}} + 2}{2}}{2^{256}} = \frac{N_{\text{tot}}(N_{\text{tot}} + 1) + (N_{\text{prim}} + 1)(N_{\text{prim}} + 2)}{2^{257}}.$$

5.6 Concrete Security Claim

TW128 attains a 128-bit security target across all three notions of Section 5.5. The birthday term $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{257}$ dominates the AE bound and is a leading term in the committing bounds, where the two-key specialization also has a co-leading root-collision term; it contributes alongside a multi-user hybrid term in the mu-ind bound. The headline work cap $N_{\text{tot}} \leq 2^{63}$ keeps the resulting total below 2^{-128} after the resource consequences of Section 4.3 are applied. Namely, $N_{\text{prim}} \leq N_{\text{tot}}$ by definition, and the pathwise bounds $q_v \leq N$ and $n_{\text{leaf}} \leq N$ allow the envelope parameters Q_v and N_{leaf} to be instantiated at values at most N_{tot} . The mu-ind user cap D remains an independent hypothesis, since created-but-idle users need not contribute to N .

- *AE.* For every nonce-respecting AE adversary against TW128 with $N_{\text{tot}} \leq 2^{63}$ KECCAK- p calls, $\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq 2^{-128}$. The flat-sponge and leaf-collision terms are each bounded by one 2^{-131} -sized birthday term, up to lower-order summands, while $(2N_{\text{prim}} + Q_v)/2^{256} \leq 3 \cdot 2^{-193}$.
- *Multi-user indistinguishability.* For every D -user mu-ind adversary against TW128 with $N_{\text{tot}} \leq 2^{63}$ KECCAK- p calls and $D \leq 2^{63}$ users, $\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq 2^{-128}$. At the threshold, the hybrid term satisfies $2DN_{\text{prim}}/2^{256} \leq 2^{-129} = 4 \cdot 2^{-131}$; the flat-sponge term is at most $2^{-131} + 2^{-194}$; the user-key birthday and leaf-collision terms are each below 2^{-131} ; and $Q_v/2^{256} \leq 2^{-193}$. Their sum is at most $7 \cdot 2^{-131} + 3 \cdot 2^{-194} < 8 \cdot 2^{-131} = 2^{-128}$.
- *Committing security.* For every $s \geq 2$ and every s -way CMTDs-4 adversary against TW128 with $N_{\text{tot}} \leq 2^{63}$ KECCAK- p calls (so in particular $N_{\text{prim}} \leq 2^{63}$), $\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq 2^{-128}$. In the two-key case $s = 2$, the flat-sponge and root-collision terms are each bounded by one 2^{-131} -sized birthday term, up to lower-order summands. For $s \geq 3$, the root-collision term decays as $2^{-256(s-1)}$ in the denominator, so it becomes negligible relative to the birthday term well before N_{tot} saturates.

Expressed in bytes of construction work, $N_{\text{tot}} \leq 2^{63}$ corresponds to processing at most $\rho \cdot 2^{63} = 167 \cdot 2^{63} < 2^{71}$ bytes of plaintext, associated data, ciphertext, and aggregation through the duplex (Section 3.2), well beyond any realistic deployment lifetime. The estimates above are direct instantiations of the closed-form expressions of Section 5.5 using the derived resource bounds from Section 4.3; no separate top-level cap on Q_v or N_{leaf} is required.

5.7 Discussion of the Bounds

The closed-form bounds of Section 5.5 aggregate four loss terms: the flat-sponge differentiating loss Lift_c , the key-guessing contribution KeyGuess_k (entering privacy and INT-CTXT independently, hence the factor of two in the AE and mu-ind bounds), the leaf tag pairwise-collision term, and the verification-tag term. The CMTDs-4 bound substitutes the s -way root-collision term $\text{RootColl}_{\tau_{\text{root}}}$ for the latter three. We comment on dominance and tightness.

Dominance. In the single-user AE bound, the birthday-on-capacity loss $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{257}$ dominates for $6 \leq N_{\text{tot}} < 2^c$. The key-guessing and verification-tag contributions $(2N_{\text{prim}} + Q_v)/2^{256}$ are linear in their resource caps and bounded by $3N_{\text{tot}}/2^{256}$, hence below Lift_c for $N_{\text{tot}} \geq 6$. The leaf tag collision term $N_{\text{leaf}}(N_{\text{leaf}} - 1)/2^{257}$ is at most Lift_c via $N_{\text{leaf}} \leq N_{\text{tot}}$. The CMTDs-4 bound is similar: the $s = 2$ root-collision term equals $(N_{\text{prim}} + 1)(N_{\text{prim}} + 2)/2^{257}$, which is within a small additive N_{tot} -term of Lift_c under $N_{\text{prim}} \leq N_{\text{tot}}$, and it decays as $2^{-256(s-1)}$ in the denominator for $s \geq 3$.

The multi-user mu-ind bound is the exception. For $D \leq N_{\text{tot}}(N_{\text{tot}} + 1)/(4N_{\text{prim}})$ users (typical for any practical deployment), the hybrid-induced term remains at most Lift_c . At the upper end of Section 5.6's parameters ($N_{\text{tot}}, D, N_{\text{prim}} \leq 2^{63}$), the hybrid-induced term $2D \cdot \text{KeyGuess}_k(N_{\text{prim}}) \leq 2DN_{\text{tot}}/2^{256}$ reaches $\approx 4\text{Lift}_c$ and is the dominant term; the user-key birthday $D(D - 1)/2^{257}$ and the leaf tag collision each contribute roughly Lift_c . The 2^{-128} figure of Section 5.6 is an upper bound on the sum of these terms at the threshold.

Tightness.

- $\text{KeyGuess}_k(Q) = Q/2^k$. Tight at constants: a direct random-guess attack on a k -bit key matches the per-query success probability $1/2^k$.
- $\text{RootColl}_{\tau}(Q, s) = \binom{Q+s}{s}/2^{\tau(s-1)}$. Tight up to lower-order terms: it is the standard s -way multi-collision bound on a τ -bit projection of RO_{flat} , and a generic birthday-style adversary against the random oracle matches the leading-order term.
- *Leaf tag collision* $N_{\text{leaf}}(N_{\text{leaf}} - 1)/2^{\tau_{\text{leaf}}+1}$. The pairwise union bound is tight up to constants: a generic birthday attack on the τ_{leaf} -bit leaf tag projection matches the leading $N_{\text{leaf}}^2/2^{\tau_{\text{leaf}}+1}$ rate.
- *Flat-sponge lift*. The term $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$ bounds the exact random-permutation differentiating probability of [BDPVA08, Lemma 5] via Section 5.2. The remaining slack is the unimproved tightness of the [BDPVA08] simulator itself, which is an open problem on the sponge side and not specific to TW128.

Cross-scheme bound-level comparisons against prior sponge AEADs and the committing-AEAD line of [BH22] are deferred to Sections 10.1 to 10.3.

6 Privacy

This section proves the random-oracle privacy bound feeding the all-in-one AE composition of Theorem 1. The public-permutation privacy statement follows by the flat-sponge lift of Section B.

Theorem 4 (Random-Oracle Privacy). *For every nonce-respecting privacy adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B}) \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) = q_{\text{RO}}(\mathcal{B})/2^k.$$

1033 *Equivalently, the resource-indexed envelope satisfies $\varepsilon_{\text{RO}}^{\text{priv}}(Q_{\text{RO}}) \leq \text{KeyGuess}_k(Q_{\text{RO}})$.*

1034 *Proof.* Let P_0 be the real-side random-oracle privacy game: the challenger samples $K \leftarrow \{0, 1\}^k$, $\mathcal{B}$ has direct access to RO_{flat} , and accepted construction-interface queries are
 1035 answered by Enc_K^{RO} . Let P_1 be the corresponding ideal-side game, augmented with an
 1036 independent auxiliary sample $K \leftarrow \{0, 1\}^k$: construction-interface queries are answered
 1037 by Rand , which returns a fresh uniformly random byte string of length $\|M\| + \tau_{\text{root}}/8$, and
 1038 direct queries are answered by the same lazy RO_{flat} interface. The auxiliary key is used
 1039 only for the abort predicate below and is ignored by Rand , so the adversary's marginal
 1040 view is the ideal term in the definition of $\text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B})$. Let q_e be an execution-uniform
 1041 cap on the number of accepted encryption queries made by $\mathcal{B}$, as in Section 4.3; this cap
 1042 indexes the finite replacement hybrid sequence and will not enter the final privacy bound.
 1043

1044 Use the common abort event bad_{key} : a direct adversary query (Y, ℓ) to RO_{flat} satisfies

$$1045 \quad \text{KeyedTWOut}(Y) = (K, \mathcal{C})$$

1046 for this game's key K (the hidden key in P_0 , the auxiliary key in P_1) and some counted
 1047 output-point class $\mathcal{C}$ of Table 7; the requested length ℓ is ignored.

1048 Let P_0^{stop} and P_1^{stop} be the immediate-abort variants of P_0 and P_1 on this event. By
 1049 the Privacy Stopped-Game Advantage Bound (Lemma 12) applied to the event $\mathcal{B} \Rightarrow 1$,

$$1050 \quad \text{Adv}_{\text{RO}}^{\text{priv}}(\mathcal{B}) = |\Pr[\mathcal{B}^{P_0} \Rightarrow 1] - \Pr[\mathcal{B}^{P_1} \Rightarrow 1]| \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})).$$

1051 Taking the supremum over nonce-respecting privacy adversaries with $q_{\text{RO}}(\mathcal{B}) \leq Q_{\text{RO}}$ gives
 1052 the envelope statement $\varepsilon_{\text{RO}}^{\text{priv}}(Q_{\text{RO}}) \leq \text{KeyGuess}_k(Q_{\text{RO}})$. $\square$

1053 **Lineage.** The proof is the flat-random-oracle analog of the standard SpongeWrap privacy
 1054 argument: stop on direct reads of hidden-key construction transcripts, replace each nonce-
 1055 respecting encryption query by fresh random output under transcript freshness, and bound
 1056 the stop event as an adaptive key test. The TW128-specific interfaces are Transcript
 1057 Freshness (Lemma 9) and the privacy accounting lemmas (Lemmas 10 to 12).

1058 7 Ciphertext Integrity

1059 This section proves the random-oracle INT-CTXT bound feeding the all-in-one AE com-
 1060 position of Theorem 1. The public-permutation INT-CTXT statement follows by the
 1061 flat-sponge lift of Section B.

1062 **Theorem 5** (Random-Oracle INT-CTXT). *For every INT-CTXT adversary $\mathcal{B}$ in the*
 1063 *TW128 random-oracle model (Section 4.2),*

$$1064 \quad \text{Adv}_{\text{RO}}^{\text{int-ctxt}}(\mathcal{B}) \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v(\mathcal{B})}{2^{\tau_{\text{root}}}}.$$

1065 *Equivalently, the resource-indexed envelope satisfies*

$$1066 \quad \varepsilon_{\text{RO}}^{\text{int-ctxt}}(Q_{\text{RO}}, N_{\text{leaf}}, Q_v) \leq \text{KeyGuess}_k(Q_{\text{RO}}) + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{Q_v}{2^{\tau_{\text{root}}}}.$$

1067 *Proof.* Let G_0 be the real random-oracle INT-CTXT game: the challenger samples $K \leftarrow \{0, 1\}^k$, $\mathcal{B}$ has direct access to RO_{flat} , and construction queries are answered by Enc_K^{RO} and
 1068 Ver_K^{RO} . Each encryption query (N, A, M) returns $C \parallel T = \text{Enc}_K^{\text{RO}}(N, A, M)$ and records the
 1069 tuple (N, A, C, T) ; $\mathcal{B}$ wins if any verification query returns 1 on a tuple (N, A, C, T) that
 1070 was not previously recorded by the encryption oracle.
 1071

Let bad_{key} be the event that some direct adversary query (Y, ℓ) to RO_{flat} satisfies $\text{KeyedTWOut}(Y) = (K, \mathcal{C})$ for a counted output-point class $\mathcal{C}$ of Table 7; the requested length ℓ is ignored. Let G_1 be G_0 with an immediate abort on bad_{key} . The games are identical until this event, so

$$\Pr[G_0 \text{ wins}] \leq \Pr[G_1 \text{ wins}] + \Pr[\text{bad}_{\text{key}}].$$

Lemma 13 verifies the visible-prefix uniformity invariant required by the Adaptive Key-Test Bound (Lemma 7) in G_1 , with the set of already tested candidate keys recovered by KeyedTWOut . Therefore

$$\Pr[\text{bad}_{\text{key}}] \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})).$$

Let bad_{leaf} be the event that two distinct construction-generated final leaf transcripts under K receive the same τ_{leaf} -bit RF projection. Let G_2 be G_1 with an additional immediate abort on bad_{leaf} . The games are identical until this event, so

$$\Pr[G_1 \text{ wins}] \leq \Pr[G_2 \text{ wins}] + \Pr[\text{bad}_{\text{leaf}} \text{ before } \text{bad}_{\text{key}}].$$

Lemma 14 supplies the event premise for the Leaf Tag Collision Bound (Lemma 8): before bad_{key} , no direct query has pre-read the bridged input of a hidden-key final leaf transcript before the construction first creates that transcript. Thus $\{\text{bad}_{\text{leaf}} \text{ before } \text{bad}_{\text{key}}\}$ is contained in $\{\text{bad}_{\text{leaf}} \text{ before } \text{hit}_{\text{leaf}}\}$, where hit_{leaf} is the event of Lemma 8 that a direct query hits the bridged input of a hidden-key final leaf transcript before the construction first creates it; hence

$$\Pr[\text{bad}_{\text{leaf}} \text{ before } \text{bad}_{\text{key}}] \leq \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}}.$$

Bound on $\Pr[G_2 \text{ wins}]$. In G_2 , every accepting non-replay verification submission belongs to a final root transcript class. If the class is encryption-first, Lemma 15 says the submission cannot be an accepting non-replay; the remaining accepting non-replay submissions are therefore verification-first. By Lemma 16, their total probability before $\text{bad}_{\text{key}} \cup \text{bad}_{\text{leaf}}$ is bounded by the root tag guessing term:

$$\Pr[G_2 \text{ wins}] \leq q_v(\mathcal{B})/2^{\tau_{\text{root}}}.$$

Conclusion. Combining the three stopped-game bounds,

$$\text{Adv}_{\text{RO}}^{\text{int-ctxt}}(\mathcal{B}) = \Pr[G_0 \text{ wins}] \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v(\mathcal{B})}{2^{\tau_{\text{root}}}}.$$

Taking the supremum over INT-CTXT adversaries with $q_{\text{RO}}(\mathcal{B}) \leq Q_{\text{RO}}$, $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$, and $q_v(\mathcal{B}) \leq Q_v$ gives the envelope statement. $\square$

Lineage. The random-oracle decomposition of the bound into a key-guessing term and a tag-guessing term ($q 2^{-k}$ and 2^{-t} in their notation) follows the authenticity bound of the ideal scheme $\mathcal{RO}_{\text{wrap}}[\rho]$ in the SpongeWrap analysis [BDPVA11, Lemma 6], which carries no sponge-capacity term. We realize the key-guessing term as the bad_{key} abort and the tag-guessing term as the residual root tag guessing bound on $\Pr[G_2 \text{ wins}]$, with a TW128-specific leaf tag birthday abort bad_{leaf} inserted between them. The sponge-capacity differentiating loss of [BDPVA11, §5, Theorem 1] does not appear at this random-oracle layer; it enters only through the flat-sponge lift of Section B, as the [BDPVA08] indistinguishability term written $\text{Lift}_{\mathcal{C}}$. The TW128-specific steps come from Transcript Injectivity (Lemma 4), the Leaf Tag Collision Bound (Lemma 8), which bounds the bad_{leaf} abort by a birthday on the τ_{leaf} -bit leaf tag space, and the INT-CTXT auxiliary lemmas (Lemmas 13 to 16).

8 Committing Security

This section proves the random-oracle CMTDs-3 bound that the headline CMTDs-4 statement (Theorem 3) consumes via the trivial promotion of Section C. The public-permutation CMTDs-3 statement follows by the flat-sponge lift of Section B.

Theorem 6 (Random-Oracle CMTDs-3). *For every $s \geq 2$ and every s -way CMTDs-3 adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-3}}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s) = \frac{\binom{q_{\text{RO}}(\mathcal{B})+s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

Proof. Unlike the privacy and INT-CTXT proofs, the CMTDs-3 analysis has no bad_{key} event: all s keys are adversary-supplied, so a direct query to a keyed transcript is one of $\mathcal{B}$'s visible RO_{flat} queries and is counted in $q_{\text{RO}}(\mathcal{B})$.

Run the random-oracle CMTDs-3 game (Section 4.2) and define a candidate sequence for every execution. During $\mathcal{B}$'s direct oracle phase, when $\mathcal{B}$ makes a direct query to an input Y that is not already in the sequence, append Y immediately before answering the query if $\text{KeyedTWOut}(Y)$ decodes to a root tag output-point class, namely $\mathcal{R}_{\text{msg}}^+$ (the single-chunk root tag, with the aggregation phase elided when $n = 0$) or $\mathcal{R}_{\text{tag}}$ (the multi-chunk root tag). This predicate depends on Y alone through the exact decoder of Lemma 5, which ignores the requested output length, so both the decision to append and the string Y are determined before the query's RO_{flat} lookup. By Output-Point Separation (Corollary 2) each input decodes to at most one output-point class, so each distinct direct query contributes at most one candidate. Since there is no construction oracle before $\mathcal{B}$ outputs its tuples, each direct-query candidate is first touched at its append step.

After $\mathcal{B}$ outputs $C \parallel T$ and tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$, stop appending challenger candidates if the syntax, message-length, or pairwise-distinct-triple check fails. Otherwise, as the challenger evaluates the s deterministic decryption checks, let R_i be the final root transcript reached by $\text{Dec}_{K_i}^{\text{RO}}(N_i, A_i, C \parallel T)$ in check i , and append $\text{flat}(R_i)$ immediately before the root tag oracle answer is sampled, unless that input is already in the sequence. If a challenger input was already touched by a direct query, then Lemma 5 and Output-Point Separation (Corollary 2) imply that the earlier query was a final-root candidate and the input is already in the sequence. Otherwise, Corollary 2 excludes all earlier challenger lookups outside the final-root class, and Opening Triple Separation (Corollary 1) excludes earlier final-root lookups from the other checked triples. Thus no earlier lookup of any kind has touched a newly appended candidate's input. The sequence therefore satisfies the predictability and unrevealed-value premises of Adaptive Candidate Collision (Lemma 6), and its length is at most $q_{\text{RO}}(\mathcal{B}) + s$.

On a winning execution, the challenger has accepted the syntax, message-length, and pairwise-distinct-triple checks, so the tuples are valid and the (K_i, N_i, A_i) triples are pairwise distinct. By Corollary 1, the bridged final-root inputs

$$\text{flat}(R_1), \dots, \text{flat}(R_s)$$

are pairwise distinct. Thus leaf tag collisions cannot merge the root inputs relevant to a CMTDs-3 win. Since every decryption check accepts, the τ_{root} -bit projections at the s distinct challenger final-root inputs all equal the supplied tag T :

$$\lfloor \text{RO}_{\text{flat}}(\text{flat}(R_1)) \rfloor_{\tau_{\text{root}}} = \dots = \lfloor \text{RO}_{\text{flat}}(\text{flat}(R_s)) \rfloor_{\tau_{\text{root}}} = T.$$

Hence the win event is contained in the s -way candidate-collision event. Applying Lemma 6 with $Q = q_{\text{RO}}(\mathcal{B})$ and $\tau = \tau_{\text{root}}$ gives

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-3}}(\mathcal{B}) \leq \binom{q_{\text{RO}}(\mathcal{B})+s}{s} \cdot 2^{-\tau_{\text{root}}(s-1)} = \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s).$$

□

9 Performance Evaluation

9.1 Optimized Implementations

We evaluate three implementations of TW128 sharing a common structure: a portable reference written in Go, an `amd64` implementation, and an `arm64` implementation. The two architecture-specific implementations replace only the `KECCAK-p[1600, 12]` permutation and the inner absorb/squeeze loops with hand-written assembly; the framing, domain separation, padding, and tree bookkeeping remain in the shared Go code. The portable reference uses a scalar permutation and a scalar eight-instance loop, and serves both as a correctness oracle for the assembly cores and as the fallback on other architectures.

The tree structure of TW128 exposes two distinct permutation workloads, and both architectures provide a separate core for each. The root duplex—which absorbs the associated data, processes chunk 0, and absorbs the leaf tag aggregation transcript—is inherently sequential, since each of its calls consumes the previous call’s output; it is driven by a *single-duplex* ($\times 1$) core that permutes one 1600-bit state at a time. The leaf duplexes, by contrast, operate on disjoint state and disjoint inputs (Section 3), so the implementation batches eight complete leaf chunks and runs them in lockstep through an *eight-way* ($\times 8$) core. Because every leaf chunk is exactly $\beta = 49\rho$ bytes, the eight instances step through an identical sequence of 49 full ρ -blocks (the first 48 closed with `MSG_MORE`, the last with `MSG_LAST`), so a single shared block position drives all eight. The eight-way state is held lane-major—25 lanes each holding the corresponding 64-bit word of all eight instances—so that the batched cores can address the eight instances of a lane with a single vector load.

Messages are decomposed into eight-chunk batches run on the $\times 8$ core, and the leftover complete chunks are drained by narrower paths. Where a sub-eight kernel exists the remainder is processed in place: on `arm64` a 2-wide kernel consumes it two chunks at a time, and on the AVX-512 path a register-resident kernel handles a two-to-seven-chunk remainder directly when it forms the entire (cache-resident) leaf region. Otherwise a remainder of five to seven chunks is padded to a full batch of eight and run on the $\times 8$ core—faster than repeated single-duplex calls even at that utilization—while any final straggler, a partial trailing chunk, and the count-aggregation tail fall back to the single-duplex ($\times 1$) core. In every batched core the final `MSG_LAST` block and the leaf tag extraction are performed in portable Go, shared with the reference path; the assembly kernels handle only the 48 full `MSG_MORE` blocks, which keeps the byte-granular tail logic out of assembly.

amd64. The `amd64` implementation provides AVX-512 and AVX2 cores and selects between them once at startup from the `AVX512VL` CPUID feature bit. The single-duplex permutation has two variants: a general-purpose-register core adapted from the standard library’s `KECCAK-p` permutation (using the lane-complement trick so that χ requires no separate negation), and an AVX-512 variant; the root duplex uses whichever matches the selected core. The eight-way AVX-512 core packs the lane-major state into 25 ZMM registers, one per lane with all eight instances in the register’s eight 64-bit slots, and performs the permutation with no cross-lane shuffles: rotations use `VPROLQ`, and `VPTERNLOGQ` fuses the θ D -term formation and the χ and-not into single three-input logic instructions. The eight-way AVX2 core, lacking 512-bit registers, runs the eight instances as two groups of four (four 64-bit lanes per YMM register), rotating by shift-or and computing χ with `VPANDN`. The fused chunk kernels combine absorption with the permutation. The AVX-512 steady-state kernel reads the eight chunks with contiguous loads, transposes each rate block into the lane-major layout in registers (`VPUNPCKLQDQ/VPUNPCKHQDQ` and `VSHUFI64X2`), absorbs on the lane-major state, and transposes each output block back for a contiguous store, leaving only the 7-byte partial rate lane to a masked gather; these sequential accesses are prefetcher-friendly, whereas the earlier per-lane gather and scatter at the chunk stride β (`VPGATHERQQ/VPSCATTERQQ`) stalled on cold memory and capped the long-message rate.

The register-resident kernel that drains a small two-to-seven-chunk remainder keeps that masked gather/scatter form, reading the active chunks in place without a scratch buffer.

arm64. The `arm64` path uses NEON plus the Armv8.2 SHA-3 extension (FEAT_SHA3), available on Apple Silicon and on Neoverse V1/V2 and N2. The permutation is expressed with the extension’s fused instructions: `EOR3` (three-input XOR) for the θ column parities, `RAX1` (rotate-by-one and XOR) for the θ D -terms, `XAR` (XOR then rotate) to fuse the θ accumulation with the ρ rotation, and `BCAX` (bit-clear and XOR) for χ . The single-duplex core places one instance per 64-bit D lane across the 25 vector registers; the eight-way core packs two instances per 128-bit register and processes the batch of eight as four such pairs, using `ZIP1` and duplicating moves to interleave and extract the two instances of each lane in the fused chunk kernel. The same pair machinery backs a 2-wide kernel that drains a leaf-chunk remainder two chunks at a time—one instance per 64-bit half of a 128-bit register—at roughly twice the single-duplex per-chunk rate.

Constant time. All three implementations are constant time with respect to the key, nonce, and plaintext. The permutation and the XOR-based absorb/squeeze contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number; the AVX-512 chunk transpose uses contiguous loads and stores, and the gather and scatter that remain—for the partial rate lane and the register-resident remainder kernel—are addressed by the fixed public chunk strides under a mask set by the public chunk count. None of these depend on secret data.

9.2 Benchmark Methodology

We measure on two hosts, one per architecture. The `amd64` host is a Google Compute Engine `c4-standard-4` instance with an Intel Xeon Platinum 8581C (Emerald Rapids, 2.30 GHz base, AVX-512 including AVX512VL, AVX512_VBMI2, GFNI, and VAES), two physical cores with simultaneous multithreading enabled, 48 KiB L1d and 32 KiB L1i per core, 2 MiB L2 per core, and a 260 MiB shared L3, running Debian 12 (kernel 6.1) under KVM. The guest does not expose a `cpufreq` interface, so the frequency governor and turbo state are not under our control; we mitigate the resulting variance statistically (below). The `arm64` host is an Apple M4 Pro (10 performance cores and 4 efficiency cores; per performance core, 128 KiB L1i, 64 KiB L1d, and a 16 MiB shared L2) running macOS 26.5 (Darwin 25.5), with FEAT_SHA3 present. macOS exposes no userspace frequency control.

Measurements are collected by a standalone harness (`cmd/cpb`) that locks its goroutine to an OS thread and, for each algorithm, operation, and message length, first calibrates an iteration count that fills at least 100 ms, then records 21 samples and reports the median cycles per byte and the median wall-clock throughput. Both metrics are read from the same timed loop, so the cycles-per-byte and throughput figures for a given measurement come from one interleaved run rather than two separately scheduled ones. Each sample reuses the same source and destination buffers, so the reported figures are warm-cache, steady-state rates. The single-threaded measurement leaves the SMT sibling of the `amd64` host idle. We report a sweep of seven message lengths from 1 B to 16 MiB.

The cycle counters differ by architecture, and this affects how the cycles-per-byte figures may be read. On `amd64` the harness reads `RDTSC`; on this part the time-stamp counter is invariant and ticks at the 2.30 GHz reference frequency, so the reported figures are *reference* cycles per byte and are unaffected by turbo (and convert directly to wall-clock time). On `arm64` the harness reads `CNTVCT_ELO` and scales it by the ratio of the peak core frequency to `CNTFRQ_ELO`; since Apple exposes no frequency API, the peak is auto-detected by timing a dependent integer-add chain and taking the top observed P-state, yielding

core cycles per byte. Consequently the cycles-per-byte figures are comparable within an architecture and against the same-host AES-128-GCM baseline, but not directly across architectures; absolute throughput (Table 5) is the cross-platform metric.

The baseline is Go’s AES-128-GCM implementation (`crypto/aes` with `crypto/cipher`). It uses AES-NI and PCLMULQDQ on `amd64`, and the AES and PMULL instructions on `arm64`. Each timed AES-128-GCM call constructs its cipher, so its measurement includes the key schedule and the GHASH key-power precomputation; this mirrors the TW128 measurement, which constructs a fresh encryptor (one permutation) per call. Both directions are measured for each scheme; the two directions track each other to within 3% for TW128 and within 5% for AES-128-GCM across the sweep.

9.3 Throughput

Table 4 reports cycles per byte across the message-length sweep and Table 5 the corresponding wall-clock throughput in the bulk regime. The dominant feature of the TW128 rows is the descent produced by the tree-parallel leaf core: the per-byte cost falls as a message lengthens and its leaf chunks move onto progressively wider kernels. A short message is dominated by the serial root duplex over chunk 0; its few complete leaf chunks, when present, run on the sub-eight remainder kernels (Section 9.1), and only a message with at least eight complete leaf chunks beyond chunk 0 (about $9\beta \approx 72$ KiB) fills the eight-way ($\times 8$) core even once. The per-byte cost therefore falls for two compounding reasons: the fixed per-message root cost amortizes over more leaf bytes, and the leaf chunks occupy more SIMD lanes. On the `amd64` AVX-512 path TW128 encryption falls $1.96 \rightarrow 1.23 \rightarrow 0.66$ across the 8, 32, and 64 KiB columns—the register-resident remainder kernel engaging at 32 KiB and filling more of its eight lanes by 64 KiB—then to 0.43 at 1 MiB and 0.40 at 16 MiB as the $\times 8$ steady state takes over. The `amd64` AVX2 path has no sub-eight remainder kernel below five chunks, so it stays on the serial core through 32 KiB (2.18) and reaches wide-SIMD throughput only once a five-to-seven-chunk remainder is padded to eight at 64 KiB (1.49); its steady state is shallower (1.04 at 16 MiB), since it runs the eight instances as two groups of four rather than in a single 8-wide register, leaving the AVX-512 core about $2.6\times$ faster in bulk. The `arm64` core, whose 2-wide kernel engages at 32 KiB and which packs two instances per 128-bit register, narrows the gap more gradually ($1.95 \rightarrow 1.38 \rightarrow 1.14 \rightarrow 0.89 \rightarrow 0.88$ cycles per byte from 8 KiB to 16 MiB). In wall-clock terms TW128 reaches 5.75 GB/s on the `amd64` AVX-512 host and 5.09 GB/s on the M4 Pro for 16 MiB messages.

9.4 Latency

For short messages the cost is set by the serial root duplex and is independent of the parallel leaf machinery. A message of at most $\rho = 167$ bytes occupies a single rate block and never enters leaf mode, so it costs exactly two permutations—one to initialize the root duplex and one to close the message block and extract the tag. The 1 B and 64 B columns of Table 4 bear this out: per-message latency is flat across them at roughly 627 reference cycles on `amd64` (627 at 1 B versus $9.79 \times 64 = 627$ at 64 B) and 620 core cycles on `arm64`, i.e. about 314 and 310 cycles per KECCAK- $p[1600, 12]$ respectively. Because such messages bypass the leaf core entirely, leaf-level parallelism imposes no latency penalty on them.

The latency/throughput trade-off appears instead in the intermediate regime. A message large enough to span a few leaf chunks but too short to fill the eight-way core runs those chunks on the serial core or on a partly-filled remainder kernel, where TW128 sits at its worst cycles-per-byte (1.2–2.3 across the 8 KiB and 32 KiB columns); the per-byte cost reaches its floor only once the message is long enough to keep the $\times 8$ core in its steady state. The root duplex also bounds the tail of a long message: the final tag cannot be produced until chunk 0 has been processed and every leaf tag has been absorbed into the

Table 4: Median cycles per byte (21 samples). The **amd64** figures are RDTSC reference cycles at the 2.30 GHz invariant time-stamp counter; the **arm64** figures are core cycles at the auto-detected peak performance-core frequency. Cycle counts are not directly comparable across architectures (Section 9.2).

	Message length						
	1 B	64 B	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
<i>Intel Xeon 8581C (Emerald Rapids), AVX-512</i>							
TW128 encrypt	627	9.79	1.96	1.23	0.66	0.43	0.40
TW128 decrypt	627	9.92	1.94	1.21	0.65	0.42	0.40
AES-128-GCM seal	769	13.15	0.39	0.33	0.31	0.30	0.30
AES-128-GCM open	789	12.88	0.38	0.32	0.30	0.30	0.30
<i>Intel Xeon 8581C (Emerald Rapids), AVX2</i>							
TW128 encrypt	677	10.84	2.30	2.18	1.49	1.08	1.04
TW128 decrypt	687	10.89	2.27	2.19	1.49	1.08	1.04
<i>Apple M4 Pro, NEON + FEAT_SHA3</i>							
TW128 encrypt	619	9.99	1.95	1.38	1.14	0.89	0.88
TW128 decrypt	621	10.01	2.00	1.41	1.14	0.87	0.86
AES-128-GCM seal	1027	16.66	0.54	0.45	0.44	0.43	0.44
AES-128-GCM open	1026	16.73	0.52	0.43	0.42	0.41	0.42

Table 5: Measured wall-clock throughput (GB/s, GB = 10^9 bytes) from the same `cmd/cpb` run as Table 4, encryption direction; the decryption direction tracks it. The **amd64** AES-128-GCM row is the AVX-512 host and is unaffected by the TW128 core selection.

	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
<i>Intel Xeon 8581C (Emerald Rapids)</i>					
TW128 (AVX-512)	1.17	1.87	3.50	5.39	5.75
TW128 (AVX2)	1.00	1.06	1.54	2.12	2.22
AES-128-GCM	5.95	7.04	7.36	7.60	7.55
<i>Apple M4 Pro</i>					
TW128 (NEON+SHA3)	2.29	3.24	3.92	5.04	5.09
AES-128-GCM	8.29	9.92	10.26	10.42	10.12

aggregation transcript. That absorption is cheap—each rate block of the root absorbs five 32-byte leaf tags, so a 16 MiB message (≈ 2000 leaves) adds on the order of 400 root permutations against roughly 10^5 permutations of leaf work, under 0.5% overhead—so the serial root does not materially cap bulk throughput.

9.5 Comparison with Other AEADs

We compare against the one baseline measured under the same harness on the same hosts, the Go standard library’s hardware-accelerated AES-128-GCM (Tables 4 and 5). The harness times TW128 and AES-128-GCM back-to-back at each length and reads both metrics from the same loop, so the cycles-per-byte and throughput comparisons hold under identical conditions and agree. In the bulk regime AES-128-GCM is faster on both architectures, as expected for a primitive with dedicated AES and carryless-multiply instructions: at 16 MiB it runs at 0.30 cycles per byte on `amd64` and 0.44 on `arm64`, against 0.40 and 0.88 for TW128—about $1.3\times$ and $2\times$ faster. The wall-clock throughputs show the same margins: TW128 reaches 5.75 and 5.09 GB/s against AES-128-GCM’s 7.55 and 10.12 GB/s. The wider `arm64` gap reflects the strength of Apple’s AES and PMULL units. TW128 attains these rates with a permutation that has no dedicated hardware support, relying only on the SHA-3 extension on `arm64` and on general SIMD on `amd64`.

The ordering reverses for very short messages, where AES-128-GCM’s key schedule and GHASH key-power precomputation dominate: at 1 B AES-128-GCM costs 769 and 1027 cycles on the two hosts against TW128’s 627 and 619. This comparison is setup-bound rather than representative of steady state, since each timed call rebuilds its cipher; it nonetheless reflects the per-message cost an application pays when it does not amortize key setup across messages. In the sub-bulk regime, before TW128’s leaf chunks fill the wide core, AES-128-GCM is several times faster (e.g. 0.39 versus 1.96 cycles per byte at 8 KiB on `amd64`); TW128’s throughput advantage is confined to the parallel regime, and its intended value relative to AES-128-GCM lies in the tree structure and committing-security properties analyzed in Sections 6 to 8 rather than in raw speed.

10 Discussion

10.1 Comparison Table

Table 6 situates TW128 against representative authenticated encryption schemes along the axes that distinguish it: the underlying primitive, whether the mode is parallelizable, the key, nonce, and tag sizes, associated-data support, nonce-misuse resistance, and full-input key commitment (CMT-4). The comparison is structural; the concrete forms of TW128’s own privacy, AE, and committing bounds, and the regimes in which each term dominates, are treated in Section 5.7. The CMT-4 column records full-input key commitment: TW128 provides it within the mode, as does Ascon [KSW23], a committing transform supplies it for an arbitrary base scheme, and the remaining listed schemes do not provide it.

10.2 Comparison with Prior Sponge AEADs

TW128 belongs to the family of single-permutation duplex AEADs initiated by [BDPVA11]. The dedicated lightweight members of that family, Ascon [DEMS21] and Xoodyak [DHP⁺20], target 128-bit security on small permutations (320 and 384 bits) with a strictly serial duplex: each call depends on the previous one, so there is no mode-level parallelism to exploit. TW128 instead pays for a larger permutation, KECCAK- p [1600, 12], and a 256-bit capacity—the same round count as KangarooTwelve [BDP⁺16]—in exchange for a tree that processes leaf chunks in parallel and for the headroom that yields 128-bit security with a comfortable margin. The cost of this choice is visible in Section 9:

Table 6: Structural comparison of TW128 with representative AEADs. Sizes are in bytes; AES-GCM-SIV also admits a 32-byte key. “Misuse-res.” is nonce-misuse resistance in the sense of [RS06]; CMT-4 is full-input key-commitment in the sense of [BH22]. A dash means the entry is inherited from the base scheme. TW128’s concrete bounds appear in Section 5.7.

Scheme	Primitive	Par.	K/N/T	AD	Misuse	CMT-4
TW128	KECCAK- p [1600,12]	yes	32/32/32	yes	no	yes
Ascon-128a [DEMS21]	Ascon- p (320 b)	no	16/16/16	yes	no	yes
Xoodyak [DHP ⁺ 20]	Xoodoo (384 b)	no	16/16/16	yes	no	no
AES-GCM-SIV [GLL17]	AES	yes	16/12/16	yes	yes	no
nAE + CTX [CR22]	any nAE	—	—	yes	—	yes

on short, single-leaf inputs TW128 is slower per byte than a compact serial duplex would be, and its advantage appears only once a message spans enough leaves to fill the vector units.

Keyak [BDP⁺15] is the closest prior duplex AEAD in spirit, since its Motorist mode is already parallelizable. As discussed in Section 1.2, the difference is that Keyak fixes its degree of parallelism as a mode parameter that the encryptor and decryptor must share, whereas in TW128 the number of leaves evaluated at once is an implementation choice invisible to the ciphertext. Strobe [Ham17] is a serial sponge framework aimed at whole protocols rather than at a standalone AEAD, and like Ascon and Xoodyak it does not parallelize. Committing security is not unique to TW128 in this family: a dedicated analysis of the sponge-based NIST lightweight finalists proves Ascon committing while breaking others [KSW23], and some constructions, such as Strobe, have not been examined in that setting at all. What distinguishes TW128 is that it targets the strongest s -way full-input notion (CMT-4) as a proven, designed property of the mode, established without leaving the single-permutation duplex setting.

10.3 Comparison with Committing AEAD Constructions

The dominant route to committing AEAD is a generic transform over a non-committing base scheme. [BH22] give the UtC, RtC, and HtE transforms, which add key- or context-commitment to an arbitrary AEAD, and [CR22] give CTX, which makes any nonce-based AE scheme commit to its full context at the cost of a single hash over a short string and with no ciphertext expansion. These transforms are inexpensive and broadly applicable; CTX in particular is nearly free. TW128 differs not by undercutting their cost but by integration: it is committing as the mode itself, with the same permutation and the same tag that already provide confidentiality and integrity, so no separate commitment primitive, key-derivation step, or auxiliary hash is invoked, and the strongest s -way full-input notion follows from a single root tag collision bound (Section 8). This places TW128 alongside the dedicated committing primitives—the compactly committing AEAD of [GLR17] and the encryption of [DGRW18]—which likewise build commitment in rather than bolt it on, but which were designed for message franking rather than as general nonce-based AEADs. The contrast with transformed conventional schemes is sharpened by the attacks of Section 1.2: base schemes such as GCM and OCB are not committing on their own [CR22], and a transform is the only way to make them so.

Closest to TW128 here is the recent work of [DHM⁺24], which builds nonce-based AE directly on the SHA-3 functions SHAKE and TurboSHAKE—the latter using the same round-reduced KECCAK- p permutation as TW128—and likewise achieves CMT-4 natively, from the collision resistance of the underlying function rather than from a transform. The two designs make different structural choices: [DHM⁺24] targets session-based communication, chaining a stream of messages through intermediate tags and generalizing the keyed

1390 duplex into a *duplex cipher*, whereas TW128 is a single two-level tree whose independent
 1391 leaves expose run-time-selectable parallelism at constant memory (Sections 1.2 and 9.1).
 1392 Both demonstrate that committing AE needs no add-on transform once it is built on a
 1393 well-scrutinized KECCAK- p permutation.

1394 10.4 Limitations and Open Problems

1395 **Nonce misuse.** TW128’s privacy analysis requires a nonce-respecting adversary, and
 1396 the mode is not misuse-resistant: a repeated nonce reuses leaf keystream and breaks
 1397 confidentiality, as for any online duplex stream cipher. Schemes such as AES-GCM-SIV
 1398 [GLL17], in the deterministic/misuse-resistant lineage of [RS06], tolerate nonce repetition
 1399 at the price of a second pass. A misuse-resistant variant of TW128 is left open.

1400 **Release of unverified plaintext.** The analysis assumes that plaintext is released only
 1401 after the tag verifies. TW128 is online and single-pass, so an implementation that streams
 1402 decrypted leaf output before checking the root tag operates in the release-of-unverified-
 1403 plaintext setting of [ABL⁺14], which our bounds do not cover; plaintext-awareness and
 1404 INT-RUP security for TW128 are unstudied.

1405 **Bound tightness.** Two terms are knowingly conservative. The flat-sponge differentiating
 1406 loss $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$ bounds the indistinguishability loss of [BDPVA08],
 1407 used here as a generic lift from the random-oracle model. A direct analysis of the keyed
 1408 duplex is tighter: the modern keyed-duplex bounds synthesized by [Men23], building on the
 1409 full-state keyed-duplex line, bound a keyed duplex against an ideal permutation without
 1410 passing through indistinguishability, and recasting TW128’s leaf and root duplexes in that
 1411 framework is a promising route to a sharper capacity term. This route is not turnkey,
 1412 however: the keyed duplex bundles squeezing and absorption into a single call, so its
 1413 authenticated-encryption mode **MonkeySpongeWrap** has the decryptor *overwrite* the rate
 1414 with each ciphertext block rather than feed back the ciphertext it has just produced. On the
 1415 final, padded block this overwrite no longer matches the encryptor’s XOR-absorbed padded
 1416 plaintext outside the message bits, so the decryptor cannot reconstruct the final block of
 1417 duplex output and recomputes the tag from a divergent state [Men23, Alg. 8]. TW128
 1418 sidesteps this by absorbing the ciphertext in both directions (Section 3), so recasting it on
 1419 the keyed duplex would first require repairing the mode. We retain the indistinguishability
 1420 lift for its modularity, as the resulting Lift_c already meets the 128-bit target (Section 5.7).
 1421 The leaf tag collision term $N_{\text{leaf}}(N_{\text{leaf}} - 1)/2^{7\text{leaf}+1}$ is a pairwise union bound whose necessity
 1422 for the AE guarantee, as opposed to an artifact of the accounting, is not settled.

1423 **Beyond-birthday and leakage.** All bounds are birthday-type on the 256-bit capacity,
 1424 delivering the 128-bit target but no more; a beyond-birthday analysis is open. The model
 1425 is also single-stage and leakage-free: side-channel resistance is addressed only at the
 1426 implementation level, through the constant-time properties of Section 9.1, and is not part
 1427 of the provable-security claims.

1428 11 Conclusion

1429 TW128 is a two-level tree of keyed duplexes over a single KECCAK- p [1600, 12] permutation
 1430 that combines mode-level parallelism, constant memory, and committing security. We
 1431 analyzed it in the public-permutation model, proving concrete bounds for all-in-one
 1432 authenticated encryption, multi-user indistinguishability, and the strongest s -way full-
 1433 input committing notion (CMTDs-4) by reasoning first against an intermediate random
 1434 oracle and then lifting through the flat-sponge transform; at the implemented parameters

all three meet a 128-bit target. The accompanying `amd64` and `arm64` implementations turn the tree's leaf parallelism into multi-gigabyte-per-second throughput, within a small factor of hardware-accelerated AES-128-GCM and with a ciphertext that is independent of the degree of parallelism actually used.

Several questions remain open (Section 10.4): a nonce-misuse-resistant variant, security under the release of unverified plaintext, and sharper bounds—whether by recasting the leaf and root duplexes on the modern keyed-duplex analysis or through a beyond-birthday argument. More broadly, TW128 adds to the growing evidence that a single, well-scrutinized KECCAK- p permutation suffices to build authenticated encryption that is at once fast, parallel, and committing.

Acknowledgments

A hearty thank-you to my wife, who remains reasonably mystified at my choice of hobbies.

References

- [ABL⁺14] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Nicky Mouha, and Kan Yasuda. How to securely release unverified plaintext in authenticated encryption. In *Advances in Cryptology – ASIACRYPT 2014*, volume 8873 of *Lecture Notes in Computer Science*, pages 105–125. Springer, 2014.
- [ADG⁺22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308. USENIX Association, 2022.
- [BDP⁺15] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. CAESAR submission: Keyak v2. CAESAR Competition, Round 3 Candidate, 2015.
- [BDP⁺16] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. KangarooTwelve: Fast hashing based on Keccak-p. *Cryptology ePrint Archive*, Paper 2016/770, 2016.
- [BDPVA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indistinguishability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.
- [BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In *Selected Areas in Cryptography – SAC 2011*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.
- [BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13276 of *Lecture Notes in Computer Science*, pages 845–875. Springer, 2022.
- [BN00] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *Advances in Cryptology – ASIACRYPT 2000*, volume 1976 of *Lecture Notes in Computer Science*, pages 531–545. Springer, 2000.

- [BT16] Mihir Bellare and Björn Tackmann. The multi-user security of authenticated encryption: AES-GCM in TLS 1.3. In *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *Lecture Notes in Computer Science*, pages 247–276. Springer, 2016.
- [CR22] John Chan and Phillip Rogaway. On committing authenticated encryption. In *Computer Security – ESORICS 2022*, Lecture Notes in Computer Science. Springer, 2022.
- [DEMS21] Christoph Dobraunig, Maria Eichseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.
- [DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryptment. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 155–186. Springer, 2018.
- [DHM⁺24] Joan Daemen, Seth Hoffert, Silvia Mella, Gilles Van Assche, and Ronny Van Keer. Shaking up authenticated encryption. Cryptology ePrint Archive, Paper 2024/1618, 2024.
- [DHP⁺20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodoo, a lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology*, 2020(S1):60–87, 2020.
- [GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Specification and analysis. Cryptology ePrint Archive, Paper 2017/168, 2017.
- [GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 66–97. Springer, 2017.
- [Ham17] Mike Hamburg. The STROBE protocol framework. Cryptology ePrint Archive, Paper 2017/003, 2017.
- [IR23] Takanori Isobe and Mostafizar Rahman. Key committing security analysis of AEGIS. Cryptology ePrint Archive, Paper 2023/1495, 2023.
- [KSW23] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges: Security analysis of the NIST LWC finalists. Cryptology ePrint Archive, Paper 2023/1525, 2023.
- [LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 195–212. USENIX Association, 2021.
- [Men23] Bart Mennink. Understanding the duplex and its security. *IACR Transactions on Symmetric Cryptology*, 2023(2):1–46, 2023.
- [MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. Context discovery and commitment attacks: How to break CCM, EAX, SIV, and more. In *Advances in Cryptology – EUROCRYPT 2023*, Lecture Notes in Computer Science. Springer, 2023.

- [Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication FIPS PUB 202, U.S. Department of Commerce, August 2015.
- [RS06] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390. Springer, 2006.
- [WP14] Hongjun Wu and Bart Preneel. AEGIS: A fast authenticated encryption algorithm. In *Selected Areas in Cryptography – SAC 2013*, volume 8282 of *Lecture Notes in Computer Science*, pages 185–201. Springer, 2014.

A Supporting Lemmas

The proof core separates structural decoding from probabilistic accounting. The well-formed transcript language of Definition 1 fixes which flattened queries are valid construction prefixes. Lemmas 3 to 5 then show how such a query is flattened, parsed as a typed transcript, assigned an output point, and decoded to a candidate key, when any. Lemma 7 turns those decoded keys into an adaptive key-test bound. The remaining proof interfaces expose the events and stopped-game equalities their consumers must control: privacy freshness and hybrid replacement, INT-CTXT hidden-key exposure and root tag guessing, leaf tag collisions, and the committing candidate-sequence collision event. Thus the privacy, INT-CTXT, and committing proofs only have to derive the relevant local event premise before applying the corresponding accounting argument.

A.1 Duplex Flattening

Lemma 3 (Duplex Flattening). *Every TW128 root or leaf duplex output is equal to a [BDPVA11] sponge output on the corresponding flattened duplex-call transcript, and the ciphertext-absorbing message phase admits a flattened duplex transcript that is well defined for both encryption and decryption.*

Proof. Let D be a duplex object using permutation π , rate r , and padding pad10^*1 . For a sequence of duplex calls $Z_i = D.\text{duplexing}(\sigma_i, \ell_i)$ with each σ_i short enough to fit in one duplex block after padding, [BDPVA11, Lemma 3] gives

$$Z_i = \text{Sponge}[\pi, \text{pad10}^*1, r](\sigma_0 \parallel \text{pad}_0 \parallel \sigma_1 \parallel \text{pad}_1 \parallel \cdots \parallel \sigma_i, \ell_i),$$

where $\text{pad}_j = \text{pad10}^*1[r](|\sigma_j|)$, and Lemma 4 says that, for fixed pad10^*1 and r , the map from the duplex-call sequence to the flattened sponge input is injective.

In TW128, each duplex call absorbs a byte-aligned block σ followed by a 4-bit phase suffix $\sigma_{\text{ph}} \in \{\sigma_{\text{init}}, \sigma_{\text{ad}}^-, \dots, \sigma_{\text{agg}}^+\}$ (Section 3.4), so the duplex-call input is $\sigma \parallel \sigma_{\text{ph}}$ followed by pad10^*1 . With $r = 1344$ and $\rho_{\text{max}}(\text{pad10}^*1, r) = r - 2 = 1342$ bits, the maximum byte-aligned input is $\rho = 167$ bytes (Section 3.2), giving a worst-case duplex-call input of $8 \cdot 167 + 4 = 1340 \leq 1342$ bits. The two initialization calls are shorter still: $|p_{\text{root}} \parallel K \parallel N| + 4 = 8 \cdot (6 + 32 + 32) + 4 = 564$ bits and $|p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8| + 4 = 8 \cdot (6 + 32 + 32 + 8) + 4 = 628$ bits. [BDPVA11, Lemmas 3 and 4] therefore apply independently to every TW128 duplex object.

The root duplex is initialized by absorbing $p_{\text{root}} \parallel K \parallel N$ under σ_{init} (Algorithm 2), then absorbs the associated-data blocks under $\sigma_{\text{ad}}^- / \sigma_{\text{ad}}^+$, the root ciphertext blocks in the message phase under $\sigma_{\text{msg}}^- / \sigma_{\text{msg}}^+$, and the leaf tag aggregation blocks under $\sigma_{\text{agg}}^- / \sigma_{\text{agg}}^+$. By the duplex-to-sponge equality, every root duplex output is the sponge function evaluated

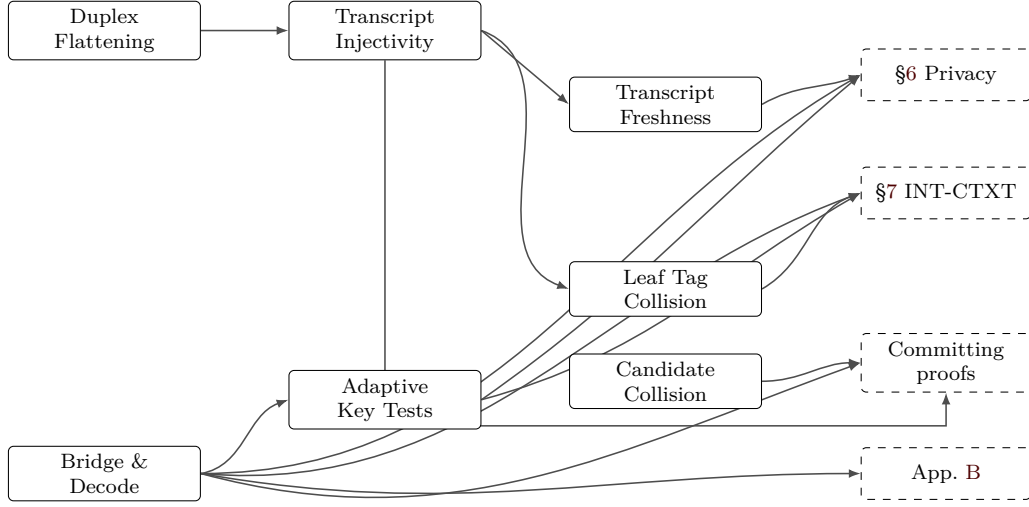


Figure 2: Dependency graph of the supporting lemmas in this appendix and their consumers. Solid boxes are lemmas; dashed boxes are the proof sections that consume them. The edge from Lemma 4 into the committing proofs of Sections C and 8 is mediated by Corollary 1 for final-root distinctness and Corollary 2 for candidate-sequence filtering; the edge from Lemma 6 records the collision accounting used by the CMTDs-3 and CMTs proofs. The edge from Lemma 4 into Lemma 9 uses Corollary 2; the verification-first root tag guessing accounting reached through the same separation is carried out in Section A.10 (Lemma 16) and consumed by Section 7. The Lemma 7 edge records the shared bad_{key} accounting used by privacy and INT-CTXT.

1564 on the root transcript prefix accumulated up to the relevant output point. The same
 1565 holds for each leaf duplex, initialized by $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ and absorbing only its own
 1566 ciphertext blocks in the message phase. In decryption and verification, Algorithm 3 absorbs
 1567 the submitted ciphertext blocks before the root tag comparison, so the same flattened
 1568 message-phase transcript is defined whether the comparison accepts or rejects. $\square$

1569 A.2 Well-formed Transcript Prefixes

1570 For a duplex-call prefix $((\sigma_0, \sigma_{\text{ph},0}), \dots, (\sigma_i, \sigma_{\text{ph},i}))$, write

$$1571 \quad \text{Flat}((\sigma_0, \sigma_{\text{ph},0}), \dots, (\sigma_i, \sigma_{\text{ph},i})) = \left\|_{j=0}^{i-1} (\sigma_j \parallel \sigma_{\text{ph},j} \parallel \text{pad}_j) \parallel \sigma_i \parallel \sigma_{\text{ph},i}.$$

1572 The current call's padding pad_i is not part of Flat; it is applied internally by the sponge
 1573 function. Equivalently, after the i -th call the duplex state is the sponge state after absorbing
 1574 $\text{Flat}(\cdot) \parallel \text{pad}_i$.

1575 **Definition 1** (Well-Formed TW128 Transcript Prefixes). A typed TW128 transcript prefix
 1576 is *well formed* if it is a prefix, ending at a construction transcript lookup point, of one of
 1577 the following two call languages.

- 1578 • A root transcript starts with $(p_{\text{root}} \parallel K \parallel N, \sigma_{\text{init}})$, with fixed-width fields $K \in \{0,1\}^k$
 1579 and $N \in \{0,1\}^\nu$. It then absorbs the associated-data byte string (only when the
 1580 associated data is nonempty; the phase is otherwise elided, Section 3.5), the root

- 1581 ciphertext byte string, and the aggregation byte string (only when $n \geq 1$; when
 1582 $n = 0$ the aggregation phase is elided, Section 3.5, and the root tag is emitted by the
 1583 final σ_{msg}^+ message call), in that order. Each present phase byte string is partitioned
 1584 by the block rule of Algorithm 1: the empty string gives one empty final block, and
 1585 a nonempty string gives zero or more ρ -byte non-final blocks followed by one final
 1586 block. The phases use respectively $\sigma_{\text{ad}}^-/\sigma_{\text{ad}}^+$, $\sigma_{\text{msg}}^-/\sigma_{\text{msg}}^+$, and $\sigma_{\text{agg}}^-/\sigma_{\text{agg}}^+$.
- 1587 • A leaf- j transcript starts with $(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8, \sigma_{\text{init}})$, where $1 \leq j < 2^{64}$ and
 1588 the fields have fixed width. It then absorbs the leaf ciphertext byte string under
 1589 $\sigma_{\text{msg}}^-/\sigma_{\text{msg}}^+$, using the same block rule.

1590 The message byte string in this language is always the ciphertext byte string absorbed by the
 1591 construction: encryption obtains it by XORing the plaintext with the current keystream,
 1592 while decryption and verification use the submitted ciphertext. Thus a verification
 1593 transcript is well formed for every in-domain ciphertext before the tag comparison is made;
 1594 acceptance is not part of the transcript language.

1595 The root aggregation byte string has the form $T_1 \parallel \dots \parallel T_n \parallel \langle n \rangle_8$, where each T_j has
 1596 fixed byte length $\tau_{\text{leaf}}/8$. The trailing fixed-width field therefore gives a unique parse of
 1597 the leaf tag sequence. The output-point class of a well-formed counted prefix is the class
 1598 in Table 7 determined by its duplex instance and final-call suffix.

1599 Name the tag-emitting transcript families:

1600 $\mathcal{R}_{\text{tag}}$ = root transcript prefix whose sponge output gives the final root tag,
 1601 $\mathcal{L}_{\text{tag}}[j]$ = leaf- j transcript prefix whose sponge output gives the leaf tag.

1602 The final root tag is emitted by the σ_{agg}^+ aggregation call (the $\mathcal{R}_{\text{tag}}$ family) when $n \geq 1$,
 1603 and by the closing σ_{msg}^+ message call (an $\mathcal{R}_{\text{msg}}^+$ prefix, in the notation of Corollary 2) when
 1604 $n = 0$ and the aggregation phase is elided. Since $n = \text{leaves}(C)$ is fixed by $\|C\|$, every
 1605 computation on a given ciphertext emits its tag at the one of these two output points
 1606 selected by $\|C\|$. Intermediate construction transcript lookup points (init, non-final AD,
 1607 non-final message, and non-final aggregation) are also typed prefixes covered by Lemma 5;
 1608 the exact keyed direct-query decoder names them in Table 7.

1609 A.3 Transcript Injectivity

1610 **Lemma 4** (Transcript Injectivity). *Well-formed TW128 flattened duplex-call transcript*
 1611 *prefixes are injectively encoded and separated by duplex instance (root vs leaf), leaf index*
 1612 *j , phase suffix, and duplex-call order. For final-root prefixes, equality of flattened inputs*
 1613 *recovers the same root initialization, AD blocks, root ciphertext blocks, aggregation input,*
 1614 *and leaf tag sequence. When the compared final-root prefixes are construction-generated,*
 1615 *if no two distinct construction-generated final leaf transcripts produce the same leaf tag,*
 1616 *equality also recovers the same leaf transcript sequence and therefore the same full absorbed*
 1617 *ciphertext.*

1618 *Proof.* By Lemma 3, every well-formed output point can be written as a [BDPVA11]
 1619 flattened sponge input for the corresponding duplex-call prefix, namely Flat as defined
 1620 above. The deterministic current-call pad10^*1 is applied internally by the sponge function
 1621 and is not part of the input to which [BDPVA11, Lemma 4] is applied. That lemma
 1622 recovers the duplex-call sequence from the flattened sponge input; in TW128, each call
 1623 input is $\sigma \parallel \sigma_{\text{ph}}$ with byte-aligned σ and fixed 4-bit suffix σ_{ph} , so recovering the call input
 1624 recovers the pair $(\sigma, \sigma_{\text{ph}})$.

1625 Definition 1 fixes the root and leaf initialization fields, the admissible phase order,
 1626 and the final-call suffixes. The distinct prefixes p_{root} and p_{leaf} separate root from leaf
 1627 transcripts; the fixed-width leaf field $\langle j \rangle_8$ separates leaves; and the seven suffix values of

Table 3 separate phases and non-final/final calls. Thus equality of flattened inputs forces equality of the recovered instance type, key, nonce, leaf index when present, phase suffixes, and duplex-call order.

Phase-recovery sub-claim. For each present phase $P \in \{\text{ad}, \text{msg}, \text{agg}\}$ (the ad phase is present exactly when the associated data is nonempty; see Section 3.5), TW128 absorbs the phase’s input byte string s_P by partitioning into ρ -byte blocks (with the empty case yielding a single empty block, per Section 3.5) and feeding each block to the duplex with suffix σ_P^- (non-final) or σ_P^+ (final). Lemma 3 and [BDPVA11, Lemma 4] recover the corresponding $(\sigma, \sigma_{\text{ph}})$ sequence from the flattened input. Because the partition map is deterministic and uniquely invertible on its image (the empty case yielding the single empty block carrying σ_P^+), concatenating the recovered σ blocks reconstructs s_P exactly. The map from s_P to the phase’s $(\sigma, \sigma_{\text{ph}})$ sequence is therefore injective.

Applied phase by phase to a well-formed final-root transcript:

- **AD.** The associated-data phase contributes a σ_{ad}^+ -tagged block exactly when the associated data is nonempty; an empty associated data elides the phase and contributes no block. When the phase is present the sub-claim recovers the AD byte string. Thus distinct associated-data strings $A_i \neq A_j$ are separated either by the presence versus absence of the AD phase (when exactly one is empty) or, when both are nonempty, by their distinct AD σ -block sequences.
- **MSG.** Distinct absorbed root or leaf ciphertext byte strings produce distinct MSG σ -block sequences; the message language of Definition 1 is ciphertext-absorbing for encryption and decryption alike.
- **AGG.** The aggregation phase is present exactly when $n \geq 1$ (Section 3.5). When it is present, the sub-claim applied to the aggregation byte string $T_1 \parallel \dots \parallel T_n \parallel \langle n \rangle_8$ recovers that byte string. The final eight bytes give $\langle n \rangle_8$, and each leaf tag has fixed byte length $\tau_{\text{leaf}}/8$, so the byte string has a unique parse as a leaf tag sequence. Two root aggregation encodings are therefore equal exactly when their leaf tag sequences are equal. Under the structural condition that distinct construction-generated final leaf transcripts receive distinct leaf tag values, equality of leaf tag sequences implies equality of the corresponding final leaf transcripts. When $n = 0$ the aggregation phase is absent and the final-root transcript ends at the recovered σ_{msg}^+ root message call; the MSG bullet recovers the root ciphertext byte string, and the absence of any $\sigma_{\text{agg}}^-/\sigma_{\text{agg}}^+$ call distinguishes such a prefix from every $n \geq 1$ final-root prefix.

Therefore the recovered final-root transcript determines the root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the aggregation byte string and leaf tag sequence. For construction-generated final-root prefixes with $n \geq 1$, under the stated leaf tag structural condition, the leaf tag sequence determines the corresponding final leaf transcripts as well, so the same full absorbed ciphertext is recovered; for $n = 0$ the root ciphertext blocks already constitute the full absorbed ciphertext. $\square$

Bridge injectivity on transcript prefixes. For TW128, the [BDPVA11, Theorem 3] bridge has $r = r_{\text{max}}$ and maps a native flattened input U to $U \parallel 1 \parallel 0^q$, where $q = (-|U| - 2) \bmod r$. Any string in this image has a unique inverse: q is the number of trailing zero bits, the preceding bit is the inserted 1, and deleting this suffix recovers U . Thus equality of bridged images $\text{flat}(X) = \text{flat}(X')$ for well-formed typed TW128 prefixes implies equality of their native flattened inputs, after which Lemma 4 gives the same recovery consequences as above.

Corollary 1 (Opening Triple Separation). *If two TW128 root computations have distinct (K, N, A) triples, then their bridged final root transcript inputs under $\text{flat}(\cdot)$ are distinct,*

1676 *regardless of whether each computation is an encryption or a decryption check on a common*
 1677 *ciphertext.*

1678 *Proof.* Suppose for contradiction that $\text{flat}(R_i) = \text{flat}(R_j)$. By the bridge-injectivity obser-
 1679 *vation* above, the native flattened inputs of the two final-root prefixes are equal. Then
 1680 Lemma 4 recovers the full duplex-call sequence and reads off the pair $(\sigma, \sigma_{\text{ph}})$ for each
 1681 call. The seven 4-bit phase suffixes are pairwise distinct, so the σ_{init} , $\sigma_{\text{ad}}^{\pm}$, $\sigma_{\text{msg}}^{\pm}$, and (when
 1682 present) $\sigma_{\text{agg}}^{\pm}$ calls in the recovered sequence are unambiguously identified. The aggregation
 1683 phase is present exactly when $n \geq 1$, and $n = \text{leaves}(C)$ is determined by the common
 1684 ciphertext, so the two recovered sequences either both carry an aggregation phase or both
 1685 omit it; either way the recovery of (K, N, A) below uses only the σ_{init} and AD calls, which
 1686 precede any message or aggregation block.

1687 Equality of the recovered sequences forces equality of the σ_{init} call, whose σ is $p_{\text{root}} \| K \| N$
 1688 with fixed-width K and N , hence $K_i = K_j$ and $N_i = N_j$. It also forces equality of the
 1689 recovered AD subsequences. By the AD bullet of Lemma 4 the associated-data phase
 1690 contributes a σ_{ad}^+ -tagged block exactly when the associated data is nonempty, so the
 1691 recovered sequences agree on the presence of the AD phase. If exactly one of A_i, A_j were
 1692 empty the recovered sequences would differ in the presence of a σ_{ad}^+ call, contradicting
 1693 their equality; hence either both are empty, giving $A_i = A_j = \varepsilon$, or both are nonempty, in
 1694 which case the phase-recovery sub-claim of Lemma 4 inverts the ρ -byte-block partition
 1695 map on the AD byte string and equal AD σ -block sequences imply $A_i = A_j$. In all cases
 1696 $(K_i, N_i, A_i) = (K_j, N_j, A_j)$, contradicting the hypothesis of distinct triples. $\square$

1697 **Corollary 2** (Output-Point Separation). *Every well-formed TW128 typed transcript prefix*
 1698 *at a counted construction transcript lookup point belongs to exactly one of the following*
 1699 *output-point classes, identified by the recovered duplex instance and the 4-bit suffix of its*
 1700 *final call:*

- 1701 • $\mathcal{R}_{\text{init}}(\text{root duplex}, \text{suffix } \sigma_{\text{init}})$,
- 1702 • $\mathcal{L}_{\text{init}}(j)$ (leaf- j duplex, suffix σ_{init}),
- 1703 • $\mathcal{R}_{\text{ad}}^- / \mathcal{R}_{\text{ad}}^+$ (root duplex, suffix σ_{ad}^- or σ_{ad}^+),
- 1704 • $\mathcal{R}_{\text{msg}}^- / \mathcal{R}_{\text{msg}}^+$ (root duplex, suffix σ_{msg}^- or σ_{msg}^+),
- 1705 • $\mathcal{L}_{\text{msg}}^-(j)$ (leaf- j duplex, suffix σ_{msg}^-),
- 1706 • $\mathcal{L}_{\text{tag}}[j]$ (leaf- j duplex, suffix σ_{msg}^+),
- 1707 • $\mathcal{R}_{\text{agg}}(\text{suffix } \sigma_{\text{agg}}^-)$,
- 1708 • $\mathcal{R}_{\text{tag}}$ (root duplex, suffix σ_{agg}^+).

1709 *For any two such prefixes whose bridged flattened inputs are equal, the recovered duplex-call*
 1710 *sequences are equal, and therefore:*

- 1711 (i) *the prefixes have the same (K, N) pair, recovered from the first call's σ ;*
- 1712 (ii) *the prefixes have the same instance type (root or leaf), separated by p_{root} vs p_{leaf} in*
 1713 *the first call's σ ;*
- 1714 (iii) *within leaf instances, the prefixes have the same chunk index j , separated by the*
 1715 *fixed-width $\langle j \rangle_8$ field;*
- 1716 (iv) *the prefixes have the same output-point class, since the seven phase suffixes of the*
 1717 *final call are pairwise distinct;*

(v) within a single output-point class on the same duplex instance, the prefixes have the same duplex-call sequence, since two distinct prefixes in the same class differ in the contents of some recovered call.

Equivalently, any two distinct counted transcript prefixes are separated by the first of (i)–(v) that applies.

Proof. Items (i)–(iv) restate the recovery established by Definition 1 and Lemma 4: bridge injectivity first recovers equality of native flattened inputs from equality of bridged inputs, and then Lemma 4 recovers the duplex-call sequence. The first call’s σ has the form $p_{\text{root}} \parallel K \parallel N$ for root instances and $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ for leaf instances, with fixed-width K , N , and $\langle j \rangle_8$; the seven phase suffixes are pairwise distinct. Item (v) is the contrapositive: two distinct well-formed prefixes in the same class on the same duplex instance differ in some recovered call’s σ or in the sequence length, and therefore in their flattened inputs. $\square$

A.4 Bridge and Typed Decoding

Lemma 5 (Bridge and Typed Decoding). *The random-oracle games used in Sections C and 6 to 8 are ordinary random-oracle games on a single random oracle $\text{RO}_{\text{flat}} : \{0, 1\}^* \rightarrow \{0, 1\}^\infty$ over the unpadded [BDPVA08] H-input domain (Section 4.2), with TW128 root and leaf duplex outputs answered by the typed restriction*

$$\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X)), \quad \text{flat}(X) = I[r, r_{\text{max}}](\text{raw_flat}(X)),$$

for $r = r_{\text{max}} = 1344$. Here $\text{raw_flat}(X)$ is the native flattened input $\text{Flat}(\cdot)$ of Section A.2 applied to the duplex-call sequence of X . The bridge $I[r, r_{\text{max}}]$ is the [BDPVA11] Theorem 3 preprocessing map, and the induced map $X \mapsto \text{flat}(X)$ is injective on well-formed typed TW128 duplex transcript prefixes. The exact decoder KeyedTWOut defined below maps any direct-query input to either $\perp$ or a unique candidate key and counted output-point class.

Proof. The bridge $I[r, r_{\text{max}}]$ is described by the proof of [BDPVA11, Theorem 3] in three steps: (1) pad the input with $\text{pad10}^*1[r]$; (2) for each r -bit block, append $r_{\text{max}} - r$ zero bits; (3) strip the trailing pad10^* from the result. The pad10^*1 padding in step (1) appends a 1, then $q = (-|\text{raw_flat}(X)| - 2) \bmod r$ zero bits, then a closing 1; the pad10^* -unpadding in step (3) strips the $r_{\text{max}} - r$ trailing zero bits introduced by step (2) together with the closing 1 of step (1). For TW128 with $r = r_{\text{max}}$, step (2) appends an empty string and step (3) strips zero trailing zeros and the closing 1, so the net effect of steps (1) and (3) is to append $1 \parallel 0^q$. The bridge-injectivity paragraph above gives the direct inverse on this image: count the trailing zeros, delete them and the preceding inserted 1, and recover the native flattened input.

Injectivity of flat on well-formed typed TW128 prefixes then follows from Lemma 4: distinct well-formed typed prefixes have distinct $\text{raw_flat}(\cdot)$ images, and the bridge is injective on that image. Direct adversary queries on inputs outside the bridged TW128 well-formed transcript language still count in $q_{\text{RO}}(\mathcal{B})$ (Section 4.3) but cannot trigger bad_{key} .

Counted transcript lookup points are the well-formed prefixes of Definition 1 whose classes are listed in Table 7: root or leaf initialization, root AD blocks, root and leaf message blocks, root aggregation blocks, and the final root tag point. Corollary 2 separates these output points, so an input accepted by KeyedTWOut fixes at most one candidate key and one output-point class, and inputs outside this well-formed transcript language decode to none. $\square$

Exact keyed decoder.

For a direct query to RO_{flat} with input Y and finite length ℓ , define $\text{KeyedTWOut}(Y)$ as a partial map from $\{0,1\}^*$ to $\{0,1\}^k \times \mathcal{C}_{\text{out}}$. The map ignores ℓ . It returns $(K, \mathcal{C})$ iff there exists a typed TW128 duplex-call prefix X such that $Y = \text{flat}(X)$, the raw flattened prefix of X parses as a well-formed TW128 transcript prefix in the class $\mathcal{C}$ listed in Table 7, and the first call of X contains the candidate key K in the fixed-width root field $p_{\text{root}} \parallel K \parallel N$ or leaf field $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$. If no such prefix exists, $\text{KeyedTWOut}(Y) = \perp$. In particular, inputs with the right final suffix but a malformed phase order, noncanonical block partition, malformed aggregation string, or out-of-range leaf index are outside the well-formed language and decode to $\perp$.

Table 7: Counted keyed transcript classes for KeyedTWOut .

Class $\mathcal{C}$	Instance	Final call / role
$\mathcal{R}_{\text{init}}$	root	σ_{init} , root initialization
$\mathcal{R}_{\text{ad}}^-$	root	σ_{ad}^- , non-final AD block
$\mathcal{R}_{\text{ad}}^+$	root	σ_{ad}^+ , final AD block
$\mathcal{R}_{\text{msg}}$	root	σ_{msg} , non-final root message block
$\mathcal{R}_{\text{msg}}^+$	root	σ_{msg}^+ , final root message block
$\mathcal{R}_{\text{agg}}$	root	σ_{agg}^- , non-final aggregation block
$\mathcal{R}_{\text{tag}}$	root	σ_{agg}^+ , final aggregation / root tag
$\mathcal{L}_{\text{init}}(j)$	leaf j	σ_{init} , leaf initialization
$\mathcal{L}_{\text{msg}}^-(j)$	leaf j	σ_{msg}^- , non-final leaf message block
$\mathcal{L}_{\text{tag}}[j]$	leaf j	σ_{msg}^+ , final leaf message block / leaf tag

The table is a transcript-lookup table, not an observation-length table. A class is counted even when the construction’s requested output length at that call is zero, including root initialization, non-final AD and aggregation calls, a final AD call whose following root chunk requests no keystream bytes, and the final root message call. A direct query triggers the same decoder regardless of ℓ : short finite-output queries and zero-length queries $(Y, 0)$ still count as direct queries to the address Y . Leaves have no AD or aggregation classes; a leaf transcript enters the table only through leaf initialization, non-final leaf message blocks, or the final leaf tag point. When the associated data is empty the root transcript likewise has no $\mathcal{R}_{\text{ad}}^-$ or $\mathcal{R}_{\text{ad}}^+$ class (Section 3.5); the $\mathcal{R}_{\text{init}}$ class then supplies the first root keystream block, exactly as $\mathcal{L}_{\text{init}}(j)$ supplies the first leaf keystream block, and the decoder still ignores ℓ . Symmetrically, when $n = 0$ the root transcript has no $\mathcal{R}_{\text{agg}}$ or $\mathcal{R}_{\text{tag}}$ class (Section 3.5); the final root message call $\mathcal{R}_{\text{msg}}^+$ then supplies the root tag, exactly as $\mathcal{L}_{\text{tag}}[j]$ supplies a leaf tag, and the decoder again ignores ℓ . An $n \geq 1$ computation also reaches an $\mathcal{R}_{\text{msg}}^+$ point when closing chunk 0, but there the construction requests zero output and continues into the aggregation phase, so the $\mathcal{R}_{\text{msg}}^+$ value is exposed as a tag only on the $n = 0$ path; since $n = \text{leaves}(C)$ is fixed by $\|C\|$, the two uses never coincide on a single ciphertext.

A.5 Adaptive Candidate Collisions

Lemma 6 (Adaptive Candidate Collision). *Fix $s \geq 2$, a projection length τ , and a cap Q . Consider any interaction with a lazy random oracle $\text{RO}_{\text{flat}} : \{0,1\}^* \rightarrow \{0,1\}^\infty$ that builds a sequence of distinct candidate inputs $Y_1, \dots, Y_m$ with $m \leq Q + s$. The process may make arbitrary non-candidate oracle queries and may choose each candidate adaptively from the full prior transcript. Whenever a new candidate Y_j is appended, two premises hold: predictability—both the decision to append and the string Y_j are determined by the transcript preceding the append step—and unrevealed value—no oracle lookup before the append step has requested a positive number of bits of $\text{RO}_{\text{flat}}(Y_j)$; zero-length lookups at Y_j are permitted. Let $\text{coll}_{s,\tau}$ be the event that some s distinct candidates have equal τ -bit*

projections:

$$\lfloor \text{RO}_{\text{flat}}(Y_{i_1}) \rfloor_\tau = \dots = \lfloor \text{RO}_{\text{flat}}(Y_{i_s}) \rfloor_\tau \quad \text{for some } 1 \leq i_1 < \dots < i_s \leq m.$$

Then

$$\Pr[\text{coll}_{s,\tau}] \leq \binom{Q+s}{s} \cdot 2^{-\tau(s-1)}.$$

Proof. For a fixed position set $I = \{i_1 < \dots < i_s\} \subseteq \{1, \dots, Q+s\}$, let E_I be the event that these candidate positions exist and their τ -bit projections are all equal. Expose the interaction in chronological order. Condition on any positive-probability prefix immediately before candidate position i_r is appended, for $r \geq 2$, and on the values sampled at the earlier candidate positions in I . By the predictability premise, that prefix determines Y_{i_r} ; by the unrevealed-value premise, no bit of $\text{RO}_{\text{flat}}(Y_{i_r})$ has been revealed in it, so under lazy sampling the value of $\text{RO}_{\text{flat}}(Y_{i_r})$ may be sampled after the prefix is fixed and is uniform and independent of the conditioning. Its τ -bit projection therefore equals the already fixed projection of Y_{i_1} with probability $2^{-\tau}$. Multiplying over $r = 2, \dots, s$ gives

$$\Pr[E_I] \leq 2^{-\tau(s-1)}.$$

If some position in I is never appended, then E_I is false, so the same bound applies. There are $\binom{Q+s}{s}$ possible position sets, and a union bound over them gives the claim. $\square$

A.6 Adaptive Key Tests

Lemma 7 (Adaptive Key-Test Bound). *Consider a TW128 random-oracle-model game with hidden $K \leftarrow \mathcal{S} \{0,1\}^k$, direct adversary access to RO_{flat} , and an immediate abort event bad_{key} that fires on a direct query (Y, ℓ) exactly when $\text{KeyedTWOut}(Y) = (K, \mathcal{C})$ for some counted output-point class $\mathcal{C}$ of Table 7. Suppose that, before the abort, the following invariant holds for every direct query position i : after conditioning on the full visible execution prefix and on the set S_i of distinct decoded candidate keys from earlier direct queries, the hidden key is uniform on $\{0,1\}^k \setminus S_i$. Then*

$$\Pr[\text{bad}_{\text{key}}] \leq \text{KeyGuess}_k(q_{\text{RO}}) = q_{\text{RO}}/2^k,$$

where q_{RO} is the execution-uniform cap on direct RO_{flat} queries.

Proof. If $q_{\text{RO}} \geq 2^k$ the right-hand side is at least one, so the claim is trivial. Assume $q_{\text{RO}} < 2^k$.

Order the direct RO_{flat} queries. By the definition of KeyedTWOut and Lemma 5, each query input either decodes to $\perp$ or decodes to a unique candidate key and counted output-point class. Queries that decode to $\perp$, and repeated queries whose decoded key already lies in the current set S_i , do not change the set of possible first successful key tests. Removing them can only shorten the sequence, so it suffices to analyze the subsequence of distinct fresh decoded candidate keys $K'_1, \dots, K'_m$, where $m \leq q_{\text{RO}}$.

Let F_i be the event that the first i fresh decoded candidate keys all miss the hidden key. The invariant says that, conditioned on the visible prefix before the $(i+1)$ -st fresh test and on F_i , the hidden key is uniform over the $2^k - i$ keys not yet tested. The next fresh candidate key K'_{i+1} is one of those remaining keys and is determined by the adversary's next direct query, so

$$\Pr[K = K'_{i+1} \mid F_i] = \frac{1}{2^k - i}, \quad \Pr[F_{i+1} \mid F_i] = 1 - \frac{1}{2^k - i}.$$

Multiplying the miss probabilities gives

$$\Pr[F_m] = \prod_{i=0}^{m-1} \left(1 - \frac{1}{2^k - i}\right) = \prod_{i=0}^{m-1} \frac{2^k - i - 1}{2^k - i} = \frac{2^k - m}{2^k}.$$

Thus the probability that some fresh decoded candidate key equals the hidden key is $m/2^k \leq q_{\text{RO}}/2^k$. This event is exactly the first bad_{key} abort, because KeyedTWOout returns either $\perp$ or a unique candidate key. $\square$

A.7 Leaf Tag Collision Bound

Lemma 8 (Leaf Tag Collision Bound). *Consider any TW128 random-oracle-model game with a hidden key, an adversary $\mathcal{B}$ with direct access to RO_{flat} , and construction computations that may generate final leaf transcripts; $n_{\text{leaf}}(\mathcal{B})$ denotes the execution-uniform count of distinct construction-generated final leaf transcripts of Section 4.3. Let hit_{leaf} be the event that, before some construction-generated keyed final leaf transcript is first created, a direct adversary query to RO_{flat} has already hit the bridged input of that hidden-key leaf transcript. Let bad_{leaf} be the event that two distinct construction-generated keyed final leaf transcript prefixes $X \neq X'$ in leaf tag output-point classes receive the same τ_{leaf} -bit projection from $\text{RF}(\cdot)$. Then the probability that bad_{leaf} occurs before hit_{leaf} is bounded by*

$$\Pr[\text{bad}_{\text{leaf}} \text{ before } \text{hit}_{\text{leaf}}] \leq \binom{n_{\text{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\text{leaf}}} = \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}}.$$

Proof. Construction-generated means generated by encryption computations or by INT-CTXT verification computations (whether they accept or reject). Repeated evaluations of the same final leaf transcript are not new collision opportunities, since the lazy-sampled RO_{flat} value is reused.

Run the relevant game and stop the leaf-collision accounting if hit_{leaf} occurs. The execution-uniform resource $n_{\text{leaf}}(\mathcal{B})$ of Section 4.3 bounds the number of distinct final leaf transcripts. Order those transcripts by first creation. At the moment such a transcript is first created by a construction computation before the stop, no earlier direct RO_{flat} query has hit the bridge image of that hidden-key transcript by definition of hit_{leaf} . Nor has an earlier construction computation queried the same RO_{flat} input under a different well-formed transcript: Lemma 4 separates distinct final leaf transcript encodings, and the [BDPVA11, Theorem 3] bridge is injective. The τ_{leaf} -bit leaf tag projection of $\text{RF}(X)$ for that transcript sampled at that moment is therefore uniform and independent of the entire prior game view, including the prior RO_{flat} table on other inputs, prior ciphertext and tag outputs, and the leaf tag projections assigned to all distinct prior final leaf transcripts.

For any fixed unordered pair of distinct final leaf transcripts created before this stop, the probability that their leaf tags are equal is $2^{-\tau_{\text{leaf}}}$. There are at most $\binom{n_{\text{leaf}}(\mathcal{B})}{2}$ such pairs, and a union bound gives the claim. By Lemma 4, conditioned on no leaf tag collision, equal root aggregation inputs identify the same leaf transcript sequence. $\square$

A.8 Transcript Freshness

Lemma 9 (Transcript Freshness). *For any fixed nonce-respecting encryption query in the TW128 random-oracle experiment, let $\text{hit}_{\text{fresh}}$ be the event that, before this query reaches some counted transcript point, a prior direct adversary query to RO_{flat} has already hit the bridged input of that hidden-key point. On $\neg \text{hit}_{\text{fresh}}$, every transcript output of the query that contributes to a returned ciphertext block, to an internal leaf tag, or to the final root tag is fresh at the moment its RO_{flat} value is sampled.*

Proof. Work on an execution outside $\text{hit}_{\text{fresh}}$ for the fixed query. This rules out prior adversary reads of the hidden-key construction inputs considered below. It remains to rule out repeats from previous construction-interface outputs and from the current encryption query.

First root keystream block. The first root keystream block, when present, is produced by the root duplex after the initialization call $p_{\text{root}} \parallel K \parallel N$ and any AD absorption (elided when $A = \varepsilon$, in which case the σ_{init} call itself emits the first root keystream block). Since the privacy adversary never repeats a nonce, no previous encryption query can have used the same root initialization transcript. Within the current query, Lemma 4 separates initialization, AD, message, and aggregation calls by their suffixes and by the duplex-call encoding, so the first root keystream transcript is fresh. If the requested keystream length is zero, no ciphertext block is returned at that point and there is nothing to prove for that output.

First leaf keystream block. For each leaf chunk $j \geq 1$, the first leaf keystream block is produced after the initialization call $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$. The prefix separates leaves from the root, the nonce separates this encryption query from all previous encryption queries, and $\langle j \rangle_8$ separates leaves within this query. Hence every first leaf keystream transcript is fresh.

Later message outputs. Argue inductively within each root or leaf message stream. Suppose the current message-output transcript is fresh. If its output is used as a keystream block, its RO_{flat} value Z_i is, by freshness of the lazy sample, uniform conditioned on the full prior view; since the adaptive plaintext block M_i is a deterministic function of that prior view,

$$C_i = M_i \oplus Z_i$$

is uniform conditioned on the prior view. TW128 then absorbs C_i with the appropriate message-phase suffix (σ_{msg}^- or σ_{msg}^+) before requesting the next keystream block. Once C_i is sampled, the next transcript is fixed; Lemma 4's injectivity and domain separation imply it has not appeared earlier unless the same duplex-call prefix has appeared earlier. Nonce uniqueness excludes this across encryption queries. By Corollary 2, the next message-keystream output point can coincide with a within-query earlier output point only if both belong to the same output-point class on the same duplex instance (root or leaf- j with matching j) and have the same duplex-call sequence; the current message-stream prefix is strictly longer than every earlier prefix in that stream, so this is excluded. Across distinct instances within the same query (root vs leaf, or two leaves with distinct j), the prefixes already differ in the first call's σ by $p_{\text{root}}/p_{\text{leaf}}$ or $\langle j \rangle_8$ respectively. Each later message-output transcript that emits keystream is therefore fresh when sampled. The terminal message-output transcript of a leaf emits the leaf tag rather than a following keystream block; the same separation argument applies to that transcript, so each internal leaf tag is sampled at a fresh point. Empty inputs still induce a duplex call, but with zero keystream output they contribute no ciphertext block.

Final root tag transcript. The final root tag is sampled at the root tag output point selected by $\|C\|$: the $\mathcal{R}_{\text{tag}}$ point after the root duplex absorbs the aggregation transcript built from the leaf tags and $\langle n \rangle_8$ when $n \geq 1$, and the $\mathcal{R}_{\text{msg}}^+$ point closing the root message phase when $n = 0$ (Section 3.5). When $n \geq 1$, the leaf tags are themselves RO_{flat} outputs on final leaf transcripts; condition on their sampled values, on the sampled ciphertext, and on the adversary-visible prior view and RO_{flat} table state, so that the root aggregation transcript is fixed. When $n = 0$ there is no aggregation transcript; condition on the sampled ciphertext and the adversary-visible prior view alone. By Corollary 2, the selected root tag output-point class is separated from every other output-point class in the same query — including root initialization (suffix σ_{init}), AD-phase outputs, the remaining message-phase outputs, leaf-side output points (separated by p_{leaf}), and, in the $n \geq 1$ case, earlier $\mathcal{R}_{\text{agg}}$ -prefix output points (separated by the σ_{agg}^- vs σ_{agg}^+ suffix). Even if two encryption queries realize the same aggregation transcript (or, for $n = 0$, the same

root ciphertext), the full root transcript also contains the root initialization $p_{\text{root}} \parallel K \parallel N$, and nonce uniqueness separates that initialization from all previous encryption queries. The final root tag transcript is therefore fresh, and its τ_{root} -bit RO_{flat} output is uniform independently of the conditioned information. $\square$

A.9 Privacy Accounting Lemmas

The random-oracle privacy proof of Theorem 4 uses Transcript Freshness and the Adaptive Key-Test Bound through the privacy-specific interfaces below. These interfaces track the hidden key through the set of key values still viable after the visible stopped-game prefix. The stopped privacy games use the event bad_{key} from Theorem 4: a direct query (Y, ℓ) aborts when $\text{KeyedTWOOut}(Y) = (K, \mathcal{C})$ for the hidden or auxiliary key K and a counted output-point class $\mathcal{C}$. The stopped games inspect this predicate before returning the direct-oracle answer for the offending query; if it holds, no answer for that query is included in the adversary's view.

Lemma 10 (Privacy Viable-Key Prefix). *Fix a nonce-respecting privacy adversary and an index $i \geq 0$. Let H_i be the stopped privacy hybrid that samples $K \leftarrow \$ \{0, 1\}^k$, answers the first i accepted encryption queries by Rand , answers all later accepted encryption queries by Enc_K^{RO} , answers direct RO_{flat} queries by the lazy table, and aborts on bad_{key} . Condition on any positive-probability visible prefix that includes the submitted $(i+1)$ -st accepted encryption query but not its oracle answer, and on no earlier bad_{key} . Let S be the set of distinct keys K' for which some prior direct query input Y satisfies $\text{KeyedTWOOut}(Y) = (K', \mathcal{C})$ for a counted output-point class $\mathcal{C}$, and let*

$$\mathcal{V} = \{0, 1\}^k \setminus S.$$

Then the hidden key is uniform on $\mathcal{V}$ under this conditioning. Moreover, for every $K' \in \mathcal{V}$, the online execution of $\text{Enc}_{K'}^{\text{RO}}$ on the pending query reaches no counted transcript point whose bridged input was previously queried directly by the adversary. The bridged transcript inputs reached for two distinct keys in $\mathcal{V}$ are disjoint.

Proof. Before the $(i+1)$ -st accepted encryption query in H_i , the accepted construction-interface queries, if any, were answered by Rand . Thus the visible encryption answers and the lazy answers to direct RO_{flat} queries are independent of the hidden key except through the immediate-abort rule. A prior direct query aborts exactly when its decoded candidate key equals the hidden key. Conditioning on the visible prefix and on no earlier abort therefore removes exactly the keys in S , leaving the hidden key uniform on $\{0, 1\}^k \setminus S = \mathcal{V}$.

Fix $K' \in \mathcal{V}$ and run the pending encryption computation online, with each later transcript address determined by the already fixed query input and by the outputs sampled earlier in this same computation. If a counted transcript point reached in this execution had appeared as a prior direct query input Y , then Lemma 5 would give $\text{KeyedTWOOut}(Y) = (K', \mathcal{C})$ for that point's counted class $\mathcal{C}$, so K' would lie in S , contradicting $K' \in \mathcal{V}$. Hence no prior direct query has hit any counted point reached by the pending computation for a viable key. Finally, if two reached bridged transcript inputs for keys $K'_0 \neq K'_1$ were equal, bridge injectivity and Lemma 4 would recover the same first duplex call. That call contains the fixed-width key field, forcing $K'_0 = K'_1$, a contradiction. $\square$

Lemma 11 (Privacy One-Query Replacement). *Fix a nonce-respecting privacy adversary and an index $i \geq 0$. Let H_i be the stopped privacy hybrid that samples $K \leftarrow \$ \{0, 1\}^k$, answers the first i accepted encryption queries by Rand , answers all later accepted encryption queries by Enc_K^{RO} , answers direct RO_{flat} queries by the lazy table, and aborts on bad_{key} . On executions with fewer than $i+1$ accepted encryption queries, the two hybrids coincide. Then*

1982 H_i and H_{i+1} can be coupled so that the adversary's visible transcript and the abort indicator
 1983 are identical. In particular, they have identical joint distributions of the adversary's view
 1984 and the abort indicator.

1985 *Proof.* Use the same adversary coins, the same key K , and the same answers to all direct
 1986 queries and encryption queries before the $(i + 1)$ -st accepted encryption query. If the
 1987 adversary never makes that query, the coupling is immediate. Otherwise condition on
 1988 any common positive-probability visible prefix that includes the submitted query but not
 1989 its oracle answer, and on no earlier abort. Let the query be (N, A, M) , and let $\mathcal{V}$ be the
 1990 viable key set of Lemma 10 for this prefix.

1991 By Lemma 10, the hidden key is uniform on $\mathcal{V}$, and every viable key has no prior direct
 1992 hit on any counted transcript point reached by the pending encryption computation. For
 1993 the actual key in H_i , this no-hit condition is precisely the event $\neg \text{hit}_{\text{fresh}}$ of Lemma 9 for the
 1994 pending computation: any prior direct query that hit a counted transcript point reached
 1995 by the computation would, by Lemma 5, be decoded by **KeyedTWOOut** to the hidden key
 1996 K and would already have triggered bad_{key} . Applying Lemma 9 gives freshness for every
 1997 construction output of the query that contributes to the returned ciphertext, to an internal
 1998 leaf tag, or to the final root tag. Applying the same transcript freshness argument to
 1999 each $K' \in \mathcal{V}$, and using the disjointness of key-indexed transcript inputs from Lemma 10,
 2000 enumerate all such outputs in the order taken by the encryption computation.

2001 Thus, to describe the conditional law, introduce an auxiliary online family of uniform
 2002 coins indexed by a viable key and by the output position reached in that key's encryption
 2003 computation. These auxiliary coins are sampled independently of the hidden key. If the
 2004 hidden key is K' , the H_i computation reads the K' -row and installs only those row entries
 2005 in the lazy table, at the transcript addresses actually reached by $\text{Enc}_{K'}^{\text{RO}}$. The coins for other
 2006 viable keys are not lazy-table entries; they only witness that every possible hidden-key row
 2007 induces the same distribution on the returned string. A later direct query to a nonactual
 2008 viable-key address is therefore answered by the ordinary shared lazy table in both hybrids,
 2009 not by a coin sampled during this replacement coupling.

2010 Each plaintext block is fixed before the corresponding keystream coin is drawn, so
 2011 XOR with that coin makes the ciphertext block uniform. Once a ciphertext block is fixed,
 2012 the next message transcript is fixed and fresh at its sampling time. When $n \geq 1$, leaf tags
 2013 are sampled from fresh coins as well; after these tags and the ciphertext are fixed, the
 2014 aggregation transcript is fixed and the final root tag is sampled from another fresh coin
 2015 at the $\mathcal{R}_{\text{tag}}$ point. When $n = 0$ the aggregation phase is elided and the final root tag is
 2016 instead the fresh coin at the closing $\mathcal{R}_{\text{msg}}^+$ root message point. Hence the returned string
 2017 (C, T) is jointly uniform of length $\|M\| + \tau_{\text{root}}/8$ and independent of K conditioned on
 2018 the prefix. In the coupling, the **Rand** answer in H_{i+1} is sampled as this same independent
 2019 string.

2020 It remains to compare the lazy-table state exposed after this query. The entries that
 2021 may be written in H_i but not in H_{i+1} are construction-internal entries at hidden-key
 2022 counted transcript inputs for nonce N . A later direct adversary query to any such input
 2023 would be decoded by **KeyedTWOOut** to K and would trigger bad_{key} before returning an
 2024 oracle answer, so the possibly different table entry is never exposed. Later real encryption
 2025 queries use nonces different from N by nonce-respectingness, and their typed construction
 2026 inputs are therefore disjoint from this query's inputs by Corollary 2. Thus the extra
 2027 lazy-table entries are unobservable before the abort. Continuing the coupling with shared
 2028 answers on all exposed direct-oracle inputs and all later real construction inputs gives
 2029 identical visible transcripts and identical abort indicators. $\square$

2030 **Lemma 12** (Privacy Stopped-Game Advantage Bound). *Let P_0 and P_1 be the unstopped*
 2031 *real and ideal privacy games of Theorem 4, with P_1 carrying the auxiliary key used only*
 2032 *for the abort predicate, and let $\mathsf{P}_0^{\text{stop}}$ and $\mathsf{P}_1^{\text{stop}}$ be their immediate-abort variants. Assume*

the adversary $\mathcal{B}$ has an execution-uniform cap q_e on accepted encryption queries. Then for any event E determined by the adversary's final output in the unstopped games,

$$|\Pr[E \text{ in } P_0] - \Pr[E \text{ in } P_1]| \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})).$$

Proof. Stopped-game equality. Define hybrids $H_0, \dots, H_{q_e}$ as in Lemma 11. Then $H_0 = P_0^{\text{stop}}$. Since every accepted encryption query is among the first q_e such queries, H_{q_e} is P_1^{stop} . Applying Lemma 11 for $i = 0, \dots, q_e - 1$ gives the same joint distribution of the adversary's view and the abort indicator in adjacent hybrids, and telescoping gives the same joint distribution in the two stopped games. Write

$$p = \Pr[\text{bad}_{\text{key}} \text{ in } P_1^{\text{stop}}];$$

by the stopped-game equality the abort probability is also p in P_0^{stop} . For each side $b \in \{0, 1\}$, the unstopped game P_b and its stopped variant P_b^{stop} agree exactly until the first bad_{key} direct query and have the same occurrence probability for that event. Hence, for the final-output event E ,

$$\Pr[E \text{ in } P_b] = \Pr[E \wedge \neg \text{bad}_{\text{key}} \text{ in } P_b^{\text{stop}}] + x_b$$

for some $0 \leq x_b \leq p$. The first terms are equal for $b = 0$ and $b = 1$ by stopped-game equality, while $x_0, x_1 \in [0, p]$, so

$$|\Pr[E \text{ in } P_0] - \Pr[E \text{ in } P_1]| \leq p.$$

Abort bound. It remains to show $p \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B}))$. Let P_1^{aux} be the non-aborting version of the ideal privacy game: it samples the auxiliary key $K \leftarrow \$\{0, 1\}^k$, answers construction queries by fresh random strings exactly as Rand does, answers direct RO_{flat} queries by the ordinary lazy table, and never aborts. For each direct query input Y_i , Lemma 5 either rejects Y_i as outside the bridged well-formed TW128 transcript language or recovers a unique candidate key from the decoded keyed transcript prefix.

The visible-prefix uniformity invariant required by Lemma 7 holds in P_1^{aux} . Condition on any positive-probability visible execution prefix immediately before a direct query, on no earlier bad_{key} , and on the set S_i of distinct candidate keys decoded from prior direct-query inputs. Construction responses are fresh random strings, direct RO_{flat} answers are lazy random-oracle values, and both are independent of the auxiliary key K ; the no-earlier-abort condition removes exactly the keys in S_i . Thus K remains uniform on $\{0, 1\}^k \setminus S_i$.

The first bad_{key} event in P_1^{stop} has the same probability as the first successful adaptive key test in P_1^{aux} , because the two games are identical up to that query. Applying Lemma 7 with $q_{\text{RO}} = q_{\text{RO}}(\mathcal{B})$ gives $p \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B}))$. Combining with the stopped-game bound proves the claim. $\square$

A.10 INT-CTXT Accounting Lemmas

The INT-CTXT proof of Theorem 5 uses the generic key-test and leaf-collision accounting lemmas together with the INT-specific facts below. The *root tag output point* of a computation is the output point that emits its root tag: $\mathcal{R}_{\text{tag}}$ (suffix σ_{agg}^+) when $n \geq 1$, and the closing root message point $\mathcal{R}_{\text{msg}}^+$ (suffix σ_{msg}^+) when $n = 0$ and the aggregation phase is elided (Section 3.5). Because $n = \text{leaves}(C)$ is fixed by $\|C\|$, all computations on a common ciphertext share the same root tag output point. A *final root transcript class* is the set of construction computations whose encryption or verification computation realizes the same bridged final root input at this root tag output point. Such a class is *encryption-first* if its first construction generation is by an encryption query, and *verification-first* if its first construction generation is by a non-replay verification query.

Lemma 13 (INT Visible-Prefix Key Uniformity). *In the random-oracle INT-CTXT game stopped at bad_{key} as in Theorem 5, order the direct RO_{flat} queries. For a direct query position i , let S_i be the set of distinct candidate keys decoded by KeyedTWOOut from earlier direct-query inputs. Condition on any positive-probability visible execution prefix immediately before query i , on no earlier bad_{key} , and on S_i . Then the hidden key K is uniform on $\{0, 1\}^k \setminus S_i$. Consequently the uniformity invariant required by Lemma 7 holds in the stopped INT-CTXT game.*

Proof. The visible prefix contains the adversary’s oracle queries and replies: direct RO_{flat} outputs, encryption outputs and the resulting replay table, verification bits, and the adversary state determined by these values. Fix such a prefix and two keys $K_0, K_1 \notin S_i$.

Define a transcript-renaming map $\Phi_{0 \rightarrow 1}$ on typed construction prefixes by replacing the first-call key field K_0 by K_1 and leaving every other recovered field unchanged: the nonce, leaf index, phase sequence, AD blocks, ciphertext blocks, leaf tag bytes appearing in root aggregation, and the root/leaf instance identifier are preserved. Let $\Phi_{1 \rightarrow 0}$ be the inverse map. By Lemma 5 and Corollary 2, $\text{flat} \circ \Phi_{0 \rightarrow 1} \circ \text{flat}^{-1}$ is a bijection between the bridged construction-addresses whose decoded key is K_0 and those whose decoded key is K_1 ; it preserves output-point classes and equality relations among addresses.

This bijection is applied online, not to a pre-existing fixed address set. Whenever a construction computation under K_0 would first query a hidden-key address Y , the coupled computation under K_1 queries $\Phi_{0 \rightarrow 1}(Y)$ and receives the same lazy-table value; repeated queries are coupled consistently through the same map. If the queried address is a root aggregation address containing leaf tag bytes, those bytes are already coupled equal because the corresponding leaf tag lookups were either repeated or first sampled earlier under the same renaming rule. Thus adaptively generated message transcripts and root aggregation transcripts are renamed step by step with the same visible ciphertext, leaf tag, and root tag bytes.

Prior direct-query inputs are not renamed: they are adversary-chosen visible strings. None lies in either hidden-key construction-address set for K_0 or K_1 , because any prior direct query to such an address would be decoded by KeyedTWOOut to K_0 or K_1 and would put that key in S_i . Direct-query lazy-table values are therefore coupled identically and remain disjoint from the renamed hidden construction addresses.

Use the same adversary coins, the same direct-query lazy-table values, and the online renamed construction-internal lazy-table values. Encryption outputs have the same distribution under the coupling, because every construction lookup that determines ciphertext, leaf tags, or the final root tag is paired with a distinct renamed lookup carrying the same sampled value. Replay verification decisions depend only on the visible replay table. Non-replay verification computations are renamed in the same online way as encryption computations, and the final root tag comparison therefore gives the same accept/reject bit. Verification does not reveal plaintext on rejection or acceptance; it reveals only that bit.

Thus every visible prefix with hidden key $K_0 \notin S_i$ has the same conditional probability as the corresponding execution with hidden key $K_1 \notin S_i$. Since K is initially uniform and the condition “no earlier bad_{key} ” removes exactly the keys in S_i , all keys in $\{0, 1\}^k \setminus S_i$ remain equally likely. $\square$

Lemma 14 (INT Leaf Freshness Before BadKey). *In the stopped INT-CTXT game of Theorem 5, let hit_{leaf} be the event of Lemma 8: before some construction-generated hidden-key final leaf transcript is first created, a direct adversary query has already hit its bridged input. Then hit_{leaf} cannot occur before bad_{key} . Hence*

$$\Pr[\text{bad}_{\text{leaf}} \text{ before } \text{bad}_{\text{key}}] \leq \Pr[\text{bad}_{\text{leaf}} \text{ before } \text{hit}_{\text{leaf}}].$$

Proof. A hidden-key final leaf transcript is one of the counted leaf classes of Table 7. By Lemma 5, any direct query to its bridged input is decoded by KeyedTWOOut to the hidden

key K and the corresponding $\mathcal{L}_{\text{tag}}[j]$ class, independently of the requested output length. Such a direct query triggers bad_{key} immediately. Therefore no execution prefix strictly before bad_{key} contains a hit_{leaf} event. Hence, on every execution path where bad_{leaf} occurs before bad_{key} (including paths where bad_{key} never occurs), bad_{leaf} occurs before hit_{leaf} . $\square$

Lemma 15 (INT Encryption-First Root Replay). *In the random-oracle INT-CTXT game, before bad_{leaf} , no accepting non-replay verification submission belongs to an encryption-first final root transcript class.*

Proof. Fix a verification submission (N, A, C, T) whose final root transcript belongs to a class first generated by an encryption query returning (N_e, A_e, C_e, T_e) . Since the final root transcript is the same and bad_{leaf} has not occurred, the final-root determinacy part of Lemma 4 gives $(N, A, C) = (N_e, A_e, C_e)$.

The correct root tag for this transcript is the lazy-sampled value already returned as T_e by the encryption oracle. If $T = T_e$, then (N, A, C, T) is exactly a recorded encryption tuple and the verification is a replay. If $T \neq T_e$, the final tag comparison fails. Thus the submission cannot be both accepting and non-replay. $\square$

Lemma 16 (INT Verification-First Root Guessing). *In the random-oracle INT-CTXT game stopped before $\text{bad}_{\text{key}} \cup \text{bad}_{\text{leaf}}$, let win_{vf} be the event that some accepting non-replay verification submission belongs to a verification-first final root transcript class. Then*

$$\Pr[\text{win}_{\text{vf}}] \leq q_v(\mathcal{B})/2^{\tau_{\text{root}}}.$$

Proof. Work in the game stopped at the first occurrence of bad_{key} or bad_{leaf} . For accounting, define an all-reject continuation: encryption queries are answered normally, replay verification queries accept normally, and every non-replay verification submission performs the same construction computation but returns reject to the adversary. Lazy sampling is deferred for each verification-first class's final root tag value: the continuation records the class address and the submitted tag, but while the class remains verification-first it does not expose that root tag value to the adversary. The execution-uniform cap $q_v(\mathcal{B})$ applies to this continuation.

Fix a verification-first final root transcript class $\mathcal{C}$ and follow the all-reject path until the first later encryption query generating the same class, if any. Let $G_{\mathcal{C}}$ be the set of distinct candidate tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff. Repeated tags only shrink this set. The class's correct root tag $U_{\mathcal{C}}$ is the τ_{root} -bit projection of RO_{flat} at the class's bridged root tag input. Before the stop, no direct query has hit this hidden-key input: such a query would be decoded by $\text{KeyedTWO}_{\text{out}}$ to the hidden key and would trigger bad_{key} . Before the cutoff, no encryption query has revealed $U_{\mathcal{C}}$ as an encryption tag. (When the class's root tag point is $\mathcal{R}_{\text{msg}}^+$, i.e. $n = 0$, an encryption query on a longer message that shares chunk 0 may touch this same address while closing its chunk-0 message phase, but it requests zero output there and continues into its aggregation phase, so it does not reveal $U_{\mathcal{C}}$; its own tag is emitted at a later $\mathcal{R}_{\text{tag}}$ point.) The all-reject path and the set $G_{\mathcal{C}}$ are therefore functions only of the adversary's coins, visible oracle replies, and lazy-table values at inputs other than this class's root tag address. By lazy sampling, conditioned on those values and on the fixed path, $U_{\mathcal{C}}$ may be sampled after $G_{\mathcal{C}}$ is fixed and is uniform independently of $G_{\mathcal{C}}$.

After the cutoff, if it exists, class $\mathcal{C}$ contributes no accepting non-replay submission. Let the cutoff encryption query return (N_e, A_e, C_e, T_e) . Any later verification submission in the same final root transcript class has, by the final-root determinacy part of Lemma 4 and the absence of bad_{leaf} , the same (N, A, C) as that encryption computation. If the submitted tag is T_e , the tuple is a recorded encryption tuple and the verification is a replay; if the submitted tag is not T_e , the final tag comparison fails.

Consider the event that the first accepting non-replay verification submission in the stopped game belongs to class $\mathcal{C}$. Up to that global first accept, the adversary's visible

answers match the all-reject continuation: every earlier non-replay submission rejects, replay queries accept in both executions, and encryption answers are unchanged. The construction computations also agree at every address except that the all-reject continuation defers the final root tag value for class $\mathcal{C}$ until the comparison is needed. Therefore the first accepting non-replay submission can belong to class $\mathcal{C}$ only before the cutoff, and only if $U_{\mathcal{C}} \in G_{\mathcal{C}}$, which has probability at most $|G_{\mathcal{C}}|/2^{\tau_{\text{root}}}$.

Each non-replay verification submission deterministically reconstructs one final root transcript at the root tag output point. If the class is verification-first and the submission occurs before that class's encryption cutoff, the submitted tag contributes to at most one set $G_{\mathcal{C}}$; otherwise it contributes to none. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \leq q_v(\mathcal{B})$, and a union bound over the verification-first classes gives the claim. $\square$

B The Flat-Sponge Lift

This appendix expands the sketch of Section 5.4 into the full argument: it supplies the flat-sponge lift machinery (Lemmas 18 and 19) invoked by the headline theorems of Section 5.1, and the public-permutation CMTDs-3 corollary (Corollary 3) consumed by Section C.

B.1 Simulator Convention

Throughout this appendix, $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ denotes the public-primitive simulator supplied by [BDPVA08, Theorem 2] — the indifferentiability theorem for the simple padded sponge over a random permutation — after applying the [BDPVA11, Theorem 3] bridge of Lemma 5 to TW128's native `pad10*1` transcript inputs. The superscript records that both simulator directions share the same internal lazy RO_{flat} table. This is the simulator referenced by the Sections 4.1 and 4.4 definitions of $\text{Adv}_{\text{TW128}}^{\text{priv}}$, $\text{Adv}_{\text{TW128}}^{\text{ae}}$, and $\text{Adv}_{\text{TW128}}^{\text{mu-ind}}$: the ideal side of each AE-style experiment exposes the construction interface specified by the game (either `Rand` or `(Rand, Replay)`), instantiated per user in `mu-ind`) together with $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ as the public-primitive interface. The RO_{flat} table in this notation is internal to the ideal system and is not an extra interface exposed to the application adversary; the INT-CTXT and committing-security advantages remain the real public-permutation success probabilities of Section 4.1, with the simulator appearing only inside the reduction to the corresponding random-oracle-model adversary.

Lemma 17 (Simulator RO-Call Filter). *For the simulator $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$, every inverse primitive-interface query and every forward primitive-interface query violating one of the following conditions induces no simulator RO_{flat} call. The symbols s , p , $\mathcal{R}$, $\mathcal{O}$, and $\mathcal{C}$ are the local internal notation of the BDPV simple-padded-sponge simulator, not TW128 resource notation:*

(C1) New edge. *The simulator's internal graph has no outgoing edge from the input node s .*

(C2) Unsaturated. $\mathcal{R} \cup \mathcal{O} \neq \mathcal{C}$.

(C3) Rooted input. *The input node is rooted.*

(C4) Paddable path. *The constructed path p decomposes uniquely as $p = p' \parallel 0^{r^j}$ with p' not ending in 0^r , and the prefix p' is a valid padded sponge input: equivalently, $p' = x \parallel \text{pad10}^*[r](|x|)$ for an unpadded BDPV08 random-oracle input x .*

When all four conditions hold, the forward primitive-interface query induces one simulator RO_{flat} call. Consequently, each primitive-interface query induces at most one simulator-side RO_{flat} call.

Proof. This is the branch structure of the [BDPVA08] simple-padded permutation simulator. The simulator invokes the random oracle only on the forward-query branch where the input node is rooted, unsaturated, paddable to an unpadded random-oracle input x , and not already extended by an outgoing edge in the simulator graph. Forward queries violating any of these conditions and all inverse queries return outputs sampled or read from the simulator's internal graph without invoking the random oracle. Under the [BDPVA11, Theorem 3] bridge fixed in Section B.1, that simulator random-oracle invocation is the simulator-side RO_{flat} call used throughout this appendix. $\square$

Lemma 18 (Flat-Sponge Lift Bound). *Let G be any of the single-stage TW128 games of Sections 4.1 and 4.4: privacy, INT-CTXT, all-in-one AE, mu-ind, CMTDs-3, or CMTs- ℓ for $\ell \in \{1, 3, 4\}$. Let $\mathcal{A}$ be a public-permutation G adversary with construction-interface cost at most N and at most N_{prim} direct primitive-interface queries. In the [BDPVA08] single-stage replacement, the combined construction-and-primitive transcript has length at most*

$$N_{\text{tot}} = N + N_{\text{prim}}.$$

Consequently replacing the real public permutation and TW128 construction by the simulator-world primitive interface $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ and the corresponding flat-RO construction interface costs at most $\text{Lift}_c(N_{\text{tot}})$.

Proof. The [BDPVA08] replacement is applied before the derived-adversary construction, so its transcript contains two kinds of public-primitive activity. First, each construction-interface computation contributes the TW128 duplex calls it performs. By Lemmas 3 and 5, each such root or leaf output point is a typed restriction of the bridged flat-sponge construction, and by the definition of N in Section 4.3 these calls are counted at the same per-block granularity as the underlying KECCAK – $p[1600, 12]$ calls. This includes verification computations whether they accept or reject, the per-user sum of construction calls in mu-ind, and the deterministic challenger decryption or encryption checks actually performed in the committing games.

Second, each direct adversary query to (π, π^{-1}) contributes one primitive-interface query and is counted in N_{prim} . These two counts are disjoint in the public-permutation experiment: N counts construction-internal use of the primitive, while N_{prim} counts only the adversary's public-primitive interface queries. The execution-uniform caps ensure that every adaptive transcript seen by the single-stage replacement has length at most $N + N_{\text{prim}} = N_{\text{tot}}$. If $N_{\text{tot}} \geq 2^c$ then $\text{Lift}_c(N_{\text{tot}}) \geq 1$ and the claimed bound holds trivially, so assume $N_{\text{tot}} < 2^c$, satisfying the hypothesis of [BDPVA08, Theorem 2]. That theorem gives distinguishing or success-probability cost at most the exact random-permutation differentiating probability $f_P(N_{\text{tot}})$ of [BDPVA08, Lemma 5], which Section 5.2 bounds by $\text{Lift}_c(N_{\text{tot}})$. $\square$

Lemma 19 (Derived-Adversary Execution Contract). *Let G be any of the single-stage TW128 games of Sections 4.1 and 4.4: privacy, INT-CTXT, all-in-one AE, mu-ind, CMTDs-3, or CMTs- ℓ for $\ell \in \{1, 3, 4\}$. Let $\mathcal{A}$ be a simulator-world G adversary whose public-primitive interface is answered by $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$, with at most N_{prim} direct primitive-interface queries. Define $\mathcal{B}_{\text{derived}}(\mathcal{A})$ to run $\mathcal{A}$ internally, answer each primitive-interface query by running the same simulator against direct finite-output queries to its own RO_{flat} oracle, and forward construction-interface queries (or, in the committing games, the final structured output) unchanged to the corresponding random-oracle experiment. Then the simulator-world execution of $\mathcal{A}$ and the random-oracle execution of $\mathcal{B}_{\text{derived}}(\mathcal{A})$ have identical joint distributions, and*

$$q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}.$$

2269 In this coupling, simulator-induced RO_{flat} calls and construction-internal RF lookups use
 2270 the same lazy table. Therefore, if a simulator-induced query is the bridged input $\text{flat}(X)$ of
 2271 a well-formed TW128 transcript prefix X and a forwarded construction computation later
 2272 reaches the same prefix X , both interfaces read the same $\text{RO}_{\text{flat}}(\text{flat}(X))$ value, projected to
 2273 the requested output length. Moreover, $\mathcal{B}_{\text{derived}}$ inherits the construction-side caps of $\mathcal{A}$:

$$\begin{aligned} 2274 \quad q_v(\mathcal{B}_{\text{derived}}) &= q_v(\mathcal{A}) \quad (\text{INT-CTXT}, \text{AE}, \text{mu-ind}), \\ 2275 \quad n_{\text{leaf}}(\mathcal{B}_{\text{derived}}) &\leq n_{\text{leaf}}(\mathcal{A}) \quad (\text{INT-CTXT}, \text{AE}, \text{mu-ind}). \end{aligned}$$

2276 In the *mu-ind* game these resource inheritances hold per user, and the **New-query** transcript
 2277 is unchanged.

2278 *Proof.* Couple the two executions by using the same lazy RO_{flat} table for the simulator
 2279 and for all construction-internal $\text{RF}(\cdot)$ lookups. In the simulator-world execution, $\mathcal{A}$ sees
 2280 construction answers from the selected game interface and public-primitive answers from
 2281 $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$. In the derived random-oracle execution, $\mathcal{B}_{\text{derived}}$ runs the same $\mathcal{A}$,
 2282 forwards construction-interface queries (or the final committing-game output) unchanged,
 2283 and answers primitive-interface queries by invoking the same simulator code with the same
 2284 lazy RO_{flat} table. The construction interface or committing challenger therefore receives the
 2285 same forwarded inputs, the simulator receives the same primitive queries in the same order,
 2286 and the lazy table supplies the same values. The full views of $\mathcal{A}$ in the simulator-world
 2287 execution and of $\mathcal{B}_{\text{derived}}$ in the random-oracle execution are identically distributed.

2288 The simulator's random-oracle calls may be on arbitrary [BDPVA08] H -input strings,
 2289 not only bridged TW128 transcript inputs. These calls are the direct RO_{flat} queries of
 2290 $\mathcal{B}_{\text{derived}}$ and are counted in $q_{\text{RO}}(\mathcal{B}_{\text{derived}})$. Construction-internal $\text{RF}(\cdot)$ lookups made by
 2291 Enc_K^{RO} , Dec_K^{RO} , Ver_K^{RO} , or a committing challenger are forwarded to the same RO_{flat} table
 2292 but are not counted in q_{RO} , exactly as in Section 4.3.

2293 By Lemma 17, each primitive-interface query induces at most one direct RO_{flat} query
 2294 by $\mathcal{B}_{\text{derived}}$, and inverse queries or non-RO-touching forward queries induce none. Since $\mathcal{A}$
 2295 makes at most N_{prim} direct primitive-interface queries, this gives $q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}$.

2296 Lemma 5's bridge places TW128 construction restrictions and simulator-induced RO_{flat}
 2297 calls in a single lazy table. If the simulator invokes RO_{flat} on $\text{flat}(X)$ for a well-formed
 2298 TW128 transcript prefix X , and a forwarded construction computation later reaches X ,
 2299 the construction lookup $\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X))$ reads the same table entry and returns
 2300 the requested projection. Since TW128 uses $r = r_{\text{max}}$, the [BDPVA11, Theorem 3] output
 2301 post-processing does not alter TW128 output bits. Thus simulator-induced direct queries
 2302 and construction-internal lookups at the same bridged input are transcript-consistent. In
 2303 the AE games, a simulator call whose input is accepted by **KeyedTWOut** for the hidden
 2304 key is still a direct RO_{flat} query and is covered by the same bad_{key} accounting used by the
 2305 random-oracle privacy and INT-CTXT theorems. The derivation step itself does not prove
 2306 a key-guessing statement; it only supplies the derived adversary and the query bound at
 2307 which the random-oracle envelopes of Section 5.3 are instantiated.

2308 Finally, the execution-uniform caps of Section 4.3 hold along every oracle-answer trace
 2309 of $\mathcal{A}$, including traces generated by the simulator on the ideal public-primitive interface.
 2310 The derived-adversary construction does not add construction-interface encryption or
 2311 verification queries and forwards $\mathcal{A}$'s construction-interface queries unchanged along each
 2312 trace. $\mathcal{B}_{\text{derived}}$ therefore inherits the relevant construction resources: $q_v(\mathcal{B}_{\text{derived}}) = q_v(\mathcal{A})$
 2313 in the INT-CTXT, AE, and mu-ind games, and $n_{\text{leaf}}(\mathcal{B}_{\text{derived}}) \leq n_{\text{leaf}}(\mathcal{A})$ for the dis-
 2314 tinct construction-generated final-leaf-transcript cap. In mu-ind, construction-interface
 2315 forwarding preserves the **New** calls and these caps user by user. $\square$

B.2 Public-Permutation Committing Corollary

Combining the lift bound of Lemma 18, the derived-adversary construction of Lemma 19, and the random-oracle CMTDs-3 bound of Theorem 6 gives the public-permutation committing corollary consumed by Section C. The public-permutation privacy and INT-CTXT advantages are not stated as separate corollaries: the all-in-one AE theorem (Theorem 1) applies this lift once to the all-in-one AE adversary and then splits the resulting random-oracle advantage into the privacy and INT-CTXT envelopes of Section 5.3, adding $\text{Lift}_c(N_{\text{tot}})$ a single time rather than the two copies that lifting privacy and INT-CTXT separately would incur.

Corollary 3 (Public-Permutation CMTDs-3). *For every $s \geq 2$ and every s -way CMTDs-3 adversary $\mathcal{A}$ against TW128 with N_{prim} direct primitive queries and total cost at most N_{tot} (both as in Section 4.3),*

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s) = \text{Lift}_c(N_{\text{tot}}) + \frac{\binom{N_{\text{prim}} + s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

Proof. The CMTDs-3 game is a win-event experiment with no construction oracle during the adversary's run. Its construction cost N counts the deterministic decryption checks performed by the challenger after $\mathcal{A}$ outputs its candidate ciphertext and opening tuples; on executions that pass the early rejecting checks, there are s such checks. By the flat-sponge lift bound (Lemma 18), replacing (π, π^{-1}) by $(S^{\text{RO}_{\text{flat}}}, S^{-1, \text{RO}_{\text{flat}}})$ and the challenger checks by their random-oracle counterparts costs at most $\text{Lift}_c(N_{\text{tot}})$. Let $\mathcal{B}_{\text{derived}} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ be the adversary of Lemma 19. The derived-adversary construction gives

$$\text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\text{CMTDs-3}}(\mathcal{B}_{\text{derived}}),$$

with $q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}$. At $\mathcal{B}_{\text{derived}}$, Theorem 6 gives

$$\text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}_{\text{derived}}), s).$$

By monotonicity of $\binom{\cdot + s}{s}$ in its first argument, this is at most $\text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s)$. $\square$

C Committing-Notion Bounds

This appendix establishes the committing-security bounds beyond the random-oracle CMTDs-3 theorem of Theorem 6. The D-notions reduce to CMTDs-3 by success-preserving forwarding. The E-notions are bounded directly by the same final-root candidate-collision argument, with the challenger encryption checks counted as construction work in N .

Lemma 20 (Forwarding Reductions). *Fix $s \geq 2$. Let X be either CMTDs-3 or CMTDs-4. For every s -way X -adversary $\mathcal{A}$ against TW128, the forwarding reduction $\mathcal{A}^* = \mathcal{A}$ is an s -way CMTDs-3 adversary satisfying*

$$\begin{aligned} \text{Adv}_{\text{TW128}}^X(\mathcal{A}) &\leq \text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}^*), \\ N_{\text{prim}}(\mathcal{A}^*) &= N_{\text{prim}}(\mathcal{A}), \\ N_{\text{tot}}(\mathcal{A}^*) &\leq N_{\text{tot}}(\mathcal{A}). \end{aligned}$$

Proof. The CMTDs-3 case is the identity game. A CMTDs-4 winner is also a CMTDs-3 winner. Indeed, if a winning transcript had $(K_i, N_i, A_i) = (K_j, N_j, A_j)$ for some $i \neq j$, determinism of Dec would give $M_i = \text{Dec}_{K_i}(N_i, A_i, C \| T) = M_j$, contradicting the pairwise distinctness of the full tuples (K_i, N_i, A_i, M_i) . Thus a CMTDs-4 winner also has pairwise distinct triples and is a CMTDs-3 winner on the same output. The forwarding reduction

performs no additional primitive-interface queries. In the CMTDs-4 case, if the target CMTDs-3 game reaches its deterministic decryption checks, the source CMTDs-4 game also reaches deterministic decryption checks on the same ciphertext and tuples; otherwise the target contributes no construction work. Hence the target construction cost is at most the source construction cost. $\square$

Lemma 21 (CMTDs-1 Key-Precheck Reduction). *Fix $s \geq 2$. For every s -way CMTDs-1 adversary $\mathcal{A}$ against TW128, define $\mathcal{A}^*$ as follows. It runs $\mathcal{A}$ to obtain $C \parallel T$ and tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$. If the tuples fail the CMTDs-1 syntax, length, or pairwise-WiC₁ checks, then $\mathcal{A}^*$ outputs a fixed losing CMTDs-3 instance rejected before any decryption check. Otherwise, it forwards the same $(C \parallel T, (K_i, N_i, A_i, M_i)_{i=1}^s)$ to the CMTDs-3 challenger. Then*

$$\begin{aligned} \text{Adv}_{\text{TW128}}^{\text{CMTDs-1}}(\mathcal{A}) &\leq \text{Adv}_{\text{TW128}}^{\text{CMTDs-3}}(\mathcal{A}^*), \\ N_{\text{prim}}(\mathcal{A}^*) &= N_{\text{prim}}(\mathcal{A}), \\ N_{\text{tot}}(\mathcal{A}^*) &\leq N_{\text{tot}}(\mathcal{A}). \end{aligned}$$

Proof. The syntax and pairwise-WiC₁ prechecks are source-game rejection checks, and the eager length precheck is success preserving because TW128 decryption returns either $\perp$ or a message of length $\|C\|$. Thus no winning CMTDs-1 transcript is discarded. On the forwarding branch, pairwise WiC₁-distinctness means the keys K_i are pairwise distinct, hence the triples (K_i, N_i, A_i) are pairwise distinct. Thus every source winner forwarded by $\mathcal{A}^*$ satisfies the target CMTDs-3 syntax, length, and triple-distinctness checks, and the target decryption checks are the same deterministic checks as in the source game.

The reduction performs no additional primitive-interface queries. On the fixed-losing branch, both the source game and target reduction reject before construction work. On the forwarding branch, the target decryption checks coincide with the source decryption checks. Therefore the target construction cost is at most the source construction cost. $\square$

Lemma 22 (Random-Oracle CMTs). *Fix $s \geq 2$ and $\ell \in \{1, 3, 4\}$. For every s -way CMTs- ℓ adversary $\mathcal{B}$ in the TW128 random-oracle model,*

$$\text{Adv}_{\text{RO}}^{\text{CMTs-}\ell}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s) = \frac{(q_{\text{RO}}(\mathcal{B})+s)}{2^{\tau_{\text{root}}(s-1)}}.$$

Proof. Run the random-oracle CMTs- ℓ game: after $\mathcal{B}$'s direct RO_{flat} queries, the challenger checks syntax and pairwise WiC _{ℓ} -distinctness of the submitted tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$, computes

$$C_i \parallel T_i = \text{Enc}_{K_i}^{\text{RO}}(N_i, A_i, M_i),$$

and accepts iff all $C_i \parallel T_i$ are equal.

Build the same candidate sequence as in Theorem 6. During $\mathcal{B}$'s direct oracle phase, append every distinct direct-query input whose decoded class is a root tag output-point class — $\mathcal{R}_{\text{msg}}^+$ (the single-chunk root tag, with the aggregation phase elided when $n = 0$) or $\mathcal{R}_{\text{tag}}$ (the multi-chunk root tag) — immediately before the corresponding direct RO_{flat} lookup first samples that input. This predicate depends on the queried input alone through the exact decoder of Lemma 5, and by Corollary 2 each input decodes to at most one output-point class. Since there is no construction oracle before $\mathcal{B}$ outputs its tuples, these candidates are first touched at their append steps.

After the tuple output, stop appending challenger candidates if the syntax or pairwise-WiC _{ℓ} check fails. Otherwise, as the challenger computes the s deterministic encryption checks, let R_i be the final root transcript reached by the i -th encryption and append $\text{flat}(R_i)$ immediately before the root tag oracle answer is sampled, unless that input is already in the sequence. If such an input was already touched by a direct query, then

Lemma 5 and Corollary 2 imply that the earlier query was a final-root candidate. Earlier challenger lookups outside the final-root class are separated by Corollary 2. An earlier challenger final-root lookup with a different address does not affect the new candidate. An earlier challenger final-root lookup with the same address is either a root tag answer sampling, in which case the address was already appended and is not appended again, or the chunk-0-closing σ_{msg}^+ call of an $n \geq 1$ encryption check that reaches the same $\mathcal{R}_{\text{msg}}^+$ address (possible only for $\ell = 4$, where two checks may share (K, N, A) and the chunk-0 ciphertext); that call requests zero output bits and continues into its aggregation phase, so it reveals nothing about RO_{flat} at the address and is permitted by the unrevealed-value premise. In all cases each newly appended challenger candidate satisfies the predictability and unrevealed-value premises of Lemma 6, and the sequence length is at most $q_{\text{RO}}(\mathcal{B}) + s$.

On a winning execution, the accepted encryptions are all equal. Hence their ciphertext-body lengths are equal. Pairwise triple distinctness follows from the winning predicates: for $\ell \in \{1, 3\}$ it follows directly from pairwise WiC_ℓ -distinctness, and for $\ell = 4$ equal triples would force the equal encryption value to decrypt, by correctness and determinism, to a single message $M_i = M_j$, so the full tuples (K_i, N_i, A_i, M_i) and (K_j, N_j, A_j, M_j) would coincide, contradicting pairwise WiC_4 -distinctness. Therefore Corollary 1 makes the bridged final-root inputs

$$\text{flat}(R_1), \dots, \text{flat}(R_s)$$

pairwise distinct. Since all encryptions are equal, their root tags are the same value T , so the τ_{root} -bit projections at these s distinct final-root candidates are all equal to T . The win event is contained in the s -way candidate-collision event, and Lemma 6 with $Q = q_{\text{RO}}(\mathcal{B})$ and $\tau = \tau_{\text{root}}$ gives the claim. $\square$

Corollary 4 (Public-Permutation CMTs). *Fix $s \geq 2$ and $\ell \in \{1, 3, 4\}$. For every s -way CMTs- ℓ adversary $\mathcal{A}$ against TW128 with N_{prim} direct primitive queries and total cost at most N_{tot} ,*

$$\text{Adv}_{\text{TW128}}^{\text{CMTs-}\ell}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s).$$

Proof. The CMTs- ℓ game is a win-event experiment with no construction oracle during the adversary's run. Its construction cost N counts the deterministic encryption checks performed by the challenger after $\mathcal{A}$ outputs its tuples. By the flat-sponge lift bound (Lemma 18), replacing (π, π^{-1}) by $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ and the challenger encryption checks by their random-oracle counterparts costs at most $\text{Lift}_c(N_{\text{tot}})$. Let $\mathcal{B}_{\text{derived}} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ be the adversary of Lemma 19. The derived-adversary construction gives

$$\text{Adv}_{\text{TW128}}^{\text{CMTs-}\ell}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\text{CMTs-}\ell}(\mathcal{B}_{\text{derived}}),$$

with $q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}$. Apply Lemma 22 and use monotonicity of $\text{RootColl}_{\tau_{\text{root}}}(\cdot, s)$. $\square$

Combined bound. Combining the D-notion forwarding lemmas above with Corollary 3, and the E-notion bound of Corollary 4, gives the same root-collision envelope for all six committing notions of [BH22]: for every

$$X \in \{\text{CMTDs-1}, \text{CMTDs-3}, \text{CMTDs-4}, \text{CMTs-1}, \text{CMTs-3}, \text{CMTs-4}\}$$

and every s -way X -adversary $\mathcal{A}$ against TW128,

$$\text{Adv}_{\text{TW128}}^X(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}(\mathcal{A})) + \frac{\binom{N_{\text{prim}}(\mathcal{A}) + s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

For the D-notions, the forwarding or key-precheck reduction performs no additional primitive-interface queries and no additional construction work beyond the source-game decryption checks, so $N_{\text{prim}}(\mathcal{A}^*) = N_{\text{prim}}(\mathcal{A})$ and $N_{\text{tot}}(\mathcal{A}^*) \leq N_{\text{tot}}(\mathcal{A})$; the latter enters the displayed Lift_c term through the monotonicity of Lift_c in its argument. For the E-notions, no reduction is used: the challenger encryption checks are the source-game construction work already counted in N by Section 4.3.

D Test Vectors

The following test vectors cover the corner cases of TW128 encryption, as specified in § 3.

All byte strings are written in hexadecimal. For each vector we list the key K , nonce N , associated data A , message M , ciphertext C , and tag T ; the sealed output of encryption is $C \parallel T$. The empty string is written ε . The leaf index is encoded as a little-endian 64-bit integer and the phase suffixes are appended least-significant-bit first.

To keep the longer vectors compact, any value exceeding 32 bytes is given by its length in bytes together with its SHA-256 digest rather than in full. Every message and associated-data byte is defined by $m_i = (17i + 31) \bmod 256$ for $i = 0, 1, \dots$, so each such input is determined by its length alone and can be reconstructed and checked against the stated digest; a ciphertext given by its length and digest can likewise be checked by encrypting the corresponding message and hashing the result.

Vector 1. Empty associated data and empty message: the associated-data and aggregation phases are both elided, and the root tag is emitted by the single closing message call with no leaves.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
N	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
A	ε
M	ε
C	ε
T	da65bbda0b20b1e936f5326458166633d1e5a3f7aebc0b318d24b051a4efa697

Vector 2. Multi-block associated data with an empty message: the associated data spans two rate blocks and no message bytes are processed.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
N	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
A	168 bytes, SHA-256 4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a
M	ε
C	ε
T	2093878418f3fca96ad3a8d1748a9be4d2a8b518e917d988fd703dcb7bf72173

Vector 3. Empty associated data and a single-block message: the root message phase only, no leaves.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
N	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
A	ε
M	1f30415263748596a7b8c9daebfc0d1e2f405162738495a6b7c8d9eafb0c1d2e
C	676a1441e20daea0f1ee4571ee4543842c946d59e0d3df6f1dd2f92dc4f876aa
T	1873332d359ff2fe5197488224af7bf6ad821875fbd964100de424aa77342be

Vector 4. Empty associated data and a two-block message: the root keystream is chained across two rate blocks.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
N	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
A	ε
M	168 bytes, SHA-256 4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a
C	168 bytes, SHA-256 df3f0a98a91a69ed474edbc52e0dc1d8e54bfd6fc514966ce7b878d48be0bf9a
T	aab2413074a202cfadd5eb784663b1831eb330025972d163d03e9ed5a964b765

Vector 5. A message of exactly one chunk: the root is full, there is still no leaf, and the aggregation phase is elided so the closing root message call emits the tag.

2466

<i>K</i>	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
<i>N</i>	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
<i>A</i>	ε
<i>M</i>	8183 bytes, SHA-256 6532a370a4ab6b5f4094dd8a6016512ab8e8416abe910bc1a0b532979224151d
<i>C</i>	8183 bytes, SHA-256 8d58f8aa3413ea0ba1b8cc3ed4e9a334521d8dfe9694a0661fb8251049907e9
<i>T</i>	c2201637f1116c1303b8982622756770d23acb85384f4658fcb63c37e4b629dd

Vector 6. A message of one chunk and one further byte: one root chunk and one leaf, exercising leaf tag aggregation.

2467

<i>K</i>	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
<i>N</i>	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
<i>A</i>	ε
<i>M</i>	8184 bytes, SHA-256 c4c383bca0808e2ce44d481ddfe56efdc7a147dcd5a14c898ef5423f2604cd7
<i>C</i>	8184 bytes, SHA-256 ef1b2f7607ade503ab497fc61e62cef2b44c6328fa06e499718eb7da5fdd04d3
<i>T</i>	c96bea48e07a8431fd19721ffbe8afe6a73047f96d953e3281f10126eeca06a1

Vector 7. A multi-leaf message of two chunks and one further byte: the full tree path with two leaves.

2468

<i>K</i>	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
<i>N</i>	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
<i>A</i>	ε
<i>M</i>	16367 bytes, SHA-256 17e31f3700d0602aab8a686024ecc05b8192e80a531483c67dd90d5a26557c5e
<i>C</i>	16367 bytes, SHA-256 6d83d8fbdca7c2ef590d29a36527a45019e50d1f8c8db037959081685a46908
<i>T</i>	cdc2e50ac5380419f06772a2f3af2146050bd0340b35f61b67f839351767804e

Vector 8. Maximum associated data with a short message: the associated-data blocks dominate the cost.

2469

<i>K</i>	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
<i>N</i>	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
<i>A</i>	337 bytes, SHA-256 04ca7b24930bc6b5fe8b95ff1eb0933907657c1283cc3a7e69c5f7f02220f281
<i>M</i>	1f30415263748596a7b8c9daebfc0d1e
<i>C</i>	0dd615a0a05ed82a78c3999b9e80cbed
<i>T</i>	22e4e6725f548ad0660041d66cf2f18dc45c69b447cdaae1f147d33a3b6aa1a9

Vector 9. An all-ones key and nonce with a multi-chunk message.

2470

<i>K</i>	ff
<i>N</i>	ff
<i>A</i>	168 bytes, SHA-256 4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a
<i>M</i>	8184 bytes, SHA-256 c4c383bca0808e2ce44d481ddfe56efdc7a147dcd5a14c898ef5423f2604cd7
<i>C</i>	8184 bytes, SHA-256 49818c320ab282296639798d840cbd60751e1623f449dcfb9423371d44145dfb
<i>T</i>	310c9fff20e9651f8ed47eed9bb182a086ed1af1b3f1799e35307a4139865fc7
